

An intuitive interface for a class of sophisticated interpolating curves

A report submitted to The University of Manchester for the degree of
Bachelor of Science in Computer Science and Mathematics
in the Faculty of Science and Engineering

Year of submission: 2025

Author: Dillon Morrey

Supervisor: Dr Joseph Razavi

Department of Computer Science

Contents

Abstract **3**

Declaration of Originality **4**

Intellectual Property Statement **5**

Acknowledgements **6**

Abbreviations **7**

Notations **8**

Terminology **9**

1 Introduction **13**
1.1 Context and Motivation . . . 13
1.2 Project Aims . . . 15
1.3 Report structure . . . 16

2 Background **17**
2.1 Bézier curves . . . 17
2.1.1 Construction of Bézier curves . . . 17
2.1.2 The osculating circle . . . 19
2.2 The construction of κ-curves . . . 21
2.2.1 Introduction . . . 21
2.2.2 Notation . . . 21
2.2.3 Satisfying G^2 - continuity . . . 23
2.2.4 Maximum curvature condition . . . 23
2.2.5 Solving for $\tilde{\lambda}_i$ . . . 24
2.2.6 Off curve points . . . 24
2.3 The construction of $\epsilon\kappa$-curves . . . 26
2.3.1 Introduction . . . 26
2.3.2 Notation . . . 26
2.3.3 The Bernstein Polynomial . . . 26
2.3.4 Governing equations . . . 27
2.4 Existing Systems . . . 28
2.4.1 Summary . . . 28
2.4.2 Advantages and Disadvantages . . . 29
2.4.3 Where does this project aim to improve? . . . 29

3 Implementation **31**
3.1 Development Pipeline . . . 31
3.2 Software Requirements and Libraries . . . 31
3.3 Creating the Tkinter GUI . . . 32
3.4 Bézier Curves . . . 33
3.4.1 Key variables, Data Structures and Functions . . . 33
3.4.2 Construction of Bézier curves . . . 33
3.4.3 Construction of the osculating circle . . . 35
3.4.4 Dotting the curve . . . 40
3.4.5 Additional features . . . 43
3.5 κ-Curves . . . 44
3.5.1 Key variables, Data Structures and Functions . . . 44
3.5.2 Overview . . . 44
3.5.3 Solving for $\tilde{t}_i$ . . . 45
3.5.4 Solving for $\tilde{\lambda}_i$ . . . 46
3.5.5 Solving for $\tilde{\mathbf{Q}}_{i,1}$ . . . 46
3.5.6 The iterative process . . . 48
3.5.7 Displaying the κ-curve . . . 49

3.6 $\epsilon\kappa$-Curves . . . 51
3.6.1 Key variables, Data Structures and Functions . . . 51
3.6.2 Overview . . . 52
3.6.3 Solving for $\hat{t}_i$ . . . 52
3.6.4 Solving for $\hat{\lambda}_i$ . . . 55
3.6.5 Solving for $\hat{\mathbf{Q}}_{i,j}$ . . . 55
3.6.6 The iterative process . . . 57
3.6.7 Adjusting α intuitively . . . 60
3.7 Testing . . . 62
3.7.1 Testing correctness . . . 62
3.7.2 Investing the intuitiveness of adjusting α-values for $\epsilon\kappa$-curves . . . 62

4 Results and Evaluation **63**
4.1 Testing for correctness . . . 63
4.2 Investigating the intuitiveness of adjusting α-values for $\epsilon\kappa$-curves . . . 69
4.2.1 Post user testing changes . . . 71
4.3 Comparison of the $\epsilon\kappa$-curves system versus similar systems . . . 72

5 Conclusion **73**
5.1 Summary of Results . . . 73
5.2 Achievements . . . 73
5.3 Improvements to existing system . . . 74
5.4 Further Work . . . 75

Abstract

κ-curves are a relatively new technology that consist of interpolating quadratic Bézier curves and ensure that local curvature maximums pass through user defined control points. More recently, the formulation of $\epsilon\kappa$-curves has been introduced; an extension of the κ-curve that allows the user to adjust the curvature around extremums. However, very few works have been published around this topic, and the work that has leaves some areas for exploration. Specifically, in this report, we focus on more intuitive controls for adjusting curvature around control points. Whilst other existing systems perform better in generating $\epsilon\kappa$-curves, the implementation in this report introduces prototype tools to adjust curvature inspired by common ways of visualising curvature, namely the osculating circle. To analyse the effectiveness of this tool, the system was tested by novice users. Qualitative feedback was gathered from participants to evaluate the system's performance and provide ideas for future improvements. This feedback implied that users had more control in fine tuning the curve than other systems, and that the experience in changing curvature would feel more intuitive if certain issues were fixed. These results highlight areas for refinement, suggesting future work should include improving the responsiveness of the system and carefully considering scenarios where the osculating circle is small.

Declaration of Originality

I hereby confirm that this report is my own original work unless referenced clearly to the contrary, and that no portion of the work referred to in the report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Intellectual Property Statement

i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made only in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii. The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420), in any relevant Dissertation restriction declarations deposited in the University Library, and The University Library's regulations (see http://www.library.manchester.ac.uk/about/regulations).

Acknowledgements

Firstly, I would like to thank my supervisor, Dr. Joseph Razavi. Your passion for this topic and willingness to teach have fuelled many intriguing conversations that I will surely miss. Our discussions have been a powerful driving force behind the work presented in this report. Thank you.

To my friends and family, thank you for your constant support throughout this journey. Speaking to you about this project—despite the occasional blank faces—provided encouragement and helped me more than you may realise.

Abbreviations

CAD	Computer Aided Design
lerp	linear interpolation
GUI	graphical user interface

Notations

$\mathbb{R}^2$	2-dimensional Euclidean Space
n	order of Bézier curve
${}^nC_i$	the binomial coefficient
$det(\mathbf{a}, \mathbf{b})$	the determinant of a matrix formed by the columns of $\mathbf{a}$ and $\mathbf{b}$
$\|\mathbf{a}\|$	the norm or length of $\mathbf{a}$
$\kappa(t)$	the curvature of a curve at t (See Definition 2.1.3)
$\dot{\mathbf{a}}$	first derivative of $\mathbf{a}$ with respect to t
$\ddot{\mathbf{a}}$	second derivative of $\mathbf{a}$ with respect to t
$\tilde{n}$	Associated with κ-curves, where $\tilde{n}$ = number of control points -2
$\tilde{\mathbf{P}}_i$	κ-curve user control point indexed at i, $i \in \{0, 1, ..., \tilde{n}+1\}$
$\tilde{t}_i$	κ-curve t parameter indexed at i, $i \in \{1, 2, ..., \tilde{n}\}$
$\tilde{\lambda}_i$	κ-curve $\tilde{\lambda}$ parameter indexed at i, $i \in \{1, 2, ..., \tilde{n}-1\}$
$\tilde{\mathbf{Q}}_{i,j}$	the jth κ-curve calculated control point for the ith Bézier segment, $i \in \{0, 1, ..., \tilde{n}\}$, $j \in \{0, 1, 2\}$
$\Delta(\mathbf{A}, \mathbf{B}, \mathbf{C})$	area of triangle with vertices $\mathbf{A}, \mathbf{B}, \mathbf{C}$
$\hat{n}$	Associated with $\epsilon\kappa$-curves, where $\hat{n}$ = number of control points -2
$\hat{\mathbf{P}}_i$	$\epsilon\kappa$-curve user control point indexed at i, $i \in \{0, 1, ..., \hat{n}+1\}$
$\hat{t}_i$	$\epsilon\kappa$-curve t parameter indexed at i, $i \in \{1, 2, ..., \hat{n}\}$
$\hat{\lambda}_i$	$\epsilon\kappa$-curve $\hat{\lambda}$ parameter indexed at i, $i \in \{1, 2, ..., \hat{n}-1\}$
$\hat{\mathbf{Q}}_{i,j}$	the jth κ-curve calculated control point for the ith Bézier segment, $i \in \{0, 1, ..., \hat{n}\}$, $j \in \{0, 1, 2\}$
$\mathcal{L}_i$	left basis for the ith cubic Bézier defined in equation (2.33)
$\mathcal{M}_i$	middle basis for the ith cubic Bézier defined in equation (2.34)
$\mathcal{R}_i$	right basis for the ith cubic Bézier defined in equation (2.35)

Terminology

Convex Hull Property	A curve S is said to have the conex hull property if S is entirely contained within the convex hull of it's control points
G^0-continuity	Two curve segments S_1 and S_2 are G^0-continuous if one of the endpoints of S_1 is the also one of the end points of S_2
G^1-continuity	Two curve segments S_1 and S_2 are G^1-continuous if they are G^0-continuous and their tangent directions are equal at the point at which they meet
G^2-continuity	Two curve segments S_1 and S_2 are G^2-continuous if they are G^1-continuous and have the same curvature at the point at which they meet
Quadratic Bézier Curve	A Bézier Curve defined by three control points
Cubic Bézier Curve	A Bézier Curve defined by four control points

List of Tables

1 Table of goals and success criteria 15
2 Table of advantages and disadvantages of Salvi's [24] implementation of $\epsilon\kappa$-curves 29
3 Key variables and data structures and their mathematical representations in the Bézier system. 33
4 Key functions used in the Bézier system. 33
5 Key variables and data structures and their mathematical representations in the κ system. 44
6 Key functions used in the κ system. 45
7 Key variables and data structures and their mathematical representations in the $\epsilon\kappa$ system. 51
8 Key functions used in the $\epsilon\kappa$ system. 52
9 Table of achievements 74

List of Figures

1.1 Salvi's [24] implementation of $\epsilon\kappa$-curves, $\alpha = 2/3$ . . . 13
1.2 Salvi's [24] implementation of $\epsilon\kappa$-curves, $\alpha = 0.95$ . . . 14
2.1 An example of a two lerps between three control points . . . 17
2.2 A Bézier curve (blue) drawn by $\mathbf{P}_6$ . . . 18
2.3 The osculating circle (orange) on the Bézier curve (blue) . . . 20
2.4 The osculating circle (orange) at a different point on the Bézier curve (blue) . . . 20
2.5 Bézier curve (left) versus κ-curve (right) . . . 21
2.6 κ-curve showing labelling of control points . . . 22
2.7 κ-curve showing $\tilde{\mathbf{Q}}$ points (black boxes) and $\tilde{\mathbf{P}}$ points (red circles) . . . 22
2.8 κ-curves and their $\tilde{\mathbf{Q}}$-points . . . 25
2.9 Demonstrating internal division ratio α. Adapted from Miura et al. (2021), licensed under CC BY 4.0 (see http://creativecommons.org/licenses/by/4.0/). . . . 27
3.1 Bézier curve with 5 control points . . . 34
3.2 Bézier curve with unintended cusp . . . 35
3.3 Osculating circle at $t = 0.31$ . . . 37
3.4 Osculating circle at $t = 0.76$ . . . 38
3.5 Osculating circle with incorrect normals . . . 39
3.6 Dots placed on curve at equal intervals of t . . . 40
3.7 Bézier curve with 30 dots . . . 41
3.8 Bézier curve with 50 dots . . . 42
3.9 Bézier curve with all additional features . . . 43
3.10 Bézier curve with no additional features . . . 44
3.11 κ-curve with all additional features . . . 49
3.12 κ-curve with no additional features . . . 50
3.13 Maxima code for generating polynomial coefficients. Image adapted from Miura et al. [22] for clarity; code by Salvi [25]. Licensed under CC BY 4.0 (see http://creativecommons.org/licenses/by/4.0/). . . . 53
3.14 $\epsilon\kappa$-curve with `alpha_array=[0.66,0.66,0.66]` . . . 58
3.15 $\epsilon\kappa$-curve with `alpha_array=[0.87,0.94,0.74]` . . . 59
3.16 $\epsilon\kappa$-curve with prototype tool, example 1 . . . 60
3.17 $\epsilon\kappa$-curve with prototype tool, example 2 . . . 61
4.1 Salvi's implementation [24], 5 control points, $\alpha = 0.66$ . . . 63
4.2 Python implementation, 5 control points, $\alpha = 0.66$ . . . 64
4.3 Python implementation, 5 control points, $\alpha = 0.66$, 400 iterations . . . 65
4.4 Salvi's implementation [24], 5 control points, $\alpha = 0.9$ . . . 66
4.5 Python implementation, 5 control points, $\alpha = 0.9$ . . . 66
4.6 Salvi's implementation [24], 7 control points, $\alpha = 0.7$ . . . 67
4.7 Python implementation, 7 control points, $\alpha = 0.7$ . . . 67
4.8 Salvi's implementation [24], 7 control points, $\alpha = 0.95$ . . . 68
4.9 Python implementation, 7 control points, $\alpha = 0.95$ . . . 68
4.10 Osculating circle too small to adjust curvature . . . 70
4.11 Curvature tool updated with fixed length normals . . . 71

List of Algorithms

1 Adding control points . . . 32
2 Removing control points . . . 32
3 Moving control points . . . 32
4 Calculating points on Bézier curve . . . 34
5 Drawing the Bézier curve . . . 34
6 Calculating curvature at t . . . 35
7 Drawing the Osculating Circle . . . 37
8 Dotting the curve . . . 41
9 Solving for $\tilde{t}_i$ in the κ-curve system . . . 46
10 Solving for $\tilde{\lambda}_i$ in the κ-curve system . . . 46
11 Solving for $\tilde{\mathbf{Q}}_{i,1}$ in the κ-curve system . . . 47
12 Yan et al's [35] algorithm for constructing κ-curves . . . 48
13 Drawing the κ-curve . . . 49
14 Polynomial of degree 9 . . . 54
15 Solving for $\hat{t}_i$ in the $\epsilon\kappa$-curve system . . . 54
16 Updating $\hat{t}_i$ values in the $\epsilon\kappa$-curve system . . . 54
17 Solving for $\hat{\lambda}_i$ in the $\epsilon\kappa$-curve system . . . 55
18 Solving for $\hat{\mathbf{Q}}_{i,1}$ in the $\epsilon\kappa$-curve system . . . 56
19 Constructing $\epsilon\kappa$-curves, based on Miura et al's psuedocode [22] . . . 57

1 Introduction

1.1 Context and Motivation

Bézier curves have been a useful tool for computer aided design (CAD) and computer graphics since the 1960s, following their discovery by Pierre Bézier [5]. They have allowed for the creation of complex curves with relatively simple underlying mathematical expressions and further ensure smoothness across the entire curve (no sharp edges). Furthermore, users of this technology need not understand the mathematics behind them in order to control them effectively, by simply adjusting control points to manipulate the curve. Whilst at the time of their invention they seemed intuitive due to their convex hull property, recent advancements have improved the predictability of the shape of the resulting curve. Such curves are called κ-curves.

κ-curves are a sequence of quadratic Bézier curves with an additional constraint - points of maximum curvature lie on control points. As a result, given a set of control points, the outputted κ-curve is more predictable due to all control points lying on the curve. Therefore the manipulation of the κ-curves is more intuitive.

Finally, $\epsilon\kappa$-curves are an evolution of κ-curves and have recently been published by Miura et al [22]. In contrast to κ-curves, they use a sequence of cubic Bézier curves and allow for the adjustment of curvature near control points, allowing for greater controllability for graphic designers. Mathematically, this is done by introducing a variable α to the control points two and three in the Bernstein Polynomial form of the cubic Bézier [22, p. 4]. The implementation in this work provides the user with the ability to adjust global curvature by way of selecting from a set of fixed α values as shown in Figures 1.1 and 1.2.

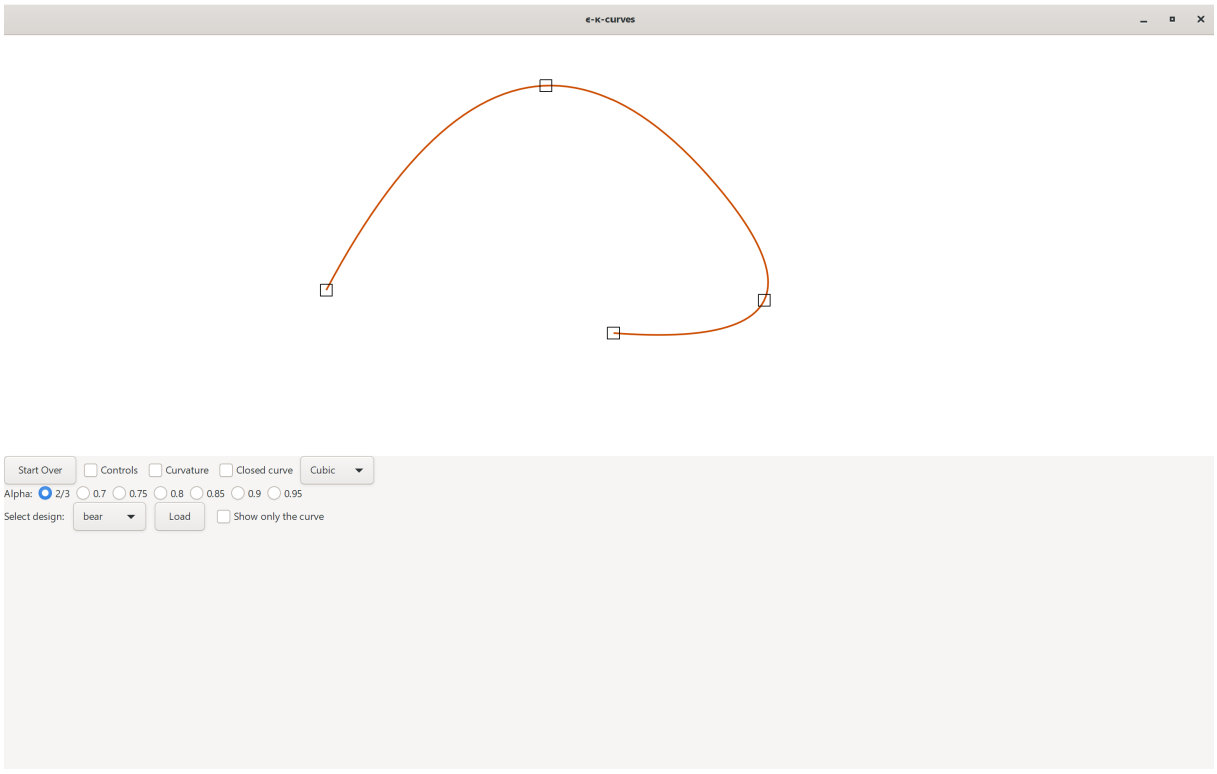


Figure 1.1: Salvi's [24] implementation of $\epsilon\kappa$-curves, $\alpha = 2/3$

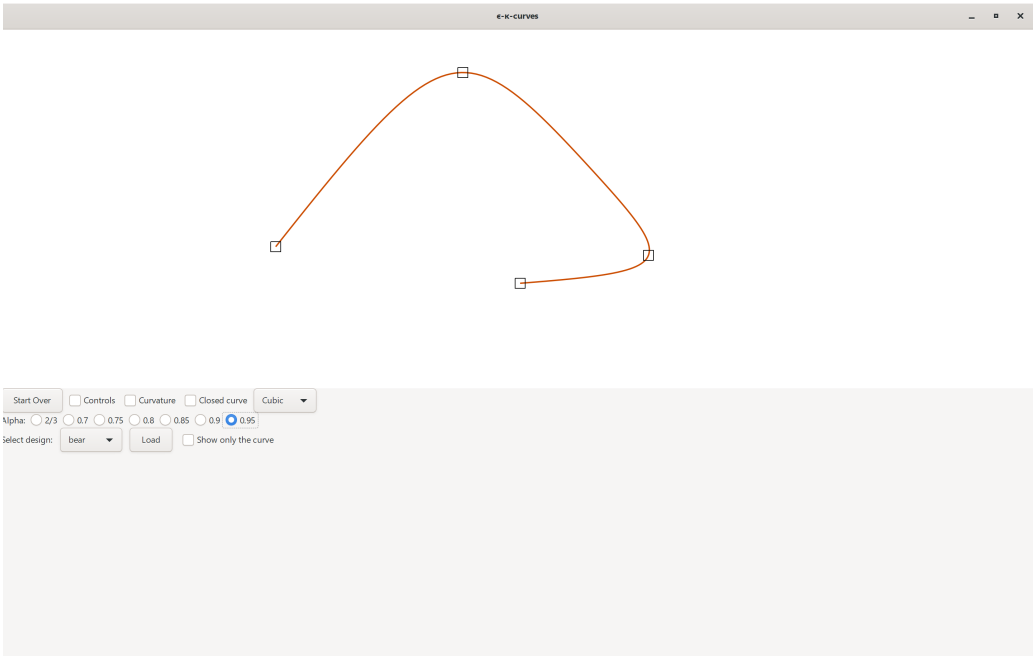


Figure 1.2: Salvi's [24] implementation of $\epsilon\kappa$-curves, $\alpha = 0.95$

Graphic designers and other users have never had to concern themselves with the mathematics of Bézier curves when manipulating them. However the implementation of $\epsilon\kappa$-curves in Figures 1.1 and 1.2 somewhat requires a user to understand the meaning behind α values and provides limited, unintuitive control of this parameter. The outcome of this project attempts to improve on these limitations by implementing a prototype tool that prioritises the following:

1. Allow the user to change curvature around selected points instead of the entire curve
2. Allow the user to adjust curvature without mention of α values
3. Allow the user a range of α values rather than a pre-determined set of values
4. Move the controls for adjusting curvature on to the curve, so that it is clear as to which point they are adjusting the curvature around.

One common method of visualising curvature is the use of an osculating circle. At any given point P on a curve S, a circle of radius $1/\kappa$ where κ is the curvature of S at P can be drawn such that the radius is normal to the curve. The curvature of the circle will match the curvature of S at P [34, 13]. As a result, this visualisation tool was implemented as a way of adjusting curvature at control points.

Evaluation of the system was performed through both correctness testing against Salvi's system [24], and qualitative usability testing with novice users, as discussed in Section 4. After qualitative feedback from the novice users who tested the system,some issues were found. A considerable drawback of this method is when κ is large and thus the radius of the osculating circle was small, making it difficult to adjust curvature at that point. In addition, results from testing indicated a need to improve the responsiveness of the system. As varying α in this report's implementation uses a slider, the values can change rapidly, resulting in numerous calculations over a short time frame.

1.2 Project Aims

The table below outlines the aims for this project that were set prior to the start of development. These aims were carefully chosen to allow me to build strong foundational knowledge of Bézier curves, and build upon this to both improve my understanding of the more technical $\epsilon\kappa$-curves as well explore new ideas for designing an intuitive interface for adjusting the curves. Some aims in the table are written in *italics*, indicating aims that I added during the development phase after acquiring further knowledge. Finally, the table includes a success criteria, which I used to determine if my aim had been met.

Goal	**Reason**	**Success Criteria**
To understand the mathematical expressions behind Bézier curves and their role in the construction of continuous, smooth curves	Understanding these expressions and implementing Bézier curves provides a solid foundation for extending to κ-curves where more parameters are introduced	1. Implement a system with adjustable control points that creates a Bézier curve.
To explore a variety of tools that will aid in the visualisation of curvature	Experimenting with such tools for the simpler Bézier curves will allow for easier implementation of intuitive controls for $\epsilon\kappa$-curves	1. Implement and assess the use of a tangent line for visualising curvature for Bézier Curves. 2. Implement and assess the use of the osculating circle for visualising curvature for Bézier Curves.
To understand the mathematical expressions behind κ-curves and implement them in my own system	Implementing κ-curves is the next logical evolution towards $\epsilon\kappa$-curves	1. Experiment with the position of control points to ensure local maximum curvature occurs only at control points. 2. Ensure the resultant κ-curve from a set of control points is correct by comparing the output to existing systems.
To understand the evolution step from κ-curves to $\epsilon\kappa$-curves	$\epsilon\kappa$-curves are almost identical to κ-curves except the former allows for the control of curvature. Understanding how this is implemented will allow for a better design of intuitive controls for this feature	1. Implement $\epsilon\kappa$-curves and verify the correctness of the implementation by comparing the outputs of my system to similar existing systems.
To create an intuitive tool for adjusting curvature around control points	This is the main aim of this project	1. Collect user feedback from novice users to gauge the intuitiveness of adjusting curvature.

Table 1: Table of goals and success criteria

To assess the system's performance, a two-part evaluation was designed. First, correctness of the system was verified by comparing the output to Salvi's implementation [24]. Second, intuitiveness was assessed through structured user feedback sessions with participants asked to pay particular attention to adjusting curvature.

1.3 Report structure

Below is an outline of the proceeding sections of this report:

- Section 2 provides the necessary theory required to implement Bézier curves, κ-curves and $\epsilon\kappa$-curves, as well as a overview of the key features of these curves. In addition, an evaluation of existing systems will be presented to convey the final goal for this project.
- Section 3 explains the method behind achieving the aims highlighted in Table 1. It introduces formulae used and a summary of the code implemented for each aim. Since this report focusses on designing an intuitive interface, derivations for such formulae are absent, but where appropriate, all formulae will be cited, where such derivations can be found.
- Section 4 evaluates the system implemented by comparing it to existing software and analysing user feedback.
- Section 5 will provided a conclusion of the results of the system, a review of the aims aforementioned and if they were achieved, but also suggestions for improvements and future work.

2 Background

2.1 Bézier curves

2.1.1 Construction of Bézier curves

Consider two points $\mathbf{P}_1, \mathbf{P}_2 \in \mathbb{R}^2$. We can express a straight line joining these two points as a function of t as follows:

$$f_1(t) = (1-t) \cdot \mathbf{P}_1 + \mathbf{P}_2 \cdot t \ , \ t \in [0,1] \tag{2.1}$$

This is known as a linear interpolation (lerp) between $\mathbf{P}_1$ and $\mathbf{P}_2$. [13]

Thus as t varies from 0 to 1, we move from point $\mathbf{P}_1$ to point $\mathbf{P}_2$. Now suppose we introduce a third point $\mathbf{P}_3 \in \mathbb{R}^2$. Once again, we can create a straight line between $\mathbf{P}_2$ and $\mathbf{P}_3$

$$f_2(t) = (1-t) \cdot \mathbf{P}_2 + \mathbf{P}_3 \cdot t \ , \ t \in [0,1] \tag{2.2}$$

Hence, we have created two straight lines between three points as shown in Figure 2.1

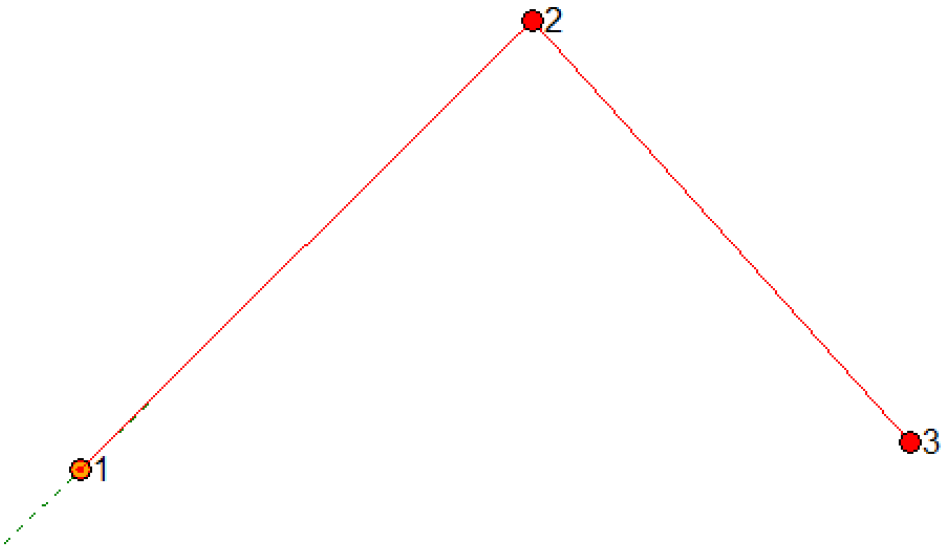


Figure 2.1: An example of a two lerps between three control points

We can extend this by imagining two addition points $\mathbf{P}_4$ and $\mathbf{P}_5$ that follow the paths defined by (2.1) and (2.2) respectively. More specifically, as t varies from 0 to 1, $\mathbf{P}_4$ moves along the line connecting $\mathbf{P}_1$ to $\mathbf{P}_2$ (starting from $\mathbf{P}_1$) and $\mathbf{P}_5$ moves along the line connecting $\mathbf{P}_2$ to $\mathbf{P}_3$ (starting from $\mathbf{P}_2$). Once again, we can perform a lerp on $\mathbf{P}_4$ and $\mathbf{P}_5$

$$f_3(t) = (1-t) \cdot \mathbf{P}_4 + \mathbf{P}_5 \cdot t \ , \ t \in [0,1] \tag{2.3}$$

Now, as t varies once again, we have the following:

i. $\mathbf{P}_4$ moves from $\mathbf{P}_1$ to $\mathbf{P}_2$ along the line connecting $\mathbf{P}_1$ and $\mathbf{P}_2$

ii. $\mathbf{P}_5$ moves from $\mathbf{P}_2$ to $\mathbf{P}_3$ along the line connecting $\mathbf{P}_2$ and $\mathbf{P}_3$

iii. An additional imaginary point $\mathbf{P}_6$ moves from $\mathbf{P}_4$ to $\mathbf{P}_5$ along the line connecting $\mathbf{P}_4$ and $\mathbf{P}_5$ generated by (2.3)

Note that i. and ii. imply that the line connecting $\mathbf{P}_4$ and $\mathbf{P}_5$ also moves as t varies. Finally, if we were to place a pen on $\mathbf{P}_6$ as it's position varies with t, we obtain a B'ezier curve as shown in Figure (2.2)

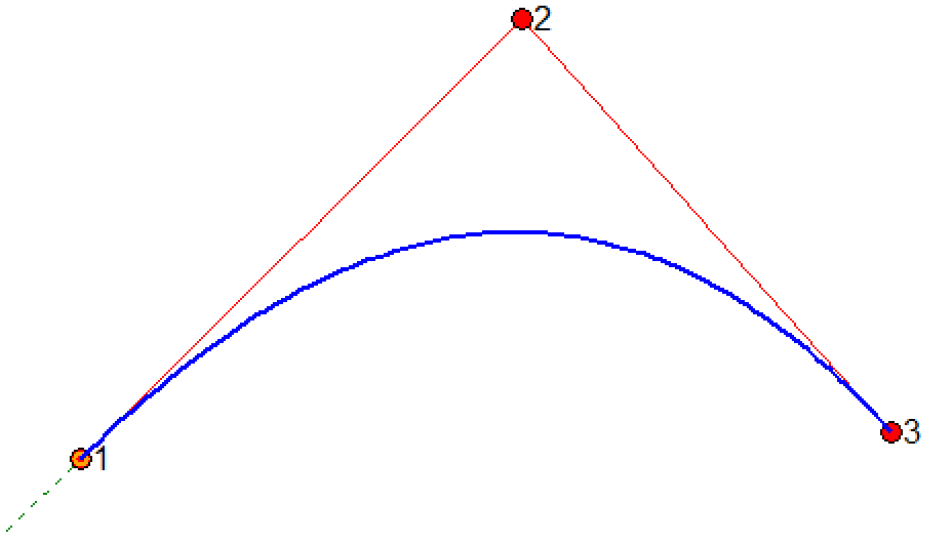


Figure 2.2: A Bézier curve (blue) drawn by $\mathbf{P}_6$

In the above, $\mathbf{P}_1, \mathbf{P}_2$ and $\mathbf{P}_3$ are called control points, as these are the points that the use controls in order to generate their desired Bézier curve. We can generalise the construction of Bézier curves to n control points for $n \geq 3$, by performing a lerp on each pair of control points ($\mathbf{P}_i$,$\mathbf{P}_{i+1}$), then placing imaginary points of each line segment [$\mathbf{P}_i$,$\mathbf{P}_{i+1}$]. We loop through the process of performing lerps on points we have imagined until only one imaginary point remains. Hence, this is the point that draws the B'ezier curve. Such a method of constructing Bézier curves is known as de Casteljau's algorithm [13, 7]

Another way of constructing Bézier curves is by formulating the Bernstein Polynomial. Such a method will be used throughout the rest of this report. Returning to the example of three control points, de Casteljau's algorithm provides a parametric expression for a Bézier curve. In summary, we have the following system:

$$\mathbf{P}_4 = (1-t) \cdot \mathbf{P}_1 + \mathbf{P}_2 \cdot t \tag{2.4}$$
$$\mathbf{P}_5 = (1-t) \cdot \mathbf{P}_2 + \mathbf{P}_3 \cdot t \tag{2.5}$$
$$\mathbf{P}_6 = (1-t) \cdot \mathbf{P}_4 + \mathbf{P}_5 \cdot t \tag{2.6}$$

Substituting (2.4) and (2.5) into (2.6), we obtain the following:

$$\mathbf{P}_6 = (1-t)^2\mathbf{P}_1 + [2t(1-t)]\mathbf{P}_2 + t^2\mathbf{P}_3 \tag{2.7}$$

Thus we have obtained a formulation for a Bézier curve in terms of it's control points. Such a formulation is called the Bernstein Polynomial form. Since this polynomial is quadratic in t for three arbitrary control points, such a Bézier curve is referred to as quadratic.

Definition 2.1.1 (Quadratic Bézier Curve)**.** *A quadratic Bézier curve is any Bézier curve that is defined by three control points*

For the same reason, we also have the following definition:

Definition 2.1.2 (Cubic Bézier Curve)**.** *A cubic Bézier curve is any Bézier curve that is defined by four control points*

Both of these definitions will be important for the construction of κ-curves and $\epsilon\kappa$-curves. In addition, note that a Bézier curve will always start at the initial control point, and finish at the last control point.

In general, the Bernstein Polynomial form of a Bézier curve with n control points is given by [8]:

$$\mathbf{P}(t) = \sum_{i=0}^{n} \mathbf{P}_i B_{i,n}(t) \quad , \quad t \in [0,1] \tag{2.8}$$

where $\mathbf{P}_i$ is the ith control point, n is the order of the curve and $B_{i,n}(t)$ is the Bernstein Polynomial defined as:

$$B_{i,n}(t) = {}^nC_i \cdot t^i \cdot (1-t)^{n-i} \tag{2.9}$$

where

$${}^nC_i = \binom{n}{i} = \frac{n!}{i!(n-i)!} \tag{2.10}$$

is the binomial coefficient.

2.1.2 The osculating circle

We begin this section with a definition of curvature for parametric curves.

Definition 2.1.3 (Curvature of a parametric curve)**.** *For any point* $\mathbf{x}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$ *on a curve* S*, the curvature at* $\mathbf{x}$*, denoted* $\kappa(t)$*, is given by* [12]

$$\kappa(t) = \frac{det(\ddot{\mathbf{x}}, \dot{\mathbf{x}})}{\|\dot{\mathbf{x}}\|^3} = \frac{\ddot{x} \cdot \dot{y} - \ddot{y} \cdot \dot{x}}{(\dot{x}^2 + \dot{y}^2)^{3/2}} \tag{2.11}$$

The osculating circle is a useful tool for visualising curvature. We can draw a circle at any point $\mathbf{x}$ on a curve such that the following two properties hold:

i. The curve at $\mathbf{x}$ is tangent to the circle

ii. The curvature at $\mathbf{x}$ is the same as the curvature of the circle

A circle that meets both these criteria is known as the osculating circle [34] (see Figures 2.3 and 2.4). In order to achieve ii., one simply calculates the radius r of the circle using equation (2.12) [34].

$$r = \frac{1}{|\kappa(t)|} \tag{2.12}$$

Equation (2.12) ensures that the curvature of the osculating circle at t matches the curvature of the Bézier curve at t. We then simply draw this radius perpendicular to the curve at t, thereby making the curve tangent to the osculating circle and satisfying criteria i.

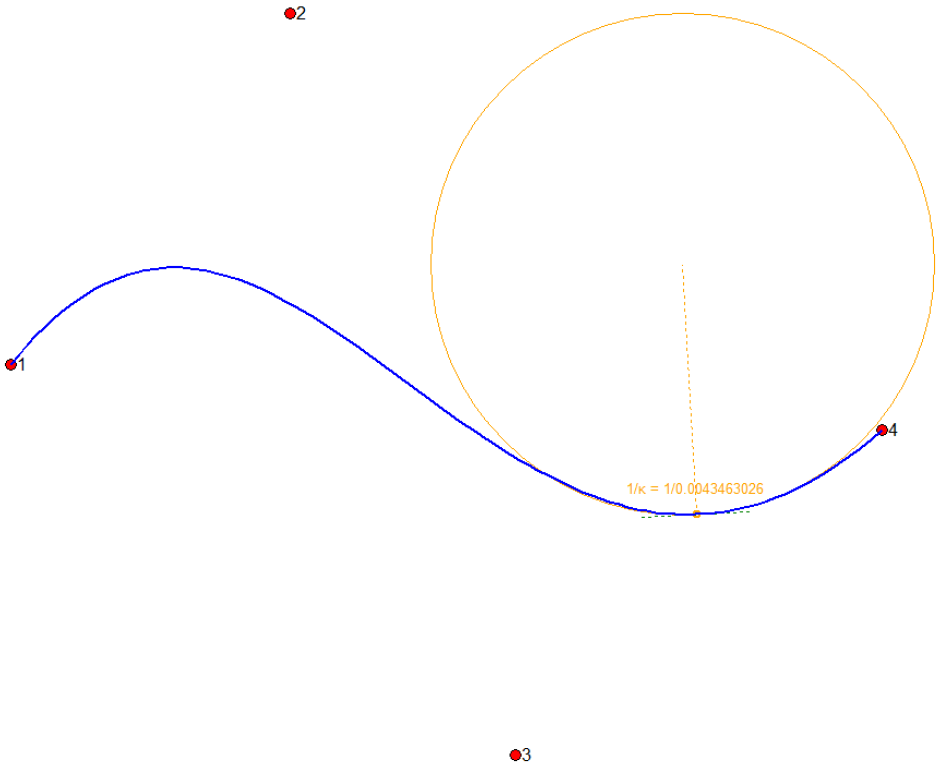


Figure 2.3: The osculating circle (orange) on the Bézier curve (blue)

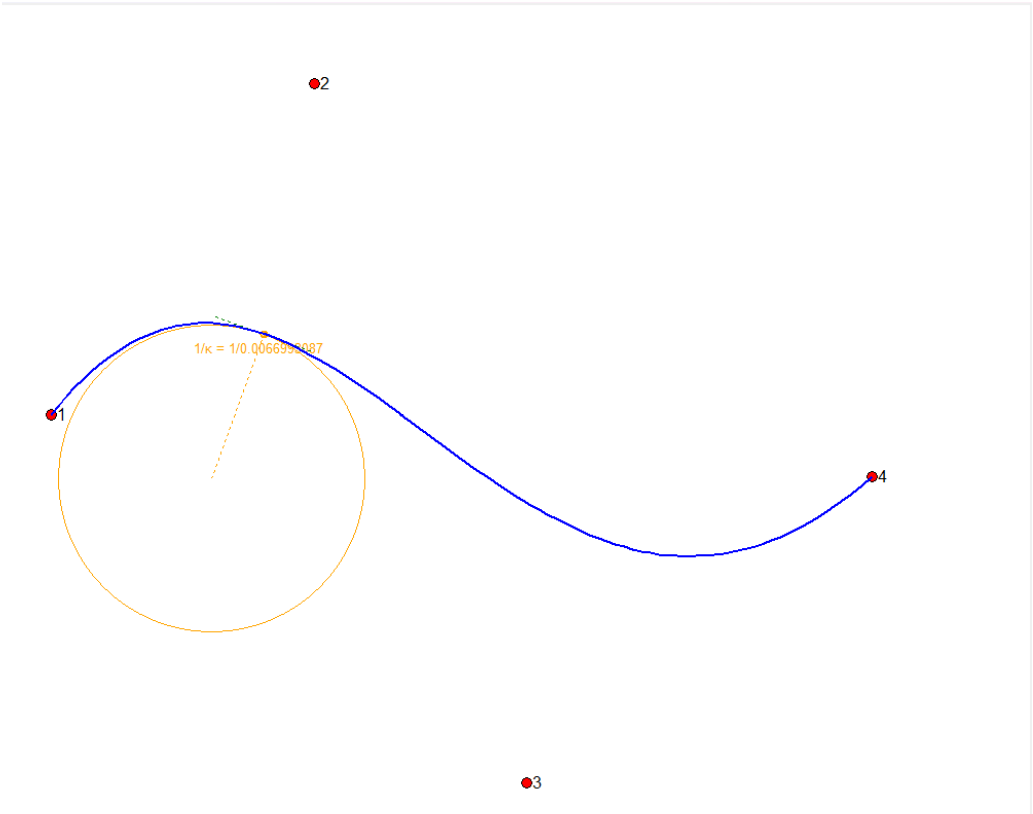


Figure 2.4: The osculating circle (orange) at a different point on the Bézier curve (blue)

2.2 The construction of κ-curves

2.2.1 Introduction

The focus of this report is to design intuitive controls for adjusting curvature in $\epsilon\kappa$-curves. As a result, this section focuses on the construction of κ-curves and their properties and omits derivations of formulas used. Such derivations can be found in Yan et al's work [36, 35]. Further, this report only discusses the implementation of open curves, whereas Yan et al also detail a formulation for closed curves.

κ-curves are manipulated by adjusting control points in the same way one would manipulate a Bézier curve. However, κ-curves interpolate all control points and local curvature maximums occur only at control points [35, p. 33](see Figure 2.5)

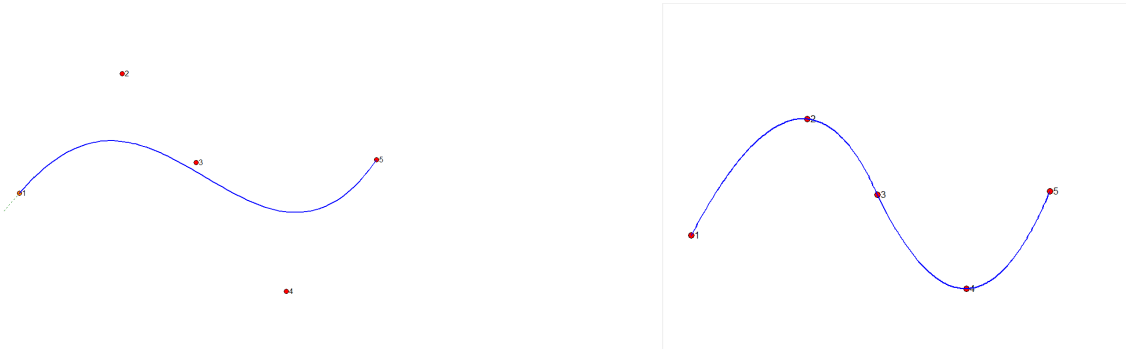


Figure 2.5: Bézier curve (left) versus κ-curve (right)

κ-curves consist of a sequence of quadratic Bézier curves, such that the end point of one Bézier curve is the start of the next Bézier curve. As a result of this construction, κ-curves automatically satisfy G^0-continuity (see Definition 2.2.1)

Definition 2.2.1 (G^0-continuity)**.** *Two curve segments S_1 and S_2 are G^0-continuous if one of the end-points of S_1 is the also one of the end points of S_2 i.e. the curve S is simply connected* [35]

In order to ensure visual smoothness for users, κ-curves must be G^2-continuous (see Definition 2.2.3. Yan et al notes that higher order geometric continuities (G^n , $n \geq 3$) provide smoother shapes but G^2 is suitably smooth for human perception [35, p. 8].

Definition 2.2.2 (G^1-continuity)**.** *Two curve segments S_1 and S_2 are G^1-continuous if they are G^0-continuous and their tangent directions are equal at the point at which they meet i.e. the tangent vector is continuous along the common boundary of S_1 and S_2* [9]

Definition 2.2.3 (G^2-continuity)**.** *Two curve segments S_1 and S_2 are G^2-continuous if they are G^1-continuous and have the same curvature at the point at which they meet i.e. the curvature vector is identical along the common boundary of S_1 and S_2* [9]

The advantage of κ-curves, as highlighted by Yan et al [35, p. 33], is that it allows users to avoid the creation of unintentional features such as cusps as a result of local curvature maximums only existing at control points.

2.2.2 Notation

For the open curve case, we take a set of control points $\{\tilde{\mathbf{P}}_0, \tilde{\mathbf{P}}_1, ..., \tilde{\mathbf{P}}_{\tilde{n}+1}\}$ and thus have $\tilde{n}$ curve segments between the control points (see Figure 2.6).

To understand the construction of κ-curves, we first introduce the following notation.

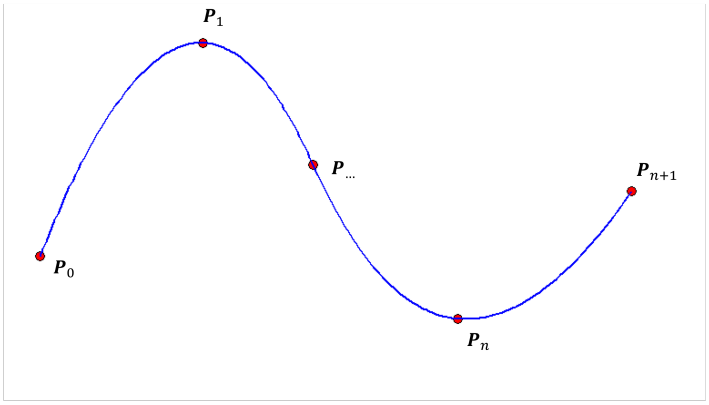


Figure 2.6: κ-curve showing labelling of control points

Definition 2.2.4 (Q points)**.** $\tilde{\mathbf{Q}}_{i,j}$ *denotes the* j^{th} *control point of the* i^{th} *Bézier curve.*

Note that in definition (2.2.4), we call $\tilde{\mathbf{Q}}_{i,j}$ a control point. However, by this we mean that the $\tilde{\mathbf{Q}}_{i,j}$ points control the shape of the ith quadratic Bézier and one cannot adjust them as they are calculated from the user controllable points labelled $\tilde{\mathbf{P}}_i$, as we will show later in this section. For the purpose of clarity:

i. $\tilde{\mathbf{P}}_i$ refers to the points controllable by the user that determine the shape of the entire κ-curve

ii. $\tilde{\mathbf{Q}}_{i,j}$ refers to the points calculated from the set $\{\tilde{\mathbf{P}}_i\}_0^{\tilde{n}+1}$ that determine the shape of the ith Bézier curve.

Also, see Figure 2.7

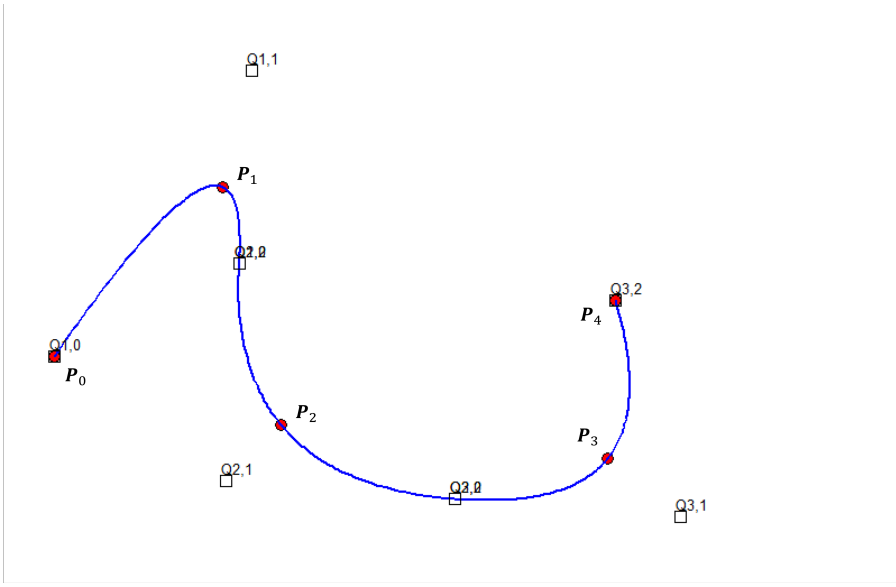


Figure 2.7: κ-curve showing $\tilde{\mathbf{Q}}$ points (black boxes) and $\tilde{\mathbf{P}}$ points (red circles)

2.2.3 Satisfying G^2 - continuity

Note that Figure 2.7 shows the following important properties of $\tilde{\mathbf{Q}}$ points:

i. $\tilde{\mathbf{Q}}_{1,0}$ and $\tilde{\mathbf{Q}}_{\tilde{n},2}$ lie on points $\tilde{\mathbf{P}}_0$ and $\tilde{\mathbf{P}}_{\tilde{n}+1}$ respectively.

ii. $\tilde{\mathbf{Q}}_{i,2}$ overlaps with $\tilde{\mathbf{Q}}_{i+1,0}$

iii. $\tilde{\mathbf{Q}}$ points can lie off the curve.

In fact, we set up $\tilde{\mathbf{Q}}$ points to adhere to i. and ii. By fixing $\tilde{\mathbf{Q}}_{1,0}$ to $\tilde{\mathbf{P}}_0$ and $\tilde{\mathbf{Q}}_{\tilde{n},2}$ to $\tilde{\mathbf{P}}_{\tilde{n}+1}$, we ensure that the κ-curve starts at the first user controllable control point and end at the last - the same behaviour we saw in Bézier curves. In addition, by setting $\tilde{\mathbf{Q}}_{i,2} = \tilde{\mathbf{Q}}_{i+1,0}$, $i \in \{1, ..., \tilde{n} - 1\}$, we automatically satisfy G^0-continuity [35, p. 36], since the end point of the ith Bézier segment is the start point of the Bézier segment $i + 1$. Moreover, to satisfy G^1-continuity, we have that [35, p. 36]:

$$\tilde{\mathbf{Q}}_{i,2} = \tilde{\mathbf{Q}}_{i+1,0} = (1 - \tilde{\lambda}_i)\tilde{\mathbf{Q}}_{i,1} + \tilde{\lambda}_i\tilde{\mathbf{Q}}_{i+1,1} \ , \ i \in \{1, ..., \tilde{n} - 1\} \tag{2.13}$$

for some $\tilde{\lambda}_i \in (0, 1)$ (which will be calculated later)

G^2-continuity is slightly more difficult to achieve. The biggest issue here is that we must allow for two curves to join to create an inflection point, where the sign for curvature is opposite for each curve, thereby making it impossible to satisfy the G^2 condition. As a result, we must weaken this condition to allow the absolute value of curvature at the joining point of two segments to be equal [35, p, 37]. Recall from equation (2.7) that the quadratic Bézier curve is given by:

$$\tilde{\mathbf{P}}_6 = (1 - \tilde{t})^2\tilde{\mathbf{P}}_1 + [2\tilde{t}(1 - \tilde{t})]\tilde{\mathbf{P}}_2 + \tilde{t}^2\tilde{\mathbf{P}}_3 = f(\tilde{t})$$

and recall from Definition 2.1.3 that we can calculate curvature using the formula:

$$\kappa(t) = \frac{det(\ddot{\mathbf{x}}, \dot{\mathbf{x}})}{\|\dot{\mathbf{x}}\|^3}$$

Combining these, and allowing for the weakened version of the G^2 condition, we obtain [35, p. 37]:

$$\frac{|\Delta(\tilde{\mathbf{Q}}_{i,0}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i,2})|}{|\tilde{\mathbf{Q}}_{i,1} - \tilde{\mathbf{Q}}_{i,2}|^3} = \frac{|\Delta(\tilde{\mathbf{Q}}_{i+1,0}, \tilde{\mathbf{Q}}_{i+1}, 1, \tilde{\mathbf{Q}}_{i_1,2})|}{|\tilde{\mathbf{Q}}_{i+1,0} - \tilde{\mathbf{Q}}_{i+1,1}|^3} \tag{2.14}$$

where $\Delta(\tilde{\mathbf{Q}}_1, \tilde{\mathbf{Q}}_2, \tilde{\mathbf{Q}}_3)$ is the area of a triangle defined by the three vertices $\tilde{\mathbf{Q}}_1, \tilde{\mathbf{Q}}_2$ and $\tilde{\mathbf{Q}}_3$, i.e.

$$\Delta(\tilde{\mathbf{Q}}_1, \tilde{\mathbf{Q}}_2, \tilde{\mathbf{Q}}_3) = \frac{1}{2} \cdot [x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)] \tag{2.15}$$

where $\tilde{\mathbf{Q}}_1 = (x_1, y_1)$, $\tilde{\mathbf{Q}}_2 = (x_2, y_2)$ and $\tilde{\mathbf{Q}}_3 = (x_3, y_3)$.

A more detailed derivation of equation (2.14) can be found in Yan et al's work [35].

2.2.4 Maximum curvature condition

To satisfy the maximum curvature condition, we must solve the cubic equation shown in equation (2.16) for $\tilde{t}_i$. [35, p. 38]

$$\|\tilde{\mathbf{Q}}_{i,2} - \tilde{\mathbf{Q}}_{i,0}\|^2\tilde{t}_i^3 + 3(\tilde{\mathbf{Q}}_{i,2} - \tilde{\mathbf{Q}}_{i,0})(\tilde{\mathbf{Q}}_{i,0} - \tilde{\mathbf{P}}_i)\tilde{t}_i^2 + (3\tilde{\mathbf{Q}}_{i,0} - 2\tilde{\mathbf{P}}_i - \tilde{\mathbf{Q}}_{i,2})(\tilde{\mathbf{Q}}_{i,0} - \tilde{\mathbf{P}}_i)\tilde{t}_i - \|\tilde{\mathbf{Q}}_{i,0} - \tilde{\mathbf{P}}_i\|^2 = 0 \tag{2.16}$$

The derivation of this formula uses the Bernstein Polynomial (2.7) and the requirement that $\kappa_i'(\tilde{t}_i) = 0$. Again, the derivation of this formula has been omitted and can once again be found in Yan et al's literature [35]. However, we will state an important theorem from this work as it will be vital for the implementation of κ-curves later in this report:

Theorem 2.2.1. *For any given distinct points $\tilde{\mathbf{Q}}_{i,0}$,$\tilde{\mathbf{Q}}_{i,2}$ and $\tilde{\mathbf{P}}_i$, equation (2.16) has a unique root $\tilde{t}_i$ in the closed interval [0,1]* [35]

Proof See Yan et al [35, p. 38-39]

2.2.5 Solving for $\tilde{\lambda}_i$

Equation (2.13) introduced a new parameter $\tilde{\lambda}_i$. Recall that this equation enables the κ-curve to satisfy the G^1 condition. To solve for $\tilde{\lambda}_i$, we use equation (2.17) [36, p. 4]

$$\tilde{\lambda}_i = \frac{\sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,0}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1})|}}{\sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,0}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1})|} + \sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1}, \tilde{\mathbf{Q}}_{i+1,2})|}} \tag{2.17}$$

Aside Note here the change in the cited literature. In Yan et al's dissertation [35], the formula given for solving for $\tilde{\lambda}_i$ is:

$$\tilde{\lambda}_i = \frac{\sqrt{|\Delta(\tilde{\mathbf{Q}}_{i-1,1}, \tilde{\mathbf{Q}}_{i+1,1}, \tilde{\mathbf{Q}}_{i+1,1})|(1-\tilde{\lambda}_{i-1})}}{\sqrt{|\Delta(\tilde{\mathbf{Q}}_{i-1,1}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1})|(1-\tilde{\lambda}_{i-1})} + \sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1}, Q_{i+2,1})|(\tilde{\lambda}_{i+1})}} \tag{2.18}$$

This formula causes a particular problem when implementing κ-curves, specifically from the term $\tilde{\mathbf{Q}}_{i+2,1}$. As we will see further in this report, for a κ-curve consisting of 5 control points, we require $\tilde{\lambda}_1$ and $\tilde{\lambda}_2$. A κ-curved defined by 5 control points requires three Bézier curves, and thus the final $\tilde{\mathbf{Q}}$ point is $\tilde{\mathbf{Q}}_{3,2}$. However, to solve for $\tilde{\lambda}_2$, equation (2.18) requires a $\tilde{\mathbf{Q}}_{4,1}$ from the $\tilde{\mathbf{Q}}_{i+2,1}$ term.
Also, note that in [36], $\tilde{\mathbf{Q}}_{i,j}$ is denoted $c_{i,j}$. **end aside**

In equation (2.17), it is possible to divide by zero, if both the areas in the denominator are zero. Hence, $\tilde{\lambda}_i$ will be undefined. To avoid such an error, we add a small constant $\epsilon = 10^{-10}$ to the square roots of each area [36, p. 4].

2.2.6 Off curve points

With equations (2.16) and (2.17), we are now in a position that allows us to calculate the off-curve points $\tilde{\mathbf{Q}}_{i,1}$. Recall that the other $\tilde{\mathbf{Q}}$ points have already been calculated by equation (2.13). For an open curve, we have input points $\{\tilde{\mathbf{P}}_0, \tilde{\mathbf{P}}_1, ..., \tilde{\mathbf{P}}_{\tilde{n}+1}\}$ with $\tilde{n}$ curve segments. The first segment should have control points $\{\tilde{\mathbf{P}}_0 = \tilde{\mathbf{Q}}_{1,0}, \tilde{\mathbf{Q}}_{1,1}, \tilde{\mathbf{Q}}_{1,2}\}$ and the last segment should have control points $\{\tilde{\mathbf{Q}}_{\tilde{n},0}, \tilde{\mathbf{Q}}_{\tilde{n},1}, \tilde{\mathbf{P}}_{\tilde{n}+1} = \tilde{\mathbf{Q}}_{\tilde{n},2}\}$. Due to the labelling of the $\tilde{\mathbf{P}}$ points, $\tilde{n}$ is simply the number of user controllable points minus two. To find each $\tilde{\mathbf{Q}}_{i,1}$, we have that [35, p. 43]:

$$\begin{cases} \tilde{\mathbf{P}}_1 = (1-\tilde{t}_1^2)\tilde{\mathbf{P}}_0 + [2(1-\tilde{t}_1)\tilde{t}_1 + (1-\tilde{\lambda}_1)\tilde{t}_1^2]\tilde{\mathbf{Q}}_{1,1} + \tilde{\lambda}_1\tilde{t}_1^2\tilde{\mathbf{Q}}_{2,1} \\ \tilde{\mathbf{P}}_i = (1-\tilde{\lambda}_{i-1})(1-\tilde{t}_i^2)\tilde{\mathbf{Q}}_{i-1,1} + [\tilde{\lambda}_{i-1}(1-\tilde{t}_i)^2 + 2(1-\tilde{t}_i)\tilde{t}_i + (1-\tilde{\lambda}_i)\tilde{t}_i^2]\tilde{\mathbf{Q}}_{i,1} + \tilde{\lambda}_i\tilde{t}_i^2\tilde{\mathbf{Q}}_{i+1,1} \quad , \quad i \in \{2,3,...,\tilde{n}-1\} \\ \tilde{\mathbf{P}}_{\tilde{n}} = (1-\tilde{\lambda}_{\tilde{n}-1})(1-\tilde{t}_{\tilde{n}})^2\tilde{\mathbf{Q}}_{\tilde{n}-1,1} + [\tilde{\lambda}_{\tilde{n}-1}(1-\tilde{t}_{\tilde{n}})^2 + 2(1-\tilde{t}_{\tilde{n}})\tilde{t}_{\tilde{n}}]\tilde{\mathbf{Q}}_{\tilde{n},1} + \tilde{t}_{\tilde{n}}^2\tilde{\mathbf{P}}_{\tilde{n}+1} \end{cases} \tag{2.19}$$

Which can be expressed as the linear system [35, p. 43-44]:

$$\underbrace{\begin{pmatrix} 2(1-\tilde{t}_1)\tilde{t}_1 + (1-\tilde{\lambda}_1)\tilde{t}_1^2 & \tilde{\lambda}_1\tilde{t}_1^2 & 0 & \cdots \\ * & * & \cdots & \cdots \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \cdots & (1-\tilde{\lambda}_{\tilde{n}-1})(1-\tilde{t}_{\tilde{n}})^2 & \tilde{\lambda}_{\tilde{n}-1}(1-\tilde{t}_{\tilde{n}})^2 + 2(1-\tilde{t}_{\tilde{n}})\tilde{t}_{\tilde{n}} \end{pmatrix}}_{\underline{\underline{\mathbf{A}}}} \cdot \begin{pmatrix} \tilde{\mathbf{Q}}_{1,1} \\ \tilde{\mathbf{Q}}_{2,1} \\ \vdots \\ \tilde{\mathbf{Q}}_{\tilde{n},1} \end{pmatrix} = \begin{pmatrix} \tilde{\mathbf{P}}_1 - (1-\tilde{t}_1)^2\tilde{\mathbf{P}}_0 \\ \tilde{\mathbf{P}}_2 \\ \vdots \\ \tilde{\mathbf{P}}_{\tilde{n}} - \tilde{t}_{\tilde{n}}^2\mathbf{P}_{\tilde{n}+1} \end{pmatrix} \tag{2.20}$$

The matrix labelled $\underline{\underline{\mathbf{A}}}$ in equation (2.20) is an $n \times n$ matrix and is the matrix of coefficients of each $\tilde{\mathbf{Q}}_{i,1}$ for each $\tilde{\mathbf{P}}_i$. Therefore, each row (excluding the first and the last) will have three non-zero entries as shown by the equations (2.19).

Example Consider a κ-curve consisting of 5 control points. Then, we have that $n = 5-2 = 3$ and thus $\underline{\underline{\mathbf{A}}}$ is a 3×3 matrix. The entries in row 1 and row 3 of $\underline{\underline{\mathbf{A}}}$ are fixed by equation (2.20). The entries in row 2 are determined by the second equation in (2.19). We have that:

$$\tilde{\mathbf{P}}_2 = (1-\tilde{\lambda}_1)(1-\tilde{t}_2^2)\tilde{\mathbf{Q}}_{1,1} + [\tilde{\lambda}_1(1-\tilde{t}_2)^2 + 2(1-\tilde{t}_2)\tilde{t}_2 + (1-\tilde{\lambda}_2)\tilde{t}_2^2]\tilde{\mathbf{Q}}_{2,1} + \tilde{\lambda}_2\tilde{t}_2^2\tilde{\mathbf{Q}}_{3,1}$$

and thus the linear system is:

$$\begin{pmatrix} 2(1-\tilde{t}_1)\tilde{t}_1 + (1-\tilde{\lambda}_1)\tilde{t}_1^2 & \tilde{\lambda}_1\tilde{t}_1^2 & 0 \\ (1-\tilde{\lambda}_1)(1-\tilde{t}_2^2) & \tilde{\lambda}_1(1-\tilde{t}_2)^2 + 2(1-\tilde{t}_2)\tilde{t}_2 + (1-\tilde{\lambda}_2)\tilde{t}_2^2 & \tilde{\lambda}_2\tilde{t}_2^2 \\ 0 & (1-\tilde{\lambda}_2)(1-\tilde{t}_3)^2 & \tilde{\lambda}_2(1-\tilde{t}_3)^2 + 2(1-\tilde{t}_3)\tilde{t}_3 \end{pmatrix} \cdot \begin{pmatrix} \tilde{\mathbf{Q}}_{1,1} \\ \tilde{\mathbf{Q}}_{2,1} \\ \tilde{\mathbf{Q}}_{3,1} \end{pmatrix} = \begin{pmatrix} \tilde{\mathbf{P}}_1 - (1-\tilde{t}_1)^2\tilde{\mathbf{P}}_0 \\ \tilde{\mathbf{P}}_2 \\ \tilde{\mathbf{P}}_3 - \tilde{t}_3^2\mathbf{P_4} \end{pmatrix}$$

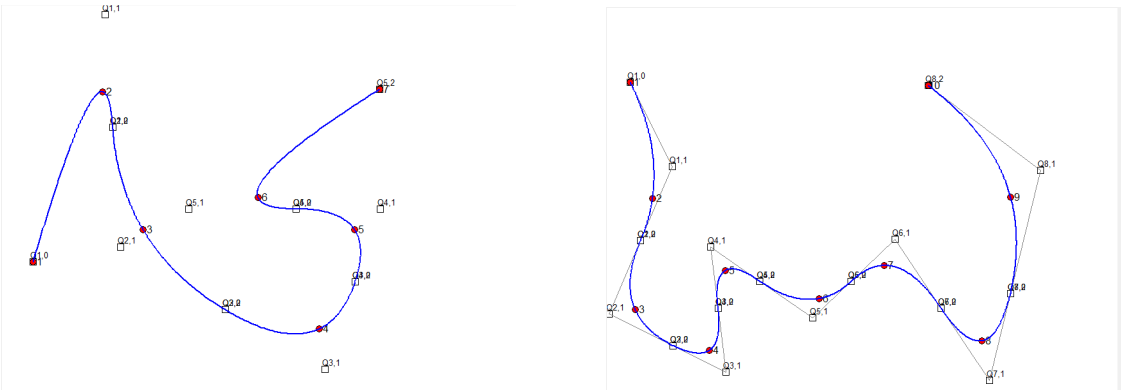
where $\tilde{t}_i$ is found by equation (2.16) and $\tilde{\lambda}_i$ is found by equation (2.17).

This example shows the issue with equation (2.18). In order to solve for $\tilde{\lambda}_2$, (2.18) requires a $\tilde{\mathbf{Q}}_{4,1}$ from the $\tilde{\mathbf{Q}}_{i+2,1}$ term in the denominator. If such a term were to exist for this system, it would further need to be included in the column vector containing all $\tilde{\mathbf{Q}}_{i,1}$ in the left hand side of linear system found in this example. As a result, the matrix of coefficients would have to be a 4×4 matrix. This leads to two issues:

i. The column vector on the right hand side of the equation would require four entries (from basic linear algebra)

ii. The matrix of coefficients would require a $\tilde{\lambda}_3$ term (from equation (2.20)) and therefore a $\tilde{\mathbf{Q}}_{5,1}$ (from (2.18)). Thus, the cycle continues infinitely

Equations (2.13) and (2.20) allow us to solve for all the required $\tilde{\mathbf{Q}}$ points. As a result, we can construct a κ-curve, by constructing $\tilde{n}$ quadratic Bézier curve segments. The ith Bézier curve has control points $\{\tilde{\mathbf{Q}}_{i,0}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i,2}\}$ where $\tilde{\mathbf{Q}}_{1,0}$ is fixed to $\tilde{\mathbf{Q}}_0$ and $\tilde{\mathbf{Q}}_{\tilde{n},2}$ is fixed to $\tilde{\mathbf{Q}}_{\tilde{n}+1}$. See Figure 2.8 for κ-curves defined by more than 5 control points.



(a) κ-curve with 7 control points

(b) κ-curve with 10 control points

Figure 2.8: κ-curves and their $\tilde{\mathbf{Q}}$-points

2.3 The construction of $\epsilon\kappa$-curves

2.3.1 Introduction

In this section, we extend and adapt the construction of κ-curves to formulate $\epsilon\kappa$-curves. This type of curve has similar behaviour to κ-curves, namely [22, p. 4]:

i. They interpolate all control points

ii. Curvature maximums occur only at control points

iii. G^2-continuity except at inflection points

However, $\epsilon\kappa$-curves give users control over the curvature across the curve. Simply, users are able to change how 'sharp' the curve is, particularly at points of maximum curvature. This extension of κ-curves was published by Miura et al [22]. Instead of using a sequence of quadratic Bézier curves, $\epsilon\kappa$-curves are constructed with a sequence of cubic Bézier curves.

2.3.2 Notation

Likewise with κ-curves, we aim to compute control points that determine the shape of each Bézier curve from the set of user controllable points. In order to distinguish between the variables from the κ-curves construction, we will denote user input points by $\hat{\mathbf{P}}_i$ and the calculated control points by $\hat{\mathbf{Q}}_{i,j}$ in order to remain consistent with our construction of κ-curves.
In Miura et al [22], note that the calculated control points are denoted $c_{i,j}$.

2.3.3 The Bernstein Polynomial

Consider a system of four control points $\{\hat{\mathbf{P}}_0, \hat{\mathbf{P}}_1, \hat{\mathbf{P}}_2, \hat{\mathbf{P}}_3\}$. Then, using equation (2.8), the Bernstein polynomial for the Bézier curve is given by equation (2.21)

$$\hat{\mathbf{P}}(\hat{t}) = (1-\hat{t})^3\hat{\mathbf{P}}_0 + 3\hat{t}(1-\hat{t})^2\hat{\mathbf{P}}_1 + 3\hat{t}^2(1-\hat{t})\hat{\mathbf{P}}_2 + \hat{t}^3\hat{\mathbf{P}}_3 \tag{2.21}$$

Our aim in this subsection is to express (2.21) in terms of $\mathbf{Q}$ points.

First, note that a Bézier curve satisfying (2.21) will start at $\hat{\mathbf{P}}_0$ and end at $\hat{\mathbf{P}}_3$. Thus, assuming that (2.21) represents the ith cubic Bézier in our construction of $\epsilon\kappa$-curves, we must fix $\hat{\mathbf{Q}}_{i,0}$ to $\hat{\mathbf{P}}_0$ and $\hat{\mathbf{Q}}_{i,2}$ to $\hat{\mathbf{P}}_3$.

Our problem now lies in the fact that we have two control points $\hat{\mathbf{P}}_1$ and $\hat{\mathbf{P}}_2$ that must be expressed in terms of the calculated control point $\hat{\mathbf{Q}}_{i,1}$. In order to do this, we introduce a variable α_i, an internal division ratio [22, p. 4] (see Figure 2.9), such that:

$$\hat{\mathbf{P}}_1 = (1-\alpha_i)\hat{\mathbf{Q}}_{i,0} + \alpha_i\hat{\mathbf{Q}}_{i,1} \tag{2.22}$$

$$\hat{\mathbf{P}}_2 = \alpha_i\hat{\mathbf{Q}}_{i,1} + (1-\alpha_i)\hat{\mathbf{Q}}_{i,2} \tag{2.23}$$

and thus equation (2.21) becomes [22, p. 4]:

$$c_i(\hat{t},\alpha_i) = (1-\hat{t})^3\hat{\mathbf{Q}}_{i,0} + 3\hat{t}(1-\hat{t})^2[(1-\alpha_i)\hat{\mathbf{Q}}_{i,0} + \alpha_i\hat{\mathbf{Q}}_{i,1}] + 3\hat{t}^2(1-\hat{t})[\alpha_i\hat{\mathbf{Q}}_{i,1} + (1-\alpha_i)\hat{\mathbf{Q}}_{i,2}] + \hat{t}^3\hat{\mathbf{Q}}_{i,2} \tag{2.24}$$

where c_i is the ith Bézier curve.

The α value The internal division ratio α gives users of $\epsilon\kappa$-curves more control over the curve. From equations (2.22) and (2.23) and from Figure 2.9, we see that larger α values result in $\hat{\mathbf{Q}}_{i,1}$ being closer to control points $\hat{\mathbf{P}}_1$ and $\hat{\mathbf{P}}_2$. As a result, the value of alpha changes the curvature of the curve around the point, therefore the user should be free to change the α value themselves. Miura et al [22] suggests restricting α such that $2/3 \leq \alpha < 1$ in order to avoid complicated curvature distribution. However, values outside of this range were experimented with in the implmentation discussed later in this report.
This part of the theory is where the main objective of this project lies. In Salvi's $\epsilon\kappa$-curve implementation [24], the user controls curvature by selecting different α values. By using the osculating circle, we propose a method for adjusting curvature that does not expose the user to α values.

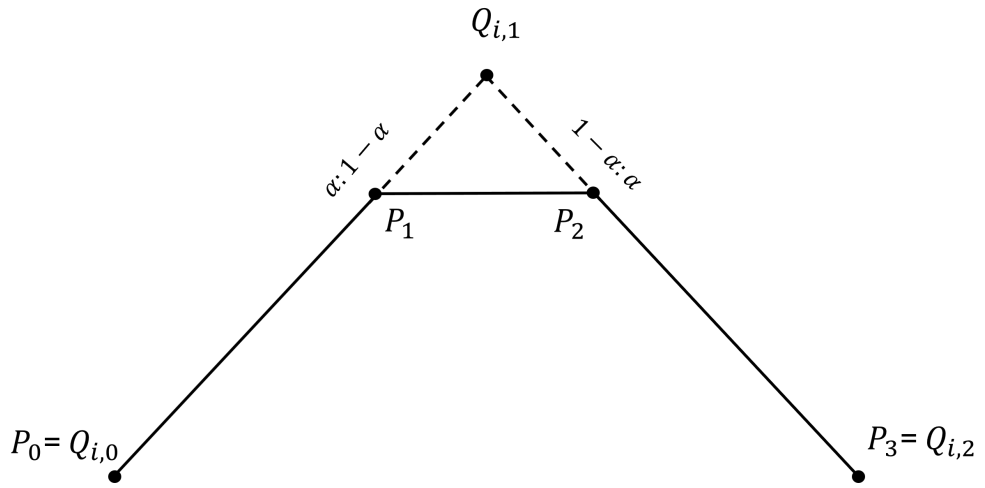


Figure 2.9: Demonstrating internal division ratio α. Adapted from Miura et al. (2021), licensed under CC BY 4.0 (see http://creativecommons.org/licenses/by/4.0/).

2.3.4 Governing equations

As in the construction of κ-curves, we must solve for parameters $\hat{t}_1$, $\hat{\lambda_i}$ and $\hat{\mathbf{Q}}_{i,j}$. Many of these formulas remain similar to those seen in the construction of κ-curves but with the inclusion of the α parameter. In fact, for the joining $\mathbf{Q}$ points we must satisfy the same equation as (2.13)[22, p. 5]:

$$\hat{\mathbf{Q}}_{i,2} = \hat{\mathbf{Q}}_{i+1,0} = (1 - \hat{\lambda}_i)\hat{\mathbf{Q}}_{i,1} + \hat{\lambda}_i\hat{\mathbf{Q}}_{i+1,1}\ ,\ i \in \{1, ..., \hat{n} - 1\} \tag{2.25}$$

In addition, the equation to solve for $\hat{\lambda}_i$ is also relatively similar. Miura et al [22] introduces some notation that this report will also adopt. Equation (2.26) solves for $\hat{\lambda}_i$

$$\hat{\lambda}_i = \frac{\sqrt{(1-\alpha_i)\Delta_i^+}}{\sqrt{(1-\alpha_i)\Delta_i^+} + \frac{\alpha_i}{\alpha_{i+1}}\sqrt{(1-\alpha_{i+1})\Delta_{i+1}^-}} \tag{2.26}$$

where [22]:

$$\Delta_i^+ = \Delta(\hat{\mathbf{Q}}_{i,0}, \hat{\mathbf{Q}}_{i,1}, \hat{\mathbf{Q}}_{i+1,1}) \tag{2.27}$$

$$\Delta_i^- = \Delta(\hat{\mathbf{Q}}_{i-1,1}, \hat{\mathbf{Q}}_{i,1}, \hat{\mathbf{Q}}_{i,2}) \tag{2.28}$$

Note how equation (2.26) is similar to equation (2.17) from [36] and not equation (2.18) from [35], which was previously shown to be invalid.

However, despite these similarities, the equation required to solve for $\hat{t}_i$ is drastically different. Recall that for the case of κ-curves, the equation for $\tilde{t}_i$ (2.16) is derived from the quadratic Bernstein polynomial (2.7) and the fact that $\kappa_i'(\tilde{t}_1) = 0$. For $\epsilon\kappa$-curves, the equation for $\hat{t}_i$ is derived in the same way, but instead we have a cubic Bernstein Polynomial. Recalling that:

$$\kappa(\hat{t}) = \frac{det(\ddot{\mathbf{x}}, \dot{\mathbf{x}})}{\|\dot{\mathbf{x}}\|^3} = \frac{\ddot{x} \cdot \dot{y} - \ddot{y} \cdot \dot{x}}{(\dot{x}^2 + \dot{y}^2)^{3/2}}$$

is the curvature at a point on a curve, we can use the formula (2.29) to find the curvature at any point on the cubic Bézier. [22, p. 4]:

$$\kappa_i(\hat{t}, \alpha_i) = \frac{det\left(\frac{\partial c_i(\hat{t},\alpha_i)}{\partial \hat{t}}, \frac{\partial^2 c_i(\hat{t},\alpha_i)}{\partial \hat{t}^2}\right)}{\left|\left|\frac{\partial c_i(\hat{t},\alpha_i)}{\partial \hat{t}}\right|\right|^3} \tag{2.29}$$

where $c_i(\hat{t}\,, \alpha_i)$ is defined in equation (2.21)

The result of combining the cubic Bernstein (2.21) and $\kappa_i'(\hat{t}_{\,1}) = 0$ results in a polynomial equation of degree 9 in $\hat{t}_{\,i}$ [22, p. 5]. Fortunately, Miura et al [22] provide Maxima [30] code [25] that calculates the coefficients for the polynomial in terms of control points $\{\hat{\mathbf{P}}_{\,i}\}_{i=0}^{n+1}$.

To find the off curve points $\hat{\mathbf{Q}}_{\,i,1}$ substitute equation (2.25) into the condition $c_i(t_i) = \hat{\mathbf{P}}_{\,i}$ to obtain [22]:

$$\begin{aligned}
\hat{\mathbf{P}}_{\,i} &= (1-\hat{t}_{\,i})^3[(1-\hat{\lambda}_{\,i-1})\hat{\mathbf{Q}}_{\,i-1,1} + \hat{\lambda}_{\,i-1}\hat{\mathbf{Q}}_{\,i,1}] \\
&+ 3(1-\hat{t}_{\,i})^2\hat{t}_{\,i}[(1-\alpha_i)\langle(1-\hat{\lambda}_{\,i-1})\hat{\mathbf{Q}}_{\,i-1,1} + \hat{\lambda}_{\,i-1}\hat{\mathbf{Q}}_{\,i,1}\rangle + \alpha_i\hat{\mathbf{Q}}_{\,i,1}] \\
&+ 3(1-\hat{t}_{\,i})\hat{t}_{\,i}^{\,2}[(1-\alpha_i)\langle(1-\hat{\lambda}_{\,i})\hat{\mathbf{Q}}_{\,i,1} + \hat{\lambda}_{\,i}\hat{\mathbf{Q}}_{\,i+1,1}\rangle + \alpha_i\hat{\mathbf{Q}}_{\,i,1}] \\
&+ \hat{t}_{\,i}^{\,3}[(1-\lambda_i)\hat{\mathbf{Q}}_{\,i,1} + \hat{\lambda}_{\,i}\hat{\mathbf{Q}}_{\,i+1,1}]
\end{aligned} \tag{2.30}$$

Miura et al [22] discuss only closed curves, but we can use the knowledge obtained from the κ-curve construction to find that:

$$\begin{cases}
\hat{\mathbf{P}}_{\,1} = \hat{\mathbf{P}}_{\,0}\left[(1-\hat{t}_{\,1})^3 + 3(1-\hat{t}_{\,1})^2\cdot\hat{t}_{\,1}(1-\alpha_1)\right] \\
\qquad + \left[\alpha_1\cdot 3(1-\hat{t}_{\,1})\cdot\hat{t}_{\,1} + (1-\hat{\lambda}_1)\left(\hat{t}_1^{\,3} + 3(1-\hat{t}_{\,1})\cdot\hat{t}_1^{\,2}(1-\alpha_1)\right)\right]\hat{\mathbf{Q}}_{1,1} \\
\qquad + \left[\hat{\lambda}_1\left(\hat{t}_1^{\,3} + 3(1-\hat{t}_{\,1})\cdot\hat{t}_1^{\,2}(1-\alpha_1)\right)\right]\hat{\mathbf{Q}}_{2,1} \\
\hat{\mathbf{P}}_{\,\hat{n}} = \hat{\mathbf{P}}_{\,\hat{n}+1}\left[\hat{t}_{\hat{n}}^{\,3} + 3(1-\hat{t}_{\,\hat{n}})\cdot\hat{t}_{\hat{n}}^{\,2}(1-\alpha_{\hat{n}})\right] \\
\qquad + \left[(1-\hat{\lambda}_{\hat{n}-1})\left((1-\hat{t}_{\,\hat{n}})^3 + 3(1-\hat{t}_{\,\hat{n}})^2\cdot\hat{t}_{\,\hat{n}}(1-\alpha_{\hat{n}})\right)\right]\hat{\mathbf{Q}}_{\hat{n}-1,1} \\
\qquad + \left[\hat{\lambda}_{\hat{n}-1}\left((1-\hat{t}_{\,\hat{n}})^3 + 3(1-\hat{t}_{\,\hat{n}})^2\cdot\hat{t}_{\,\hat{n}}(1-\alpha_{\hat{n}})\right) + \alpha_{\hat{n}}\cdot 3(1-\hat{t}_{\,\hat{n}})\cdot\hat{t}_{\,\hat{n}}\right]\hat{\mathbf{Q}}_{\hat{n},1}
\end{cases} \tag{2.31}$$

Equations (2.30) and (2.31) can be written as the linear system:

$$\begin{pmatrix}
\mathcal{M}_1 + (1-\hat{\lambda}_{\,1})\cdot\mathcal{R}_1 & \hat{\lambda}_{\,1}\cdot\mathcal{R}_1 & 0 & \cdots & 0 \\
(1-\hat{\lambda}_{\,1})\cdot\mathcal{L}_2 & \mathcal{M}_2 & \hat{\lambda}_{\,2}\cdot\mathcal{R}_2 & 0 & \cdots \\
0 & (1-\hat{\lambda}_{\,i-1})\cdot\mathcal{L}_i & \hat{\lambda}_{\,i-1}\cdot\mathcal{L}_i + \mathcal{M}_i + (1-\hat{\lambda}_{\,i})\cdot\mathcal{R}_i & \hat{\lambda}_{\,i}\cdot\mathcal{R}_i & \ddots \\
\vdots & \ddots & \ddots & \ddots & \ddots \\
0 & \cdots & 0 & (1-\hat{\lambda}_{\,\hat{n}-1})\cdot\mathcal{L}_{\hat{n}} & \hat{\lambda}_{\,\hat{n}-1}\cdot\mathcal{L}_{\hat{n}} + \mathcal{M}_{\hat{n}}
\end{pmatrix}\cdot\begin{pmatrix}
\hat{\mathbf{Q}}_{\,1,1} \\ \hat{\mathbf{Q}}_{\,2,1} \\ \vdots \\ \hat{\mathbf{Q}}_{\,\hat{n},1}
\end{pmatrix} = \begin{pmatrix}
\hat{\mathbf{P}}_{\,1} - \hat{\mathbf{P}}_{\,0}\cdot\mathcal{L}_1 \\ \hat{\mathbf{P}}_{\,2} \\ \vdots \\ \hat{\mathbf{P}}_{\,\hat{n}} - \mathbf{P}_{\hat{n}+1}\cdot\mathcal{R}_{\hat{n}}
\end{pmatrix} \tag{2.32}$$

where

$$\mathcal{L}_i = \mathcal{L}(\hat{t}_{\,i}, \alpha_i) = (1-\hat{t}_{\,i})^3 + 3(1-\hat{t}_{\,i})^2\cdot\hat{t}_{\,i}(1-\alpha_i) \qquad \text{(left basis)} \tag{2.33}$$

$$\mathcal{M}_i = \mathcal{M}(\hat{t}_{\,i}, \alpha_i) = \alpha_i\cdot 3(1-\hat{t}_{\,i})\cdot\hat{t}_{\,i} \qquad \text{(middle basis)} \tag{2.34}$$

$$\mathcal{R}_i = \mathcal{R}(\hat{t}_{\,i}, \alpha_i) = \hat{t}_{\,i}^{\,3} + 3(1-\hat{t}_{\,i})\cdot\hat{t}_{\,i}^{\,2}(1-\alpha_i) \qquad \text{(right basis)} \tag{2.35}$$

2.4 Existing Systems

2.4.1 Summary

κ-curves are implemented as the pen tool in Adobe Photoshop® and Illustrator® [36]. According to the paper, in the first six months of it's release, this new tool had a 35% usage on desktop and 66% usage on touchscreen compared to the traditional tools that experienced users were familiar with. This data suggests the usefulness and the desire for better curve drawing software.

Currently, Salvi's [24] implementation of $\epsilon\kappa$-curves using Julia [6] is the only current system using this tool. The paper by Miura et al [22] suggests that for future work, $\epsilon\kappa$-curves should also be implemented in Adobe Photoshop® [2] and Illustrator®[1] [22, p. 10].

2.4.2 Advantages and Disadvantages

Since this report focuses on improving the intuitiveness of $\epsilon\kappa$-curves, this section will summarise the advantages and disadvantages (see Table 2) of Salvi's implementation [24].

Advantages	**Disadvantages**
Fast, responsive system allowing for real time curve manipulation	Curvature is changed across the entire curve, rather than around individual control points, thereby restricting control of the curve.
Provides functionality for easily adding control points at any point on canvas (control point is added at the location of the mouse upon a left click)	Unable to remove control points, requiring the user to start over if a mistake is made
Allows for the user to change the curvature of the curve	Users can only select from a set of 7 pre-defined α values, limiting the precision one may require when adjusting the curvature of the curve
Allows for users to switch between open and closed curves	Curvature adjusts in jumps due to only being able to select from fixed values, diminishing the visual intuitiveness of manipulating the curve
Provides the ability to show/hide mathematical features such as **Q** points and curvature. This allows artists to only concern themselves with manipulating the curve without the mathematical detail, yet simultaneously enables Computer Scientists and Mathematicians to understand or further develop this class of curves.	The software exposes users to α values in order to change curvature, yet it is not immediately obvious how values of α impact the shape of the curve.

Table 2: Table of advantages and disadvantages of Salvi's [24] implementation of $\epsilon\kappa$-curves

2.4.3 Where does this project aim to improve?

While Salvi's implementation [24] achieves correct $\epsilon\kappa$-construction, its user interface requires direct adjustment if α-values through numeric inputs without visual guidance. This requires an understanding of curvature mathematics from the user, violating intuitive design principles (see Schneiderman and Nielsen's rules for interface design [18]). Consequently, non-expert users may find this implementation inefficient for manipulating curvature.

In this report, the aim is to improve upon all the disadvantages listed in Table 2, whilst also retaining some of the advantages. The output of this project is not concerned with additionally implementing closed curves, as the tool that will be proposed to improve on the disadvantages is easily transferable to such a construction. In addition, we have simply implemented a button to add control points rather than a left click on the canvas, as this was not a focus for the output of this project.

The primary improvements we intend to implement are:

i. Allow users to adjust the curvature of the curve locally (i.e. around individual control points)

ii. Create a more intuitive feel for adjusting curvature, moving away from simply selecting a value for α

iii. Allowing the user to adjust curvature without realising they are changing values of α

iv. Allow the user to see the curvature change continuously rather than in jumps

Whilst the function to delete control points has been implemented, the implementation of this feature is almost trivial and so was not a great focus for this project.

3 Implementation

3.1 Development Pipeline

In order to begin implementing an intuitive interface for adjusting curvature, it was first required to build $\epsilon\kappa$-curves. As seen in Section 2, these curves are built on strong foundations consisting of other classes of curves.

Taking inspiration from this, Bézier curves were initially implemented to gain an understanding of how they would integrate into $\epsilon\kappa$-curves. In addition, due to their simplicity, this implementation also served as a playground to test a variety of curvature visualisation techniques. As a result, this stage of implementation inspired the design of several concept tools for adjusting curvature.

Since $\epsilon\kappa$-curves are an extension of κ-curves, these were implemented next. After developing the code to solve for all $\tilde{t}_i$, $\tilde{\lambda}_i$ and $\tilde{\mathbf{Q}}_{i,j}$ (using equations (2.16), (2.17) and (2.20) respectively), the code from the previous Bézier implementation was adapted to construct κ-curves, by creating Bézier segments defined by the $\mathbf{Q}$ points rather than the user-controllable $\mathbf{P}$ points.

With the formulas developed in the more simple κ-curve system, the majority of the work for the $\epsilon\kappa$-curve system had already been realised. One requirement was to adjust the formulas for $\hat{\lambda}_i$ and $\hat{\mathbf{Q}}_{i,j}$ to match equations (2.26) and (2.32) respectively. In order to solve for $\hat{t}_i$, Salvi's Maxima [30] code [25] was used to produce the coefficients for each term in the polynomial of degree 9. The final adjustment was to adapt the Bézier segments to be cubic, rather than quadratic, as used in the κ-curve implementation.

Finally, with $\epsilon\kappa$-curves developed, it was possible to begin implementing the improvements listed in section 2.4.3. Throughout the stages of development, the ability to remove the most recent control point had been implemented due to the ease of adapting the feature to work with each class of curve. Prior to implementing an intuitive method for adjusting curvature, sliders were developed for each α value, thereby allowing users to: adjust curvature around individual control points; visualise continuous changes in curvature and have more precise control over curvature as a result of the wider range of α values available. To further enhance this, osculating circles were placed at each control point as a tool to manipulate curvature, providing the benefits of hiding α values from users and clearly indicating which control point curvature was being adjusted around.

3.2 Software Requirements and Libraries

Each class of curve was implemented in Python [31], due to being widely used for mathematical applications. Mathematics is simple to code in Python, in comparison to other languages such as C++ or Java [27] and is equipped with a variety of maths modules such as NumPy [11] and SymPy [19]. One downside of Python is that, for large systems, it can be slower than other languages [28]. However, as a result of theorems such as 2.2.1, numerically solving for values is efficient.

In addition to allowing for the experimentation of different visualisation techniques, the implementation of Bézier curves also demonstrated that some libraries would be preferable over others. The initial idea was to use Matplotlib [14] for the graphical user interface (GUI) due to its Pyplot interface for plotting functions as graphs. However, a requirement of the GUI is that it should allow for simple, intuitive human interaction. Moving control points using the PyPlot GUI provided dissatisfying results; moving a control point right would cause it to discontinuously jump from $x = 1$ to $x = 2$, limiting a user's choices when creating the curve. Instead, Tkinter [16] was chosen for the GUI, due to it's direct binding for handling events, thus making it easy to move control points and update the resultant curve as required. Tkinter is also simple to use, which is a useful aspect for testing [29, 10] different tools for adjusting curvature.

NumPy's linear algebra module was used to construct the linear systems presented in (2.20) and (2.32). In addition, NumPy arrays were used to store and manage all required parameters. However, recall that the matrix of coefficients is sparse, with a maximum of only three elements being non-zero in each row. As a result, the `spsolve` routine from SciPy [32] was used to solve the linear system.

Finally, for root finding. `brentq` [26] was used from the SciPy [32] library. This proved particularly useful, since we can specify an interval that is guaranteed to contain a root, ensuring accuracy and computational efficiency.

3.3 Creating the Tkinter GUI

The Tkinter GUI remained similar throughout the three different systems. In order to create an intuitive interface for adjusting curvature, it was essential that the basic features of the GUI were implemented to a high standard. To determine what is meant by 'high standard', the GUI implementation is based off of several literature regarding the guidelines for a system that implements direct manipulation. akob Nielsen's 10 Usability Heuristics [23], Shneiderman's Eight Golden Rules and Gestalt Principles of Visual Perception are discussed in [18].

Each system implemented required three key features: adding control points; removing control points and moving control points. In order to offer informative feedback and provide visibility of the systems status [18] , each system is required to respond in real time. Anytime one of the three actions above were performed, the resultant curve required re-drawing. To ensure this was the case, each of these actions contained the procedure `draw_curve`, which was defined differently depending on the class of curve implemented, and will be detailed in full in the proceeding sections. To add a control point, a fixed point is selected and appended to the list of control points, as seen in Algorithm 1 :

Algorithm 1 Adding control points

1: **procedure** ADD_CONTROL_POINT(control_points,...)
2: $new_point \leftarrow$ np.array([100, 100])
3: `control_points` $\leftarrow$ `control_points` $\cup$ `new_point`
4: $index \leftarrow$ length of $control_points - 1$
5: Bind drag event `B1-Motion` to `on_drag(event, index)` for object `circle`
6: ...
7: `draw_curve`()
8: **end procedure**

Where line 6 adds extra visual features depending on the system. Similarly, for deleting/moving control points:

Algorithm 2 Removing control points

1: **procedure** REMOVE_CONTROL_POINT(control_points,...)
2: **if** `length(control_points)`2 **then**
3: Delete the last circle from `control_circles` on the canvas
4: Remove the last element from `control_points`
5: Remove the last element from `control_circles`
6: ...
7: **end if**
8: `draw_curve`()
9: **end procedure**

Algorithm 3 Moving control points

1: **procedure** ON_DRAG(control_points,...)
2: Set the element at position *index* in `control_points` to $[event.x,\ event.y]$
3: Update the coordinates of the circle at *index* in `control_circles` to form a $a \times a$ square centered at $(event.x,\ event.y)$ on the canvas
4: ...
5: `draw_curve`()
6: **end procedure**

Note that the inclusion of Algorithm 2 permits easy reversal of actions [18]
The GUI also has the function to show and hide certain details, such as straight lines between control

points and osculating circles. This was included in each system to aid the creation of a minimalist design and reduce short term memory load [18]. The implementation of these features is relatively trivial and so the algorithms for these features are omitted.
In order to satisfy other GUI design heuristics, control points are always shown as red circles. During implementation, control points became difficult to manipulate, as a result of them being small so that they do not obstruct the curve. Furthermore, there were instances where additional features would overlap with control points, causing them to be difficult to move. To overcome each of these issues, an invisible hit box was developed around each control point, making them easier to drag and each control point was prioritised so that overlapping features did not interfere when attempting to move control points.

3.4 Bézier Curves

3.4.1 Key variables, Data Structures and Functions

Name	**Type**	**Mathematical Notation**	**Purpose in This System**
`control_points`	`np.ndarray[float], shape=(n, 2)`	$\{\mathbf{P}_i\}_{i=0}^{n+1}$	Stores n 2-D control points for defining Bézier Curve
`t_values`	`np.ndarray[float]`	$\{t_i\}$	Stores 100 evenly spaced values in $[0, 1]$ used to evaluate points on the Bézier curve.
`curve_point`	`np.ndarray[float], shape=(2,)`	$B_{i,n}(t)$	Stores the evaluated point on the Bézier curve at a given t.

Table 3: Key variables and data structures and their mathematical representations in the Bézier system.

Name	**Parameters**	**Purpose**
`bezier_curve`	`control_points`, `t`	Computes the point on a Bézier curve at parameter t using the Bernstein polynomial.
`draw_curve`	None	Clears and redraws the Bézier curve using the current control points and precomputed t values.
`draw_t_dots`	None	Draws evenly spaced dots on the Bézier curve at fixed t intervals to visualise curvature intuitively.
`curvature`	`control_points`, `t`	Computes the curvature at a point t on the curve using first and second derivatives.
`update_orange_dot_and_circle`	None	Calculates and draws the osculating circle to visualise curvature intuitively.

Table 4: Key functions used in the Bézier system.

3.4.2 Construction of Bézier curves

Recall from equation (2.8) that a Bézier curve is dependent on t. To display the Bézier curve in Tkinter, we calculate the position of each point on the curve at constant intervals of t for a given set of control points, as shown in Algorithm 4

Algorithm 4 Calculating points on Bézier curve

```
1: function BEZIER_CURVE(control_points, t)
2:     n ← length of control_points − 1
3:     Initialize curve_point as a zero vector of length 2
4:     for i, p in control_points do
5:         Compute the Bernstein polynomial: b ← BernsteinPolynomial(n, i, t)
6:         Add b · p to curve_point
7:     end for
8:     return curve_point
9: end function
```

In Algorithm 4, the function `BernsteinPolynomial`$(n,\ i,\ t)$ calculates the coefficient of control point $\mathbf{P}_i$ for a given value of t for a system with $n+1$ control points, as shown in equation (2.9). The `for` loop calculates the point on the Bézier curve for a given value of t by equation (2.8). In order to generate and display the Bézier curve in Tkinter, we repeat this process at set intervals of t, as shown in the `draw_curve` algorithm (Algorithm 5):

Algorithm 5 Drawing the Bézier curve

```
1: procedure DRAW_CURVE
2:     Delete all canvas objects with the tag "curve"
3:     t_values ← Generate 100 evenly spaced t values from 0 to 1
4:     bezier_points ← bezier_points(control_points, t) for t in t_values
5:     Flatten the list of Bézier points into a 1D list of coordinates
6:     Draw a blue curve on the canvas using these coordinates, with line width 2, and tag it as "curve"
7: end procedure
```

Note that generating 100 values for t in line 3 of Algorithm 5 is a deliberate choice that was made to balance visual smoothness with performance. A greater number of t values will improve visual smoothness, but increase the performance overhead.

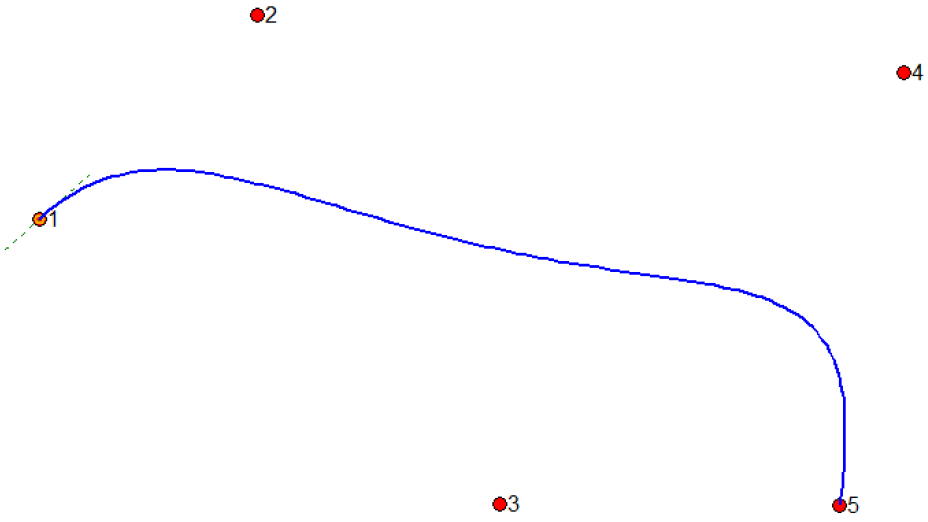


Figure 3.1: Bézier curve with 5 control points

In figure 3.2, one can observe the existence of an unintended cusp, between control points 5 and 4. One advantage with κ-curves and $\epsilon\kappa$-curves is that the creation of cusps is purely intentional. This will be shown later in this report.

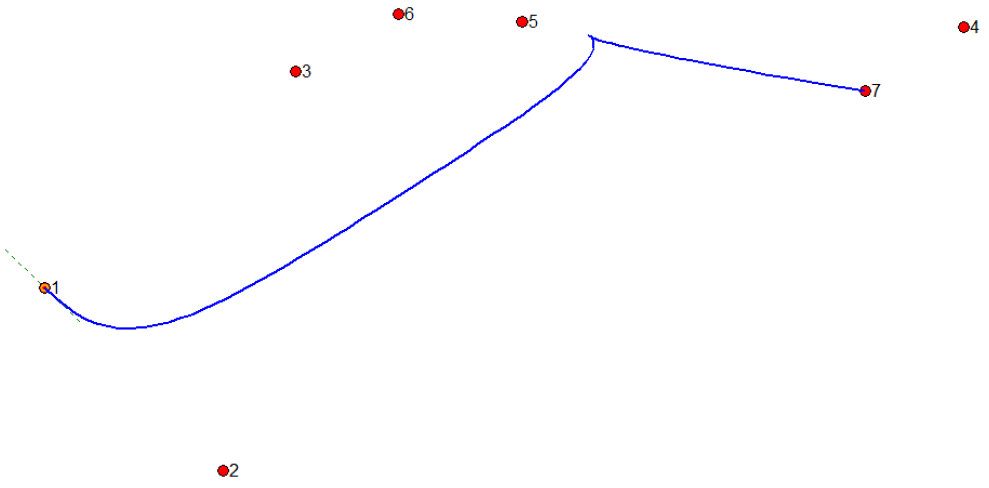


Figure 3.2: Bézier curve with unintended cusp

3.4.3 Construction of the osculating circle

As aforementioned, the Bézier curve system was used to test different visualisation techniques. Initially, the osculating circle was added to this system. In order for this feature to be implemented, it was required to calculate the curvature at a point t on the curve, $\kappa(t)$, and the normal to the curve at t. Fortunately, the system for generating Bézier curves contains a function for obtaining a point on the Bézier curve at a specific t value, namely `bezier_curve`(n, i, t). By using this function, we can obtain the x and y coordinates on the Bézier curve at t. However, by equation (2.29), the first and second derivative of the Bézier curve are also required. From equation (2.8), we get:

$$\frac{d\mathbf{P}(t)}{dt} = n\sum_{i=0}^{n-1}(\mathbf{P}_{i+1} - \mathbf{P}_i)B_{i,n-1}(t) \quad , \quad t \in [0,1] \tag{3.1}$$

$$\frac{d^2\mathbf{P}(t)}{dt^2} = n(n-1)\sum_{i=0}^{n-2}(\mathbf{P}_{i+2} - 2\mathbf{P}_{i+1} + \mathbf{P}_i)B_{i,n-2}(t) \quad , \quad t \in [0,1] \tag{3.2}$$

As a result, one can simply calculate the curvature at a point t using:

Algorithm 6 Calculating curvature at t

```
1: function CURVATURE(control_points, t)
2:     d1 ← first_derivative(control_points, t) (3.1)
3:     d2 ← second_derivative(control_points, t) (3.2)
4:     curvature ← Calculate curvature using equation (2.29)
5:     return curvature
6: end function
```

With curvature calculated in Algorithm 6, the only remaining requirement for constructing the osculating circle is to solve for the normal direction of the curve. It is necessary to find the unit normal to the curve at t, since the normal line serves as the radius of the osculating circle, which has length $1/\kappa(t)$. To find the unit normal at t, we simply approximate the tangent vector at t, divide by the norm of the tangent vector, and rotate the tangent vector 90° anticlockwise. Mathematically, we perform the following steps:

i. Approximate tangent vector:

$$\mathbf{T}(t) = \mathbf{P}(t+\delta) - \mathbf{P}(t) \quad , \quad \delta << 1 \tag{3.3}$$

ii. Normalise $\mathbf{T}(t)$:

$$\hat{\mathbf{T}}(t) = \frac{\mathbf{T}(t)}{||\mathbf{T}(t)||} \tag{3.4}$$

iii. Compute normal by rotation anti-clockwise:

$$\hat{\mathbf{n}}(t) = \begin{pmatrix} -\hat{\mathbf{T}}_y(t) \\ \hat{\mathbf{T}}_x(t) \end{pmatrix} \tag{3.5}$$

Further, for the osculating circle, we require the normal to point inwards. In order to do ensure this, one checks that:

$$\frac{d^2\mathbf{P}(t)}{dt^2} \cdot \hat{\mathbf{n}}(t) \geq 0 \tag{3.6}$$

If (3.6) is not satisfied, then we simply rotate the normal by 180° by taking the negative of the normal. This condition comes from [3]. In general, we have Proposition 1.

Proposition 1 (Normal Orientation Test)**.** *Let* $\mathbf{w} = \mathbf{Q} - \mathbf{P}$ *be a vector from a known interior point* $\mathbf{Q}$ *to a point* $\mathbf{P}$ *on a surface or curve (i.e* $\mathbf{w}$ *is a known normal), and let* $\mathbf{N}$ *be a candidate unit normal at* $\mathbf{P}$*. Then:*

$$\mathbf{w} \cdot \mathbf{N} > 0 \implies \mathbf{N} \textit{ is pointing inward}$$

$$\mathbf{w} \cdot \mathbf{N} < 0 \implies \mathbf{N} \textit{ is pointing outward}$$

The normal component of the second derivative points toward the centre of curvature i.e. the centre of the osculating circle. By taking the dot product with $\hat{\mathbf{n}}(t)$, we measure the signed curvature, with a positive value indicating that the normal points towards the centre of curvature.

In this implementation, a slider is used to determine the value of t. As the user moves this slider, the osculating circle updates in real time. With the above equations, one can now construct the osculating circle by Algorithm 7:

Algorithm 7 Drawing the Osculating Circle

procedure UPDATE_ORANGE_DOT_AND_CIRCLE
    $t \leftarrow$ `slider.get()`
    `orange_point` $\leftarrow$ `BezierCurve(control_points, t)`
    $\kappa \leftarrow$ `curvature(control_points, t)` ▷ By Algorithm 6
    radius $\leftarrow \infty$ if $\kappa = 0$ else `round`$(1/\kappa,\ 10)$ ▷ By Equation 2.12
    $d \leftarrow 0.01$
    `point_next` $\leftarrow$ `BezierCurve(control_points, t + d)`
    `direction_vector` $\leftarrow$ `point_next` $-$ `orange_point` ▷ By Equation 3.3
    Normalize `direction_vector` to have unit length ▷ By Equation 3.4
    `normal_vec` $\leftarrow [-$`direction_vector`$[1],$ `direction_vector`$[0]]$ ▷ By Equation 3.5
    if *radius* > 0 **and** *radius* is finite **then**
        $d_2 \leftarrow$ `BezierSecondDerivative(control_points, t)`
        if `dot(normal_vec,` $d_2) < 0$ **then**
            `normal_vec` $\leftarrow -$`normal_vec` ▷ Rotate normal vector to point inwards
        end if
        `circle_center` $\leftarrow$ `round(orange_point` $+$ *radius* $\cdot$ `normal_vec,` $10)$
    The following steps visually represent the osculating circle:
        Update the position of the osculating circle on the canvas using `circle_center` and *radius*
        Draw or update a line from `orange_point` to `circle_center` to represent the radius
        Position the label near the radius and update its text to $1/\kappa$
    end if
end procedure

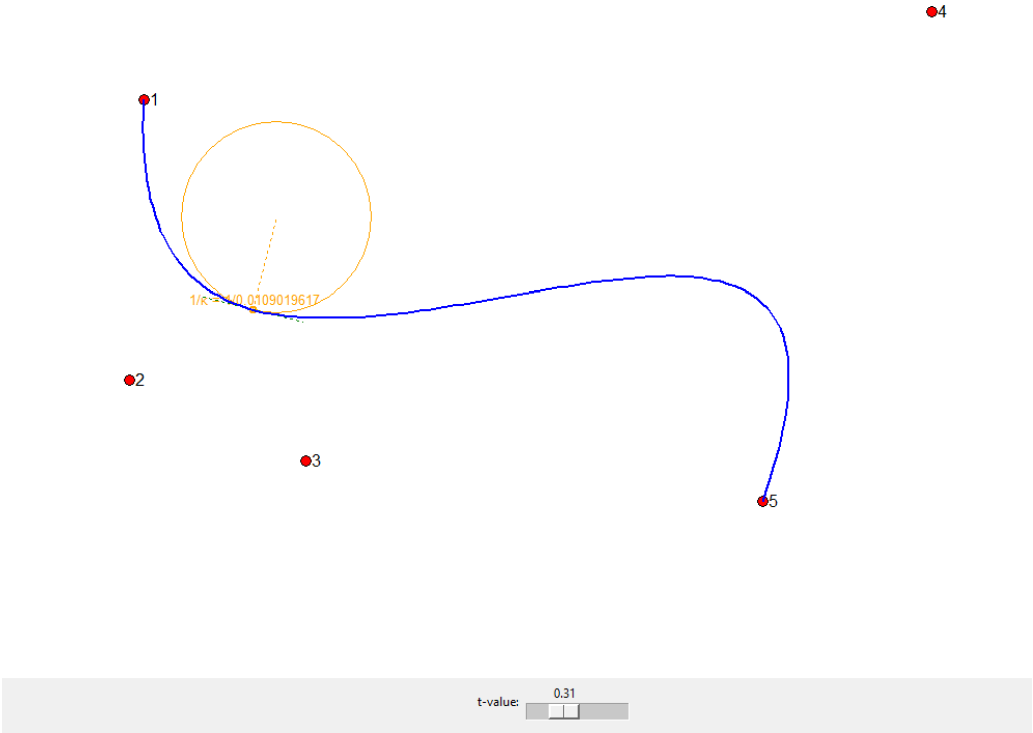


Figure 3.3: Osculating circle at $t = 0.31$

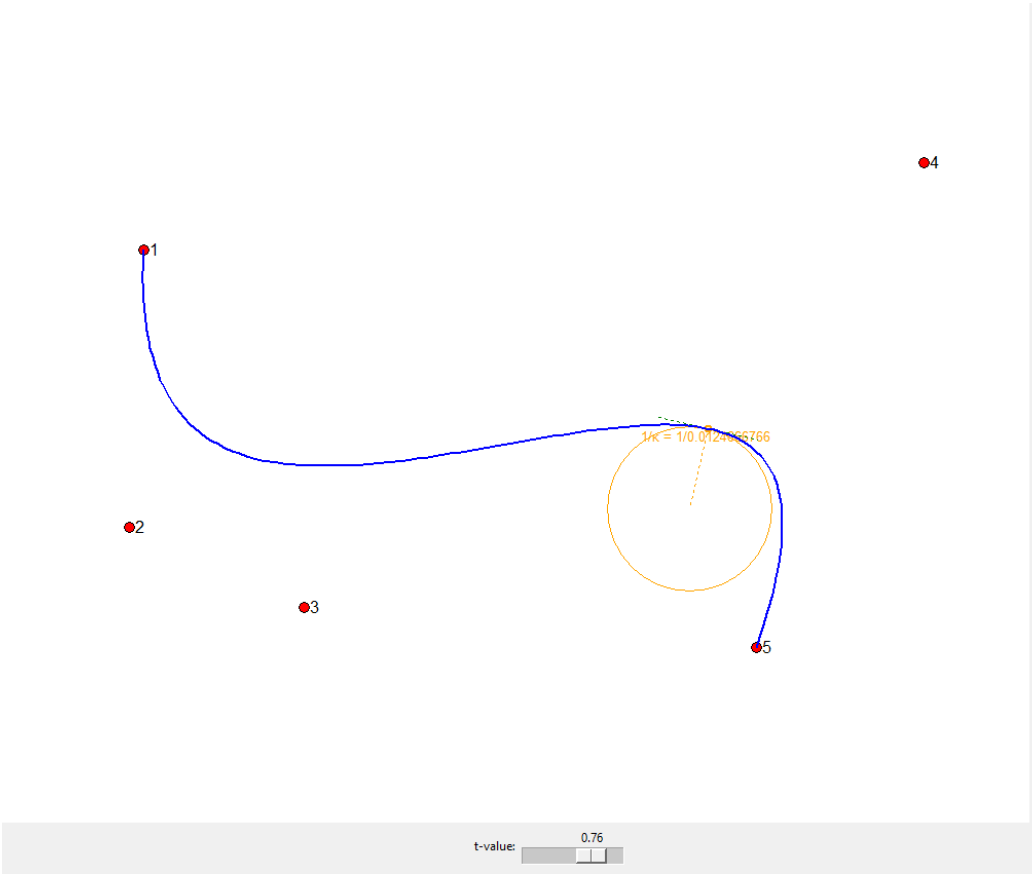


Figure 3.4: Osculating circle at $t = 0.76$

Aside Originally, it was decided to find the normal vector by using the Frenet normal vector [15] (see Equation (3.7))

$$\hat{\mathbf{n}}(t) = \frac{\ddot{\mathbf{P}}(t) - [\ddot{\mathbf{P}}(t) \cdot \dot{\mathbf{P}}(t)]\dot{\mathbf{P}}(t)}{||\ddot{\mathbf{P}}(t) - [\ddot{\mathbf{P}}(t) \cdot \dot{\mathbf{P}}(t)]\dot{\mathbf{P}}(t)||} \tag{3.7}$$

since the system already contained functions that calculate the first and second derivative. However, the use of equation 3.7 resulted in incorrect normals, as seen in Figure 3.5. This can happen if $||\dot{\mathbf{P}}(t)||$ is very small. As a result, the simpler method described above was implemented instead.

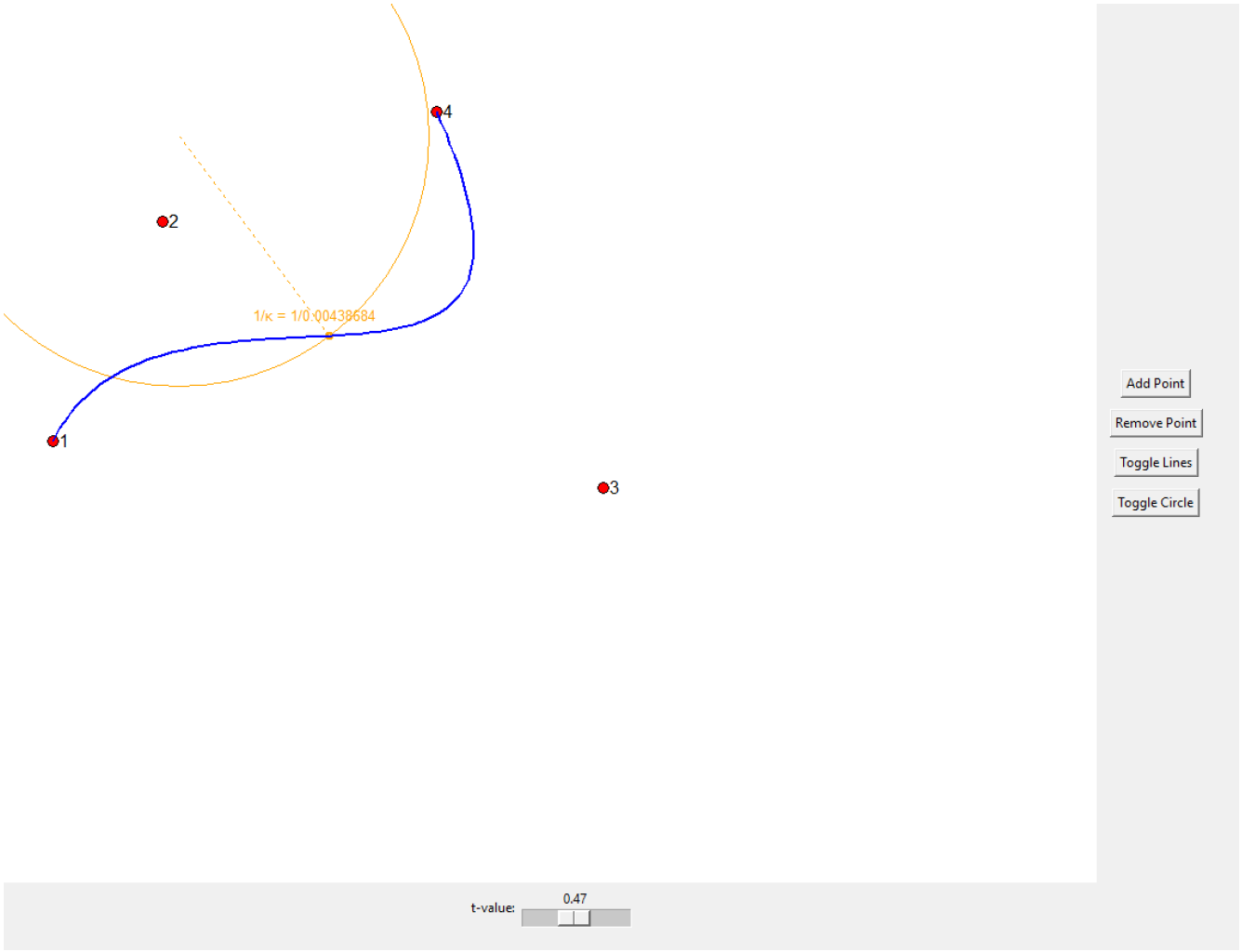


Figure 3.5: Osculating circle with incorrect normals

end aside

3.4.4 Dotting the curve

Another curvature visualisation technique implemented in the Bézier system was dotting the curve. This involves placing dots on the Bézier curve at fixed intervals of t. Due to the construction of the curve, dots that appear visually closer to their neighbouring dots represent greater curvature in comparison to dots that are further away from their neighbours (see Figure 3.6 for an example of this).

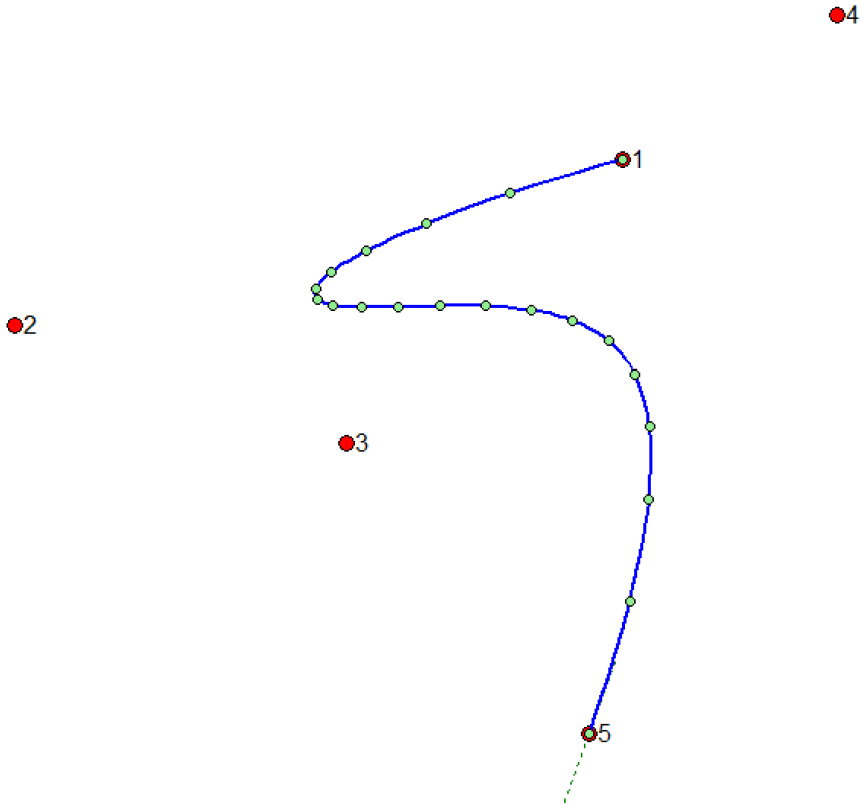


Figure 3.6: Dots placed on curve at equal intervals of t

As a result of the previously implemented `bezier_curve`(control_points,t) function in Algorithm 4, this feature was simple to develop. Initially, a fixed interval for t is set, and the point on the curve at each $t \in [0, 1]$ is calculated using the `bezier_curve`(control_points,t) function, as shown in Algorithm 8 :

Note that in line 3 of Algorithm 8, the value of `num_dots` is decided by the user. Figures 3.7 and 3.8 demonstrate how the user may adjust the quantity of dots on the curve.

Algorithm 8 Dotting the curve

1: **procedure** DRAW_T_DOTS
2: `num_dots` ← `t_dots_count.get()`
3: Generate t values evenly spaced from 0 to 1 with `num_dots` samples
4: **for** each t in the list of t values **do**
5: *point* ← `bezier_curve`(control_points,t)
6: Draw a small circle centered at `point` and tag it as `"t_dot"`
7: **end for**
8: **end procedure**

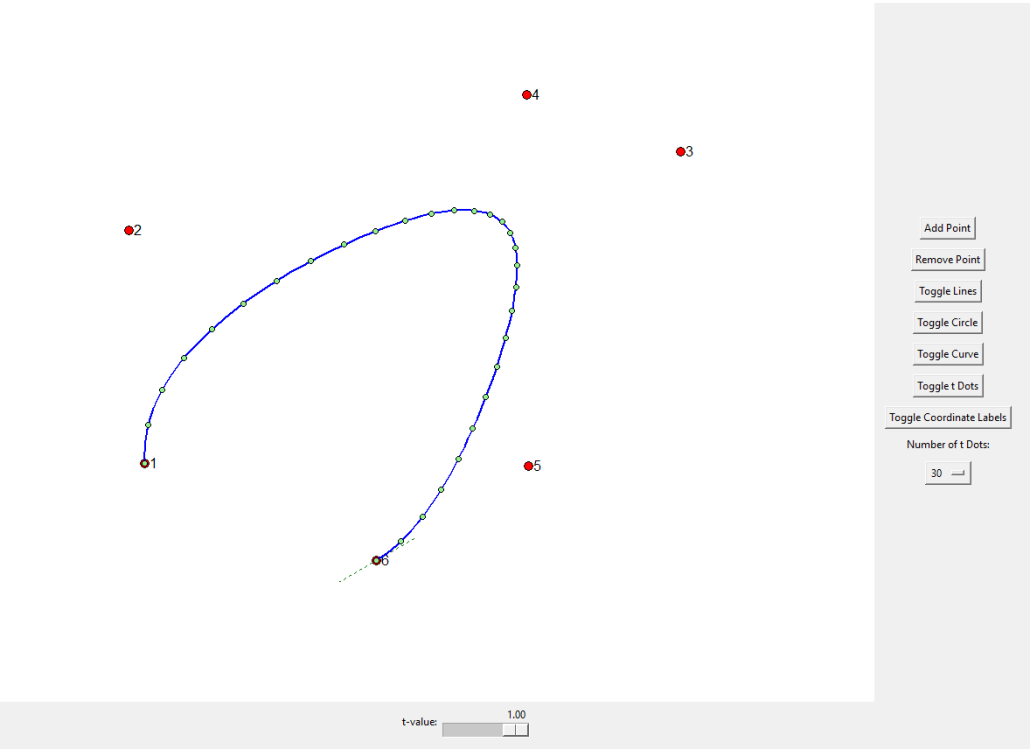


Figure 3.7: Bézier curve with 30 dots

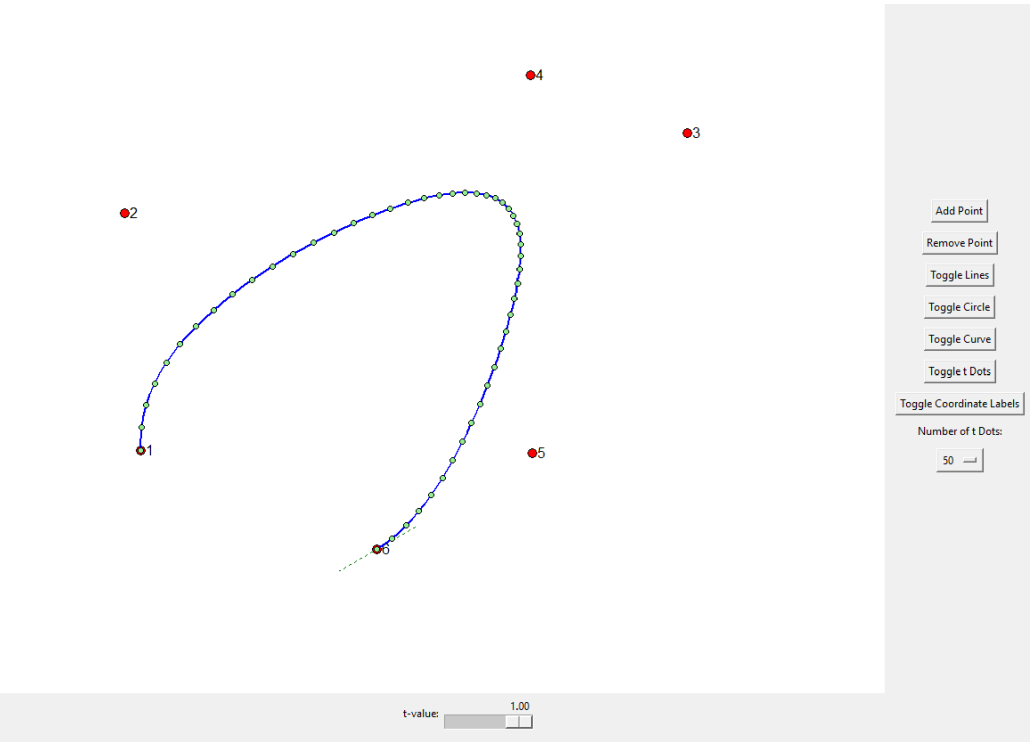


Figure 3.8: Bézier curve with 50 dots

3.4.5 Additional features

In order to retain the advantage of showing or hiding the mathematical aspects of the Bézier curve (see Table 2), several other features were added to the system alongside toggle buttons to allow the user to control what they see. The features that have not been mentioned previously include (x, y) coordinate labels for control points and straight lines between control points. Figures 3.9 and 3.10 show a Bézier curve with all additional features enabled against the same Bézier curve with no additional features. Note how Figure 3.9 is comparatively cluttered, which may draw the user's attention away from the Bézier curve and fail on Nielsen's minimalist design heuristic [18], hence reinforcing the requirement to toggle these features.

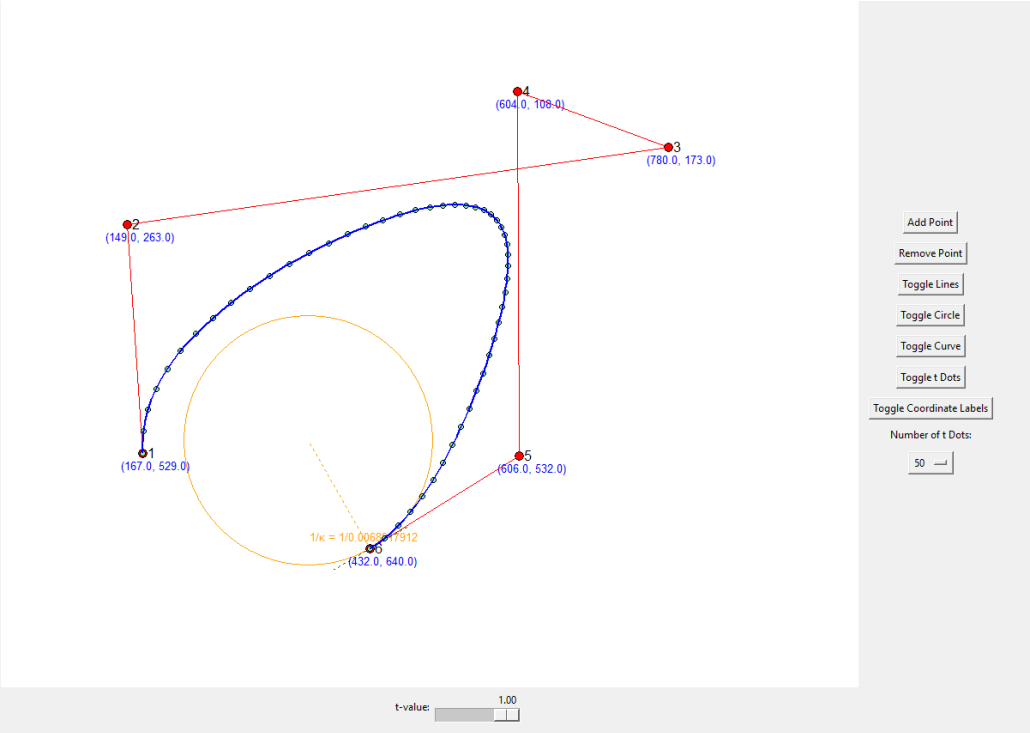


Figure 3.9: Bézier curve with all additional features

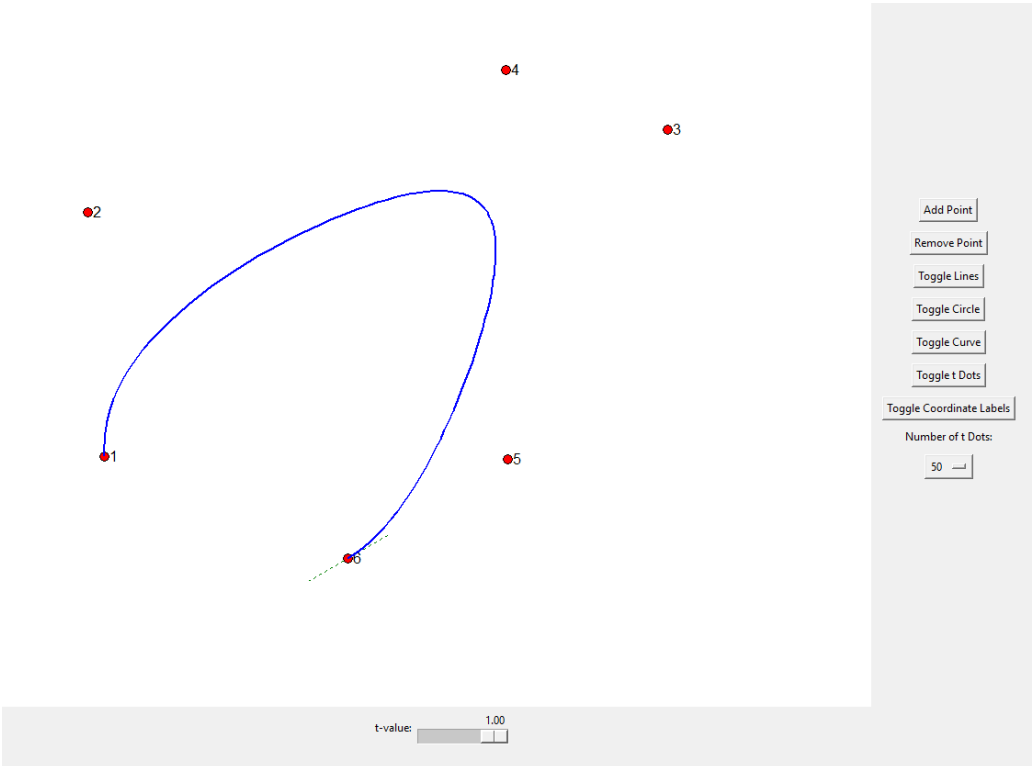


Figure 3.10: Bézier curve with no additional features

3.5 κ-Curves

3.5.1 Key variables, Data Structures and Functions

Name	Type	Mathematical Notation	Purpose in This System
`control_points`	`np.ndarray[float], shape=(n, 2)`	$\{\tilde{\mathbf{P}}_i\}_{i=0}^{\tilde{n}+1}$	Stores `n` 2-D control points for defining κ-curve
`t_values`	`np.ndarray[float], shape=(n-2,)`	$\{\tilde{t}_i\}_{i=1}^{\tilde{n}-1}$	Stores the t values calculated by Equation (2.16).
`lambda_values`	`np.ndarray[float], shape=(n-3,)`	$\{\tilde{\lambda}_i\}_{i=1}^{\tilde{n}-2}$	Stores the λ values calculated by Equation (2.17).
`Q_i_0_set`	`np.ndarray[float], shape=(n, 2)`	$\{\tilde{\mathbf{Q}}_{i,0}\}_{i=1}^{\tilde{n}-1}$	Stores the left curvature control points for each curve segment.
`Q_i_1_set`	`np.ndarray[float], shape=(n, 2)`	$\{\tilde{\mathbf{Q}}_{i,1}\}_{i=1}^{\tilde{n}-1}$	Stores the central control points, for each curve segment.
`Q_i_2_set`	`np.ndarray[float], shape=(n, 2)`	$\{\tilde{\mathbf{Q}}_{i,2}\}_{i=1}^{\tilde{n}-1}$	Stores the right curvature control points for each curve segment.

Table 5: Key variables and data structures and their mathematical representations in the κ system.

3.5.2 Overview

Algorithm 4 from the previous implementation provides a way of calculating the Bézier curve for any quantity of control points. As a result, these algorithms are integrated into the κ-curve system, but take

Name	Parameters	Purpose
`run_kappa_curve_optimization`	`control_points`, `iterations`	Main iterative function that updates $\tilde{t}_i$, $\tilde{\lambda}_i$, and $\widetilde{\mathbf{Q}}_{i,j}$ to generate κ-curves.
`t_i_update`	`control_points`, `t_values`, `Q_i_0_set`, `Q_i_2_set`	Solves the cubic equation (2.16) to update each $\tilde{t}_i$ value using the Brent root-finding method.
`lambda_i_update`	`lambda_values`, `Q_i_0_set`, `Q_i_1_set`, `Q_i_2_set`	Calculates $\tilde{\lambda}_i$ values by Equation (3.8)
`Q_i_j_update`	`control_points`, `t_values`, `lambda_values`, `Q_i_1_set`	Solves a sparse linear system to determine the middle control points $\widetilde{\mathbf{Q}}_{i,1}$ for each Bézier segment.
`draw_curve`	None	Draws the full κ-curve by plotting each Bézier segment defined by $\widetilde{\mathbf{Q}}_{i,0}, \widetilde{\mathbf{Q}}_{i,1}, \widetilde{\mathbf{Q}}_{i,2}$.

Table 6: Key functions used in the κ system.

the $\mathbf{Q}$ points as parameters. More specifically, the procedure `draw_curve` is implemented within a `for` loop, that iterates through each $\tilde{\mathbf{Q}}_{i,0}$. In each loop, `draw_curve` draws a Bézier curve defined by the points $\tilde{\mathbf{Q}}_{i,0}$, $\tilde{\mathbf{Q}}_{i,1}$ and $\tilde{\mathbf{Q}}_{i,2}$. Therefore, the primary objective for extending the Bézier implementation to one that outputs κ-curves is the calculation of the $\mathbf{Q}$ points.

It is important to note the slight, yet crucial, change in indexing. In the theory, $\tilde{n}$ is the number of control points minus two, since we label the first control point $\tilde{\mathbf{P}}_0$ and the final control point $\tilde{\mathbf{P}}_{\tilde{n}+1}$. However, in the implementation, `n` is the length of the array `control_points`. In addition, we have that `t_values[0]` is equivalent to $\tilde{t}_1$. This behaviour is the same with `lambda_values` and each $\tilde{\mathbf{Q}}$ set. However, note that `control_points[0]` is indeed referring to $\tilde{\mathbf{P}}_0$, but due to difference in `n` and $\tilde{n}$ above, `control_points[n]` is equivalent to $\tilde{\mathbf{P}}_{\tilde{n}+1}$. For the purpose of clarity, the following sections will continue to use `n` when referring to the implementation (code-level) variable, and $\tilde{n}$ when referring to the mathematical formulation.

Finally, in the algorithms below, we require certain values that appear to not yet have been obtained. However, section 3.5.6 shows the initialisation of these values prior to the use of each algorithm.

3.5.3 Solving for $\tilde{t}_i$

For a system with `n` control points, we require `n`-2 t values. In order to find each $\tilde{t}_i$, we use a `for` that loops `n`-2 times. Each loop initialises the variables required to compute Equation (2.16) from the corresponding array they are stored in. The polynomial (2.16) is solved for t using the `brentq`[26] method. By theorem 2.2.1, we restrict the search for a root to the interval $[0, 1]$ in order to improve the efficiency of the root finding algorithm. Algorithm 9 shows the pseudocode for solving for the values of each $\tilde{t}_i$.

Algorithm 9 Solving for $\tilde{t}_i$ in the κ-curve system

function T_I_UPDATE(*control_points*, *t_values*, *Q_i_0_set*, *Q_i_2_set*)
  $n \leftarrow$ length of *control_points* $- 2$
  for i from 0 to $n - 1$ **do**
    Q_i_0 $\leftarrow$ *Q_i_0_set*[i] ▷ To be used in `cubic_equation`
    Q_i_2 $\leftarrow$ *Q_i_2_set*[i] ▷ To be used in `cubic_equation`
    P_i $\leftarrow$ *control_points*[$i + 1$] ▷ To be used in `cubic_equation`
    t_initial_guess $\leftarrow$ `t_values`[i]
    result $\leftarrow$ Solve `cubic_equation` (2.16) on the interval [0, 1] using the Brent method.
    t_values[i] $\leftarrow$ round(result.root if result.converged else *t_initial_guess*, 8)
  end for
end function

3.5.4 Solving for $\tilde{\lambda}_i$

Solving for $\tilde{\lambda}_i$ caused some problems. This was primarily a result of the initially attempting to implement the formula (2.18) from [35], which was proven to be incorrect in the aside in section 2.2.5. However, once it was realised that (2.17) from [36] was the correct formula, the implementation proceeded with minimal issues. In contrast to solving for each $\tilde{t}_i$, only `n-3` $\tilde{\lambda}_i$ values are required. Recall that Equation (2.17) may result in an error if we divide by zero. In order to avoid this error, we introduce a small constant ϵ in the formula. As a result, Equation (2.17) becomes:

$$\tilde{\lambda}_i = \frac{\sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,0}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1})|}}{\sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,0}, \tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1})| + \epsilon} + \sqrt{|\Delta(\tilde{\mathbf{Q}}_{i,1}, \tilde{\mathbf{Q}}_{i+1,1}, \tilde{\mathbf{Q}}_{i+1,2})| + \epsilon}} \tag{3.8}$$

For this implementation, $\epsilon = 10^{-10}$ [36]. The function to calculate the value of each $\tilde{\lambda}_i$ begins by initialising each of the required $\tilde{\mathbf{Q}}$ points in the same way as Algorithm 9 and defining the Δ function. Then it is easy to solve Equation 3.8 (see Algorithm 10).

Algorithm 10 Solving for $\tilde{\lambda}_i$ in the κ-curve system

function LAMBDA_I_UPDATE(*lambda_values*, *Q_i_0_set*, *Q_i_1_set*, *Q_i_2_set*)
  Define *delta*(*Q*1, *Q*2, *Q*3) by Equation (2.15)
  Initialize empty lists: d_1, d_2
  $\epsilon \leftarrow 10^{-10}$
  $n \leftarrow$ length of *control_points* $- 3$
  for i from 0 to $n - 1$ **do**
    $a \leftarrow$ `delta`(*Q_i_0_set*[$i - 1$], *Q_i_1_set*[$i - 1$], *Q_i_1_set*[i])
    $b \leftarrow$ `delta`(*Q_i_1_set*[$i - 1$], *Q_i_1_set*[i], *Q_i_2_set*[i])
    Append a to d_1, and b to d_2 ▷ Stores required delta outputs for each $\tilde{\lambda}_i$ calculation in a array
  end for
  From the stored delta outputs, calculate numerator and denomiator for each $\tilde{\lambda}_i$ *calculation*
  numerator $\leftarrow \sqrt{d_1}$
  denominator $\leftarrow \sqrt{d_1} + \epsilon + \sqrt{d_2} + \epsilon$
  For each index i, update:
  $lambda_values[i] \leftarrow \dfrac{numerator_i}{denominator_i}$ ▷ Solve for each $\tilde{\lambda}_i$ by Equation (3.8)
  return *lambda_values*
end function

3.5.5 Solving for $\tilde{\mathbf{Q}}_{i,1}$

With Algorithms 9 and 10, it is now possible to compute each $\tilde{\mathbf{Q}}_{i,1}$. To implement the linear system in Equation (2.20), the algorithm begins by creating an `n-2`×`n-2` matrix of zeros, since the matrix of coefficients is an $\tilde{n} \times \tilde{n}$ and only three entries in each row are non-zero. To include these non-zero elements,

the algorithm must update the first row, last row and middles rows separately, since the coefficients in these rows are slightly different. For each middle row, the expressions are relatively the similar for the diagonal and off-diagonal entries, with the difference resulting from the choice of $\tilde{t}_i$ and $\tilde{\lambda}_i$. As a result, it is possible to create the matrix using a `for` loop, as shown in Algorithm 11.

Algorithm 11 Solving for $\tilde{\mathbf{Q}}_{i,1}$ in the κ-curve system

function Q_I_1_UPDATE($control_points, t_values, lambda_values, Q_i_1_set$)
  $n \leftarrow$ length of $control_points - 2$
  Initialize sparse matrix A of size $n \times n$
  Initialize matrix B of size $n \times 2$ filled with zeros
  for i from 0 to $n - 1$ **do**
    if $i = 0$ **then** ▷ First row of matrix
      $A[i, i] \leftarrow 2(1 - t_values[i])t_values[i] + (1 - lambda_values[i]) \cdot t_values[i]^2$
      $A[i, i + 1] \leftarrow lambda_values[i] \cdot t_values[i]^2$
      $B[i] \leftarrow control_points[i + 1] - (1 - t_values[i])^2 \cdot control_points[i]$
    else if $i = n - 1$ **then** ▷ Last row of matrix
      $A[i, i - 1] \leftarrow (1 - lambda_values[i - 1]) \cdot (1 - t_values[i])^2$
      $A[i, i] \leftarrow lambda_values[i - 1] \cdot (1 - t_values[i])^2 + 2(1 - t_values[i])t_values[i]$
      $B[i] \leftarrow control_points[i + 1] - t_values[i]^2 \cdot control_points[i + 2]$
    else ▷ Middle rows of matrix
      $A[i, i - 1] \leftarrow (1 - lambda_values[i - 1]) \cdot (1 - t_values[i])^2$ ▷ Left off-diagonal
      $A[i, i] \leftarrow lambda_values[i - 1] \cdot (1 - t_values[i])^2 + 2(1 - t_values[i])t_values[i] + (1 - lambda_values[i]) \cdot t_values[i]^2$ ▷ Diagonal
      $A[i, i + 1] \leftarrow lambda_values[i] \cdot t_values[i]^2$ ▷ Right off-diagonal
      $B[i] \leftarrow control_points[i + 1]$
    end if
  end for
  $Q_i_1_solution \leftarrow$ `spsolve`$(A,\ B)$
  Replace first n elements of $Q_i_1_set$ with $Q_i_1_solution$
  return $Q_i_1_set$
end function

In Algorithm 11, the matrix labelled A represents the matrix of coefficients and the matrix B represents the column vector on the right hand side of Equation (2.20).

3.5.6 The iterative process

Using Equations (2.16), (3.8) and (2.20), the middle $\mathbf{Q}$ points for constructing the Bézier curve segments have been calculated. In addition, we must ensure that each segment is connected to each other by (2.13). However, note that by satisfying one of Equations (2.16), (3.8), (2.20) or (2.13), may break the other three [35]. To demonstrate this, we look at an example.

> **Example** The linear system (2.20) is used to solve for each $\tilde{\mathbf{Q}}_{i,1}$, thereby upon calling Algorithm 11, the values of each $\tilde{\mathbf{Q}}_{i,1}$ change.
>
> Now note that Equation (3.8), which is used to solve for $\tilde{\lambda}_i$, is dependent on $\tilde{\mathbf{Q}}_{i,1}$. As a result, calculating each $\tilde{\mathbf{Q}}_{i,1}$ will change the value of each $\tilde{\lambda}_i$.

To solve this problem, the procedure of calculating each $\tilde{t}_i$,$\tilde{\lambda}_i$ and $\tilde{\mathbf{Q}}_{i,1}$ is iterated until converged [35]. Yan et al notes that this convergence occurs quickly, with just one iteration yielding similar results to the final solution, but with some maximum curvature points not yet coinciding with control points [35]. Algorithm 12 below provides the full algorithm for calculating κ-curves, as presented by Yan et al [35]:

Algorithm 12 Yan et al's [35] algorithm for constructing κ-curves

function GENERATE_KAPPA_CURVE(control_points)
    Initialize $lambda_values \leftarrow 0.5$
    Initialize $t_values \leftarrow 0.5$
    $Q_i_0_set \leftarrow control_points$
    for *iteration* from 1 to 60 **do**
        $t_values \leftarrow$ `t_i_update`$(control_points, t_values, Q_i_0_set, Q_i_2_set)$ ▷ By Algorithm 9
        $\lambda_values \leftarrow$ `lambda_i_update`$(lambda_values, Q_i_0_set, Q_i_1_set, Q_i_2_set)$ ▷ By Algorithm 10
        $Q_i_1_set \leftarrow$ `Q_i_j_update`$(control_points,\ t_values,\ \lambda_values,\ Q_i_1_set)$ ▷ By Algorithm 11
        $Q_i_2_set[i] \leftarrow (1 - lambda_values[i]) \cdot Q_i_1_set[i] + lambda_values[i] \cdot Q_i_1_set[i+1]$ ▷ By Equation (2.13)
    end for
    return tuples $\{(Q_i_0,\ Q_i_1,\ Q_i_2)\}$
end function

Yan et al's algorithm [35] produces the points required to construct each individual Bézier curve segment. Note that in the implementation for this report, the function `run_kappa_curve_optimization(control_points, iterations=60)` follows Algorithm 12.

3.5.7 Displaying the κ-curve

To display the curve in Tkinter, simple modifications are required for the `draw_curve` function (Algorithm 5). This function must now display multiple Bézier curves, but it can still follow the logic as seen previously. Algorithm 13 demonstrates the changes made.

Algorithm 13 Drawing the κ-curve

procedure DRAW_CURVE
    Delete all canvas objects with the tag "curve"
    ($Q_i_0_set$, $Q_i_1_set$, $Q_i_2_set$) $\leftarrow$ `run_kappa_curve_optimization`($control_points$, iterations = 60)
    $n \leftarrow$ length of $control_points - 2$
    $bezier_points \leftarrow$ `bezier_points`$(control_points, t)$ `for` t in t_values
    for i from 0 to $n - 1$ **do** ▷ For each Bézier segment
        $final_control_points \leftarrow$ `vstack`($Q_i_0_set[i]$, $Q_i_1_set[i]$, $Q_i_2_set[i]$)
        $t_values \leftarrow$ Generate 100 evenly spaced t values from 0 to 1
        $bezier_points \leftarrow$ `final_control_points`$(final_control_points, t)$
        Flatten the list of Bézier points into a 1D list of coordinates
        Draw a blue curve on the canvas using these coordinates and tag it as "curve"
    end for
end procedure

For the output of this system, see Figures 3.11 and 3.12.

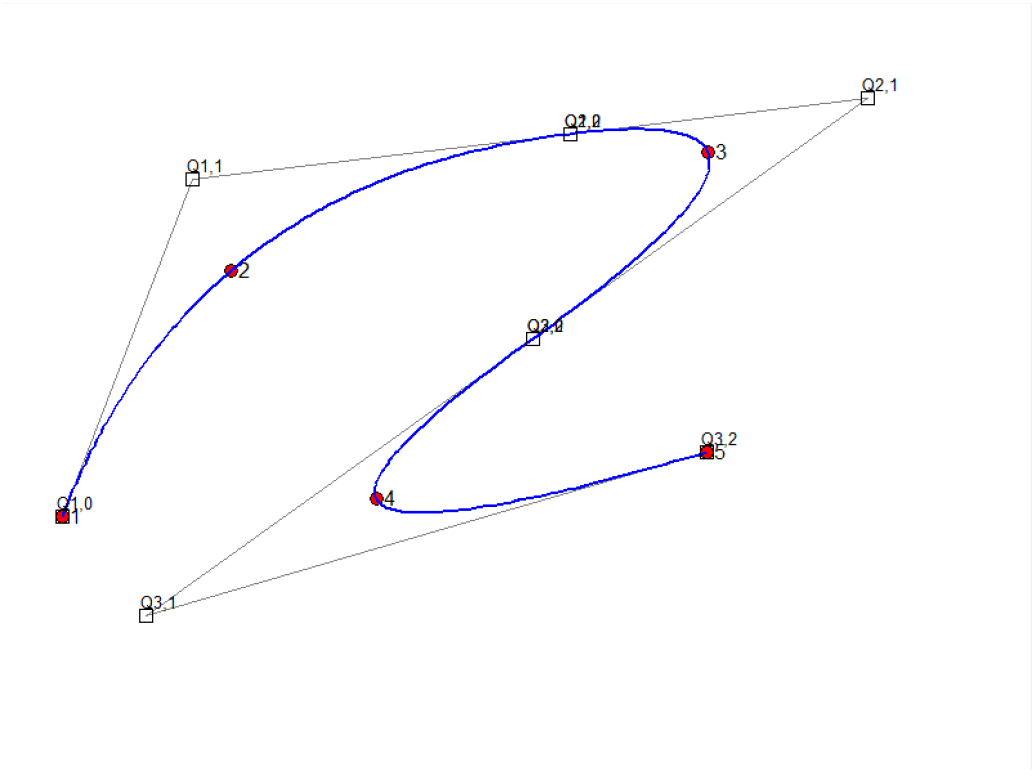


Figure 3.11: κ-curve with all additional features

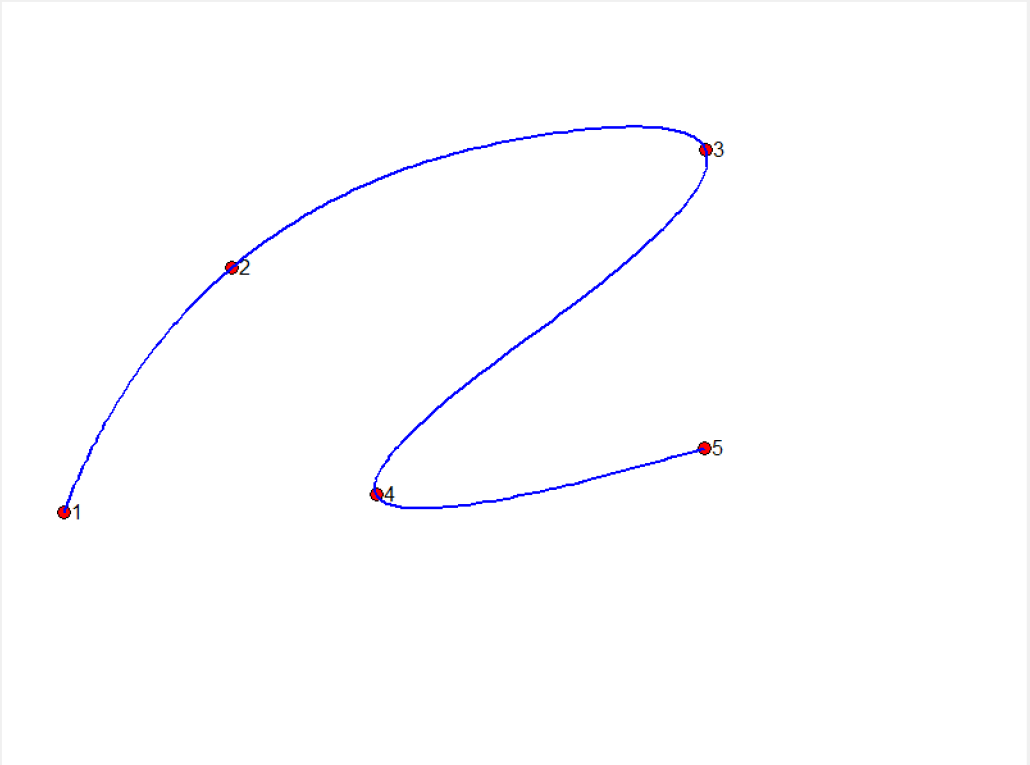


Figure 3.12: κ-curve with no additional features

3.6 $\epsilon\kappa$-Curves

3.6.1 Key variables, Data Structures and Functions

Name	Type	Mathematical Notation	Purpose
`control_points`	`np.ndarray[float], shape=(n, 2)`	$\{\tilde{\mathbf{P}}_i\}_{i=0}^{\tilde{n}+1}$	Stores `n` 2-D control points for defining κ-curve
`Q_i_0_set`	`np.ndarray[float], shape=(n, 2)`	$\{\hat{\mathbf{Q}}_{i,0}\}$	Stores left control points of each cubic segment.
`Q_i_1_set`	`np.ndarray[float], shape=(n, 2)`	$\{\hat{\mathbf{Q}}_{i,1}\}$	Stores central control points of each cubic segment.
`Q_i_2_set`	`np.ndarray[float], shape=(n, 2)`	$\{\hat{\mathbf{Q}}_{i,2}\}$	Stores right control points of each cubic segment.
`t_values`	`np.ndarray[float], shape=(n,)`	$\{\hat{t}_i\}$	Stores the internal t values solved using Brent's method.
`lambda_values`	`np.ndarray[float], shape=(n-1,)`	$\{\hat{\lambda}_i\}$	Stores local curvature blending weights for each segment.
`alpha_array`	`np.ndarray[float], shape=(n,)`	$\{\alpha_i\}$	User-defined curvature control values assigned per control point.
`coeffs`	`List[float]`	–	Stores the coefficients of the degree-9 polynomial generated by Salvi's code.

Table 7: Key variables and data structures and their mathematical representations in the $\epsilon\kappa$ system.

Name	Parameters	Purpose
`polynomial_cubic`	$x0$, $y0$, $x2$, $y2$, px, py, a	Evaluates the cubic polynomial of degree 9 with coefficients from Salvi's Maxima code. [24]
`find_root_cubic`	$x0$, $y0$, $x2$, $y2$, px, py, a	Solves the degree-9 polynomial in $[0, 1]$ using Brent's method.
`t_i_update_cubic`	`Q_i_0_set`, `Q_i_1_set`, `Q_i_2_set`, `alpha_array`	Solves for all $\hat{t}_i$ values
`lambda_i_update`	`Q_i_0_set`, `Q_i_1_set`, `Q_i_2_set`, `alpha_array`	Computes all $\hat{\lambda}_i$ values using Equation (2.26) with the constant $\epsilon = 10^{-10}$
`Q_i_1_update`	`Q_i_0_set`, `Q_i_1_set`, `Q_i_2_set`, `alpha_array`, `control_points`, `t_values`, `lambda_values`	Constructs and solves a linear system to obtain $\hat{\mathbf{Q}}_{i,1}$ points.
`run_epsilon_kappa_curve_optimization`	`control_points`, `iterations`	Full iterative procedure to calculate all Q-sets for rendering the final $\epsilon\kappa$-curve.

Table 8: Key functions used in the $\epsilon\kappa$ system.

3.6.2 Overview

The structure developed for κ-curves serves as a natural framework for extending to $\epsilon\kappa$-curves. Whilst $\epsilon\kappa$-curves consist of cubic Bézier curves, recall that one can represent the middle control points by Equations (2.22) and (2.23). Hence, to adapt the κ-curve implementation, one is only required to change the functions `t_i_update`, `lambda_i_update` and `Q_i_j_update` to coincide with the equations noted in section 2.3. Furthermore, the implementation differs slightly from Miura et al's work [22], since in this project's implementation, a different value of α is allowed at each user control point. As a result, each algorithm must use the corresponding value `alpha_array[i]`, rather than a single α for each calculation.

3.6.3 Solving for $\hat{t}_i$

In order to update the function to solve for each $\hat{t}_i$, one must first construct the polynomial of degree 9. Fortunately, Salvi's Maxima [30] code [25] produces the coefficients for each t^m , $m \in \{0, 1, ..., 9\}$. The code for this can be found in Figure 3.13.

```
/*
 Input:
   (x0,y0) and (x2,y2) - the endpoints
   (px,py)             - the point to interpolate at the maximal curvature
   a                   - alpha, the ratio for the conversion between quadratic & cubic
 Output:
   <a 9th-degree polynomial in t, having one real solution in [0,1]>
 */

/* (cx,cy) is the curve, (dx,dy) and (ddx,ddy) are the 1st and 2nd derivatives */
cx: (1-t)^3*x0+3*(1-t)^2*t*((1-a)*x0+a*x1)+3*(1-t)*t^2*((1-a)*x2+a*x1)+t^3*x2$
cy: (1-t)^3*y0+3*(1-t)^2*t*((1-a)*y0+a*y1)+3*(1-t)*t^2*((1-a)*y2+a*y1)+t^3*y2$
dx: diff(cx,t)$
dy: diff(cy,t)$
ddx: diff(dx,t)$
ddy: diff(dy,t)$

/* n and d are the numerator and denominator of the curvature, respectively */
n: dx*ddy-ddx*dy$
d: (dx^2+dy^2)^(3/2)$

/* The numerator of the curvature's derivative; we need to solve dk = 0 */
dk: diff(n,t)*d-n*diff(d,t)$
/* Looking at factor(dk), we can see that there is some room for simplification */
dk1: factor(dk/(162*a*(x1*y2-x0*y2-x2*y1+x0*y1+x2*y0-x1*y0)*sqrt(dx^2+dy^2)))$
solution: rhs(solve(dk1,t)[1])$

/* (x1,y1) is set s.t. the curve interpolates (px,py) */
x1: (px-((1-t)^3+3*(1-t)^2*t*(1-a))*x0-(3*(1-t)*t^2*(1-a)+t^3)*x2)/(3*(1-t)*t*a)$
y1: (py-((1-t)^3+3*(1-t)^2*t*(1-a))*y0-(3*(1-t)*t^2*(1-a)+t^3)*y2)/(3*(1-t)*t*a)$

/* Generate a string representation that can be inserted in a program */
display2d: false$ /* programming-friendly output */
collectterms(expand(num(xthru(ev(solution,x1=x1,y1=y1)))),t);
```

Figure 3.13: Maxima code for generating polynomial coefficients. Image adapted from Miura et al. [22] for clarity; code by Salvi [25]. Licensed under CC BY 4.0 (see http://creativecommons.org/licenses/by/4.0/).

In Salvi's implementation of $\epsilon\kappa$-curves [24], an array was used to store the coefficients. This same structure was adopted in the Python implementation presented in this report, by directly copying the array of coefficients from Salvi's code, which is licensed under CC BY 4.0, as shown in Algorithm 14.

Algorithm 14 Polynomial of degree 9

function POLYNOMIAL_CUBIC($x0,\ y0,\ x2,\ y2,\ px,\ py,\ a$)
    $coeffs \leftarrow [\cdots]$ ▷ $[\cdots]$ refers to the array containing all coefficients [24]
    $val \leftarrow 0$
    for each index i and coefficient c in $coeffs$ **do**
        $val \leftarrow val + c \cdot t^i$ ▷ Create polynomial
    end for
    return val
end function

where the parameters are as indicated in Figure 3.13.

With a function to generate the polynomial of degree 9, it is now possible to solve for each $\hat{t}_i$. Once again, we use `brentq` to solve the polynomial, restricting the search for a root to the interval $[0, 1]$ as stated in Miura et al [22, p. 5] (see Algorithm 15).

Algorithm 15 Solving for $\hat{t}_i$ in the $\epsilon\kappa$-curve system

function FIND_ROOT_CUBIC($x0,\ y0,\ x2,\ y2,\ px,\ py,\ a$)
    Define $f(t)$ as `polynomial_cubic`($t,\ x0,\ y0,\ x2,\ y2,\ px,\ py,\ a$)
    Attempt to return `brentq`($f,\ 0,\ 1$)
        ▷ If an exception occurs, return 0.5
end function

Finally, to update $\hat{t}_i$, Algorithm 16 is required:

Algorithm 16 Updating $\hat{t}_i$ values in the $\epsilon\kappa$-curve system

function T_I_UPDATE_CUBIC(Q_i_0_set, Q_i_1_set, Q_i_2_set, alpha_array)
    $n \leftarrow$ length of $control_points - 2$
    Initialize $t_values \leftarrow []$
    for i from 0 to $n - 1$ **do**
        $(x0,\ y0) \leftarrow Q_i_0_set[i]$
        $(x2,\ y2) \leftarrow Q_i_2_set[i]$
        $(px,\ py) \leftarrow Q_i_1_set[i]$
        $ai \leftarrow alpha_array[i]$
        $sol \leftarrow$ `find_root_cubic`($x_0,\ y_0,\ x_2,\ y_2,\ p_x,\ p_y,\ a_i$)
        Append sol to t_values
    end for
    return t_values as array
end function

3.6.4 Solving for $\hat{\lambda}_i$

Since the equation to solve for $\tilde{\lambda}_i$ in the κ-curve system is similar to the equation for solving $\hat{\lambda}_i$ in the $\epsilon\kappa$-curve system, we simply just adjust the formula in Algorithm 10 to match Equation (2.26) with the constant $\epsilon = 10^{-10}$ included using the same logic as in Equation (3.8).

Algorithm 17 Solving for $\hat{\lambda}_i$ in the $\epsilon\kappa$-curve system

function LAMBDA_I_UPDATE(Q_i_0_set, Q_i_1_set, Q_i_2_set, a)
    $n \leftarrow$ length of $control_points - 3$
    Initialize $lam \leftarrow np.zeros(n-1)$
    for i from 0 to $n-1$ **do**
        $tmp \leftarrow \sqrt{(1-a[i]) * \texttt{delta}(Q_i_0_set[i],\ Q_i_1_set[i],\ Q_i_1_set[i+1])}$
        $denom \leftarrow tmp + \frac{a[i]}{alpha_array[i+1]} \sqrt{(1-a[i+1]) * \texttt{delta}(Q_i_1_set[i],\ Q_i_1_set[i+1],\ Q_i_2_set[i+1])}$
        $lam[i] \leftarrow \dfrac{tmp}{denom}$
    end for
    return lam
end function

Note that the full calculation for each $\hat{\lambda}_i$ has been moved inside the `for` loop, rather than using arrays to calculate each value as in Algorithm 10. This is a deliberate choice in order to handle the α values more easily.

3.6.5 Solving for $\hat{\mathbf{Q}}_{i,j}$

In equation (2.32), the functions $\mathcal{L}(t)$, $\mathcal{M}(t)$ and $\mathcal{R}(t)$ were introduced in order to more easily represent the matrix of coefficients for the $\epsilon\kappa$-curve system. Hence, to solve for each $\hat{\mathbf{Q}}_{i,j}$ in the Python implementation, the algorithm begins by defining each of these functions. Following the same logic as in Algorithm 11, the matrix of coefficients is first initialised with zeros, after which its tridiagonal entries are updated accordingly. Note that in Algorithm 18, the parameter names have been shortened to provide more clarity in the pseudocode.

During implementation, this stage caused some issues, as the linear system (2.32) was not immediately obvious from Miura et al's publication [22]. In order to formulate the linear system, inspiration was taken from the system for κ-curves in Yan et al's work [35].

Algorithm 18 Solving for $\hat{\mathbf{Q}}_{i,1}$ in the $\epsilon\kappa$-curve system

function Q_I_1_UPDATE($Q_i_0_set, Q_i_1_set, Q_i_2_set, lam, t, control_points, a_array$)
  $n \leftarrow$ length of $control_points - 2$
  Initialize sparse matrix A of size $n \times n$
  Initialize matrix B of size $n \times 2$ filled with zeros
  function LEFT_BASIS($t,\ a$)
    return $(1-t)^3 + 3(1-t)^2 \cdot t \cdot (1-a)$
  end function
  function RIGHT_BASIS($t,\ a$)
    return $t^3 + 3(1-t) \cdot t^2 \cdot (1-a)$
  end function
  function MIDDLE_BASIS($t,\ a$)
    return $3a \cdot (1-t) \cdot t$
  end function
  for i from 0 to $n-1$ **do**
    $t \leftarrow t[i]$
    $a_i \leftarrow a_array[i]$
    if $i > 0$ **then**
      $A[i, i-1] \leftarrow (1 - lam[i-1]) \cdot$ `left_basis`$(t,\ a_i)$ ▷ Middle row left off-diagonal
    else
      $Q0_0 \leftarrow Q_i_0_set[0]$
      $fixed[i] \leftarrow fixed[i] - Q0_0 \cdot$ `left_basis`$(t,\ a_i)$ ▷ Update matrix B
    end if
    if $i < n-1$ **then**
      $A[i, i+1] \leftarrow lam[i] \cdot$ `right_basis`$(t,\ a_i)$ ▷ Middle rows right off-diagonal
    else
      $Qn_2 \leftarrow Q_i_2_set[n-1]$
      $fixed[i] \leftarrow fixed[i] - Qn_2 \cdot$ `right_basis`$(t,\ a_i)$ ▷ Update matrix B
    end if
    if $n = 1$ **then** ▷ Update second row diagonal
      $A[i, i] \leftarrow$ `middle_basis`$(t,\ a_i)$
    else if $i = 0$ **then** ▷ Update first row diagonal
      $A[i, i] \leftarrow$ `middle_basis`$(t,\ a_i) + (1 - lam[i]) \cdot$ `right_basis`$(t,\ a_i)$
    else if $i = n-1$ **then** ▷ Update final row diagonal
      $A[i, i] \leftarrow lam[i-1] \cdot$ `left_basis`$(t,\ a_i) +$ `middle_basis`$(t,\ a_i)$
    else ▷ Update middle rows diagonal
      $A[i, i] \leftarrow lam[i-1] \cdot$ `left_basis`$(t,\ a_i) +$ `middle_basis`$(t,\ a_i)$
        $+(1 - lam[i]) \cdot$ `right_basis`$(t,\ a_i)$
    end if
  end for
  $B \leftarrow control_points + fixed$ ▷ Finalise matrix B
  return `solve`$(A,\ B)$
end function

3.6.6 The iterative process

As with the κ-curve system, it is required to iterate through Algorithms 15, 17 and 18 until convergent. Algorithm 19 has foundations from Miura et al's pseudocode [22] for the entire iterative process. The modifications made are simply a change in notation for consistency in this report, and the inclusion of α values to allow for a different α at each control point.

Algorithm 19 Constructing $\epsilon\kappa$-curves, based on Miura et al's psuedocode [22]

function RUN_EPSILON_KAPPA_CURVE_OPTIMIZATION(control_points)
    $n \leftarrow$ length of $control_points - 2$
    $Q_i_0_set[0] \leftarrow control_points[0]$ ▷ $\hat{\mathbf{Q}}_{1,0} = \hat{\mathbf{P}}_0$
    $Q_i_2_set[-1] \leftarrow control_points[-1]$ ▷ $\hat{\mathbf{Q}}_{\hat{n},2} = \hat{\mathbf{P}}_{\hat{n}+1}$
    for $i \leftarrow 1$ to $n - 2$ **do**
        $Q_i_1_set[i] \leftarrow control_points[i + 1]$ ▷ $\hat{\mathbf{Q}}_{i,1} = \hat{\mathbf{P}}_i$
    end for
    Initialize $lambda_values \leftarrow 0.5$ for all i (length $n - 1$)
    Initialize $t_values \leftarrow 0.5$ for all i (length n)
    $alpha_array \leftarrow$ `get_alpha_array`() ▷ Gets alpha values from user input
    for each iteration from 1 to 60 **do**
        $lambda_values \leftarrow$ `lambda_i_update`($Q_i_0_set$, $Q_i_1_set$, $Q_i_2_set$) ▷ Algorithm 17
        `update_Q_i_0_Q_i_2`($Q_i_0_set$, $Q_i_1_set$, $Q_i_2_set$, $lambda$) ▷ Eq. 2.25
        $t_values \leftarrow$ `t_i_update_cubic`($Q_i_0_set$, $Q_i_1_set$, $Q_i_2_set$, $alpha_array$) ▷ Algorithm 16
        $new_Q_i_1 \leftarrow$ `compute_central_cps_cubic`(
         $Q_i_0_set$, $Q_i_1_set$, $Q_i_2_set$,
         $lambda_values$, t_values, $control_points[1: -1]$, $alpha_array$) ▷ Algorithm 18
        $Q_i_1_set \leftarrow new_Q_i_1$
    end for
    return $Q_i_0_set$, $Q_i_1_set$, $Q_i_2_set$
end function

In order to draw the curve, the same `draw_curve` procedure can be used from the κ-curves implementation, but it requires small adaptations due to function names. In addition, it must be updated to get the corresponding α value in each iteration of the `for` loop.

Figures 3.14 and 3.15 show the $\epsilon\kappa$-curve system and the sliders used to manipulate curvature, prior to the implementation of intuitive controls.

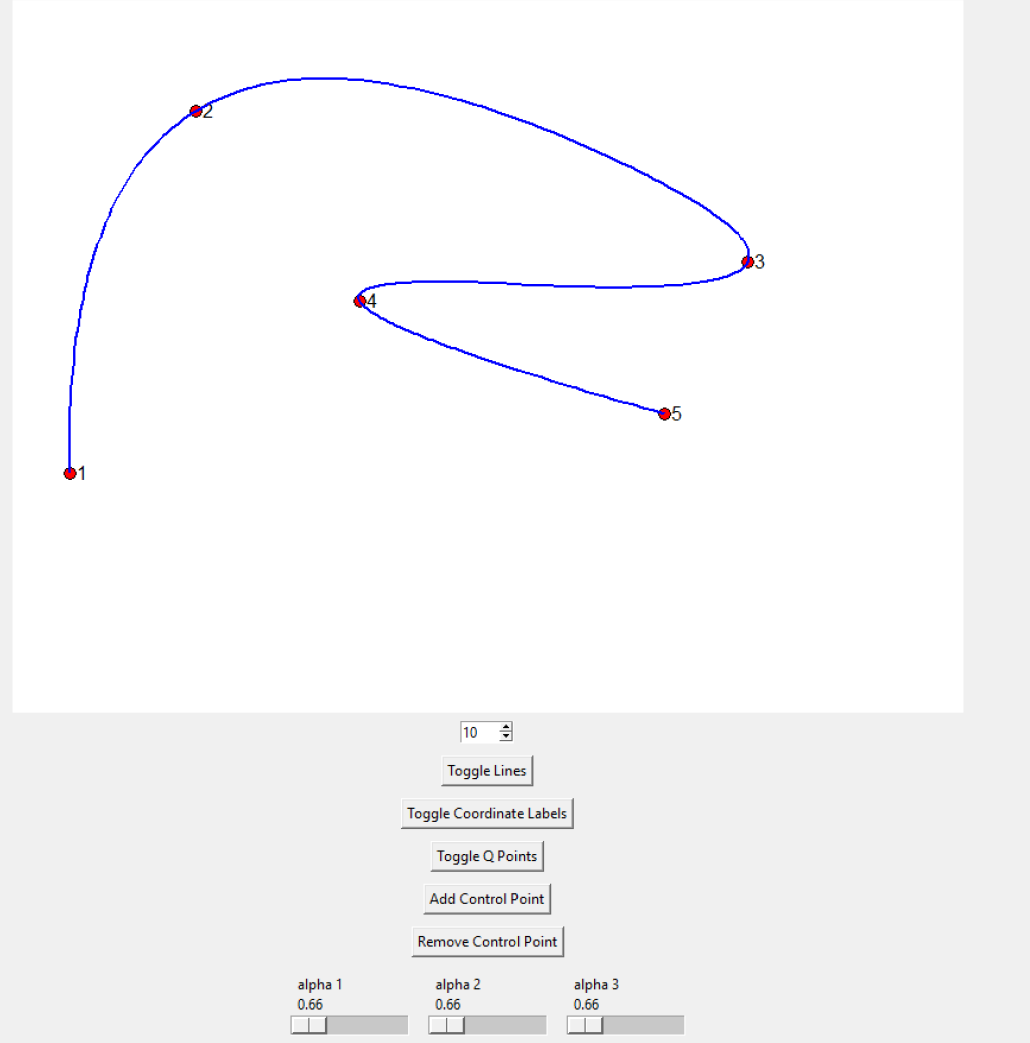


Figure 3.14: $\epsilon\kappa$-curve with `alpha_array=[0.66,0.66,0.66]`

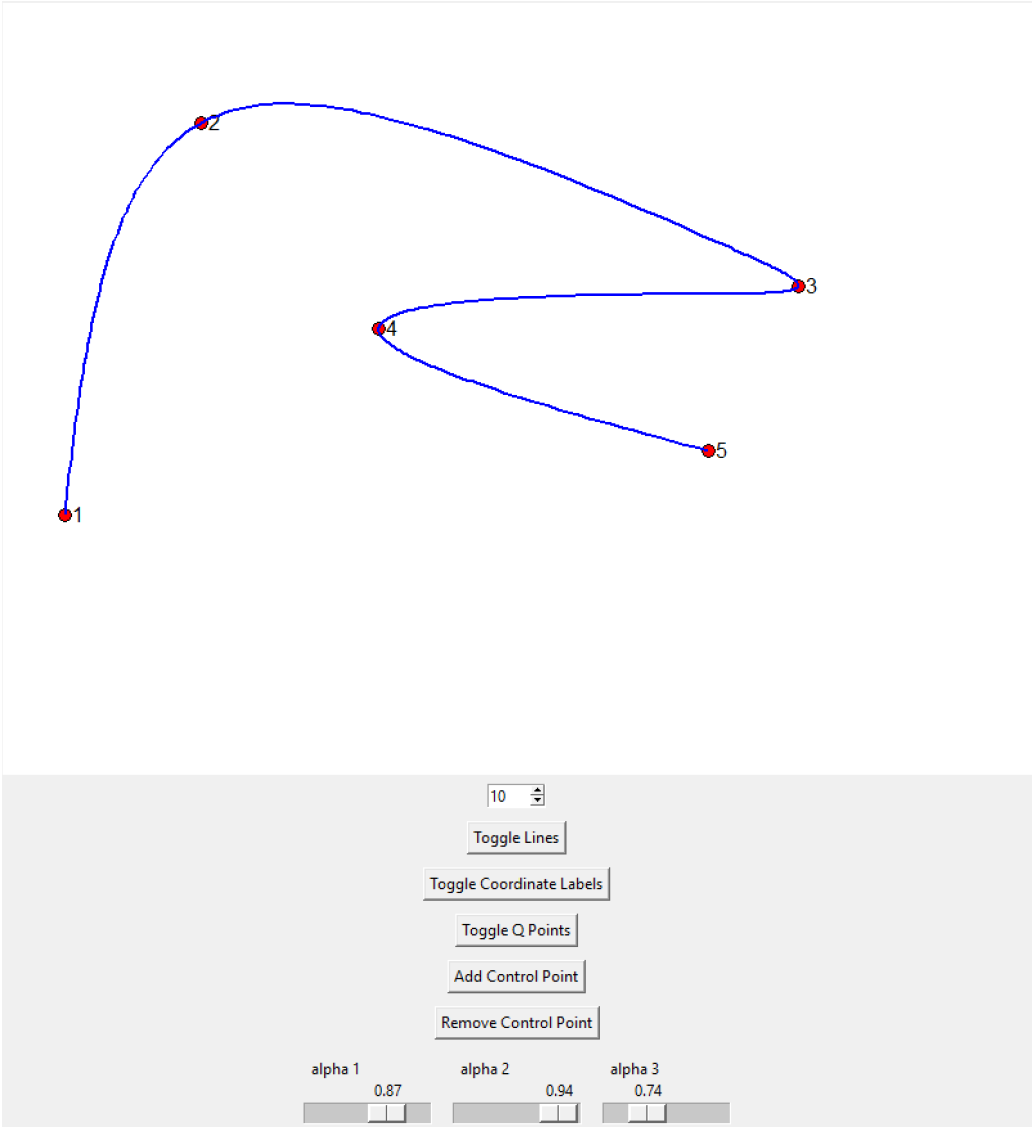


Figure 3.15: $\epsilon\kappa$-curve with `alpha_array=[0.87,0.94,0.74]`

3.6.7 Adjusting α intuitively

The implementation displayed in Figures 3.14 and 3.15 uses sliders to allow the user to adjust curvature. Whilst this improves upon the discontinuous jumps in curvature, it still leaves some issues that need to be resolved. Namely:

i. The user is still exposed to α values

ii. Curvature is controlled by interactions made off the curve

iii. It is not immediately obvious as to which α value corresponds to each part of the curve

Unfortunately, the implementation of both κ-curves and $\epsilon\kappa$-curves consumed more time than was initially expected due to the issues discussed previously. However, a prototype tool was constructed in order to test a way of solving each of the above three issues.

The initial idea for designing intuitive controls was to gain insight into the psychology of curvature visualisation in humans. Most of this research however was inapplicable to designing such controls. For example, Bar and Neta [4] examine the attitudes that people have to objects based on if they are curved or sharp angled. As a result, the focus shifted to the curvature visualisation techniques that were implemented earlier in the project.

The osculating circle was experimented with in the Bézier curve system for visualising curvature. Since α values control the curvature around control points, the intuitive control for adjusting curvature requires placing an osculating circle at each control point, using it's radius as a slider to adjust the corresponding α value (see Figures 3.16 and 3.17).

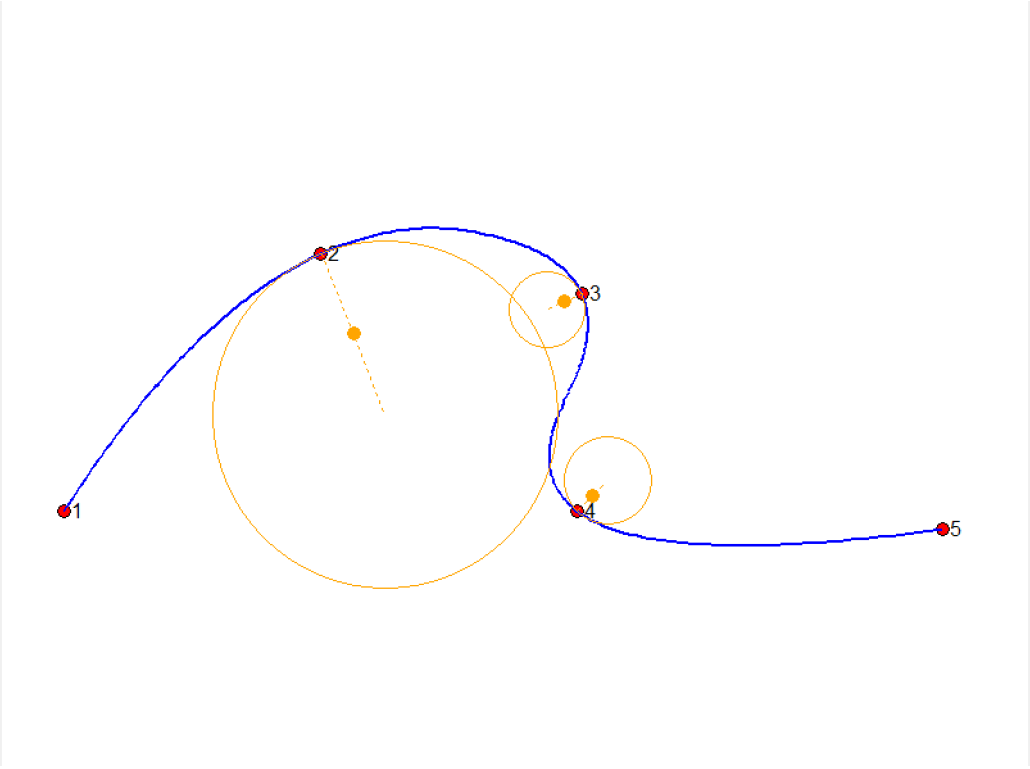


Figure 3.16: $\epsilon\kappa$-curve with prototype tool, example 1

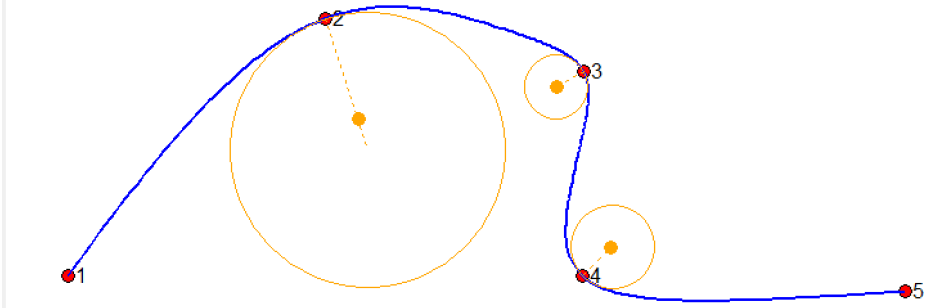


Figure 3.17: $\epsilon\kappa$-curve with prototype tool, example 2

The majority of the code used to implement the osculating circle in the Bézier curve system proved to be a good framework for developing the intuitive control in the $\epsilon\kappa$-curve system. Rather than obtaining the t value from the user's input and using this to calculate the (x, y) coordinate on the curve for which the osculating circle should be tangent to, the algorithm simply obtains the (x, y) coordinates of each user control point (excluding the end points).

While this tool improves upon the three issues mentioned above, there were some other issues that arose from this implementation. These issues will be discussed later in this report.

3.7 Testing

The main objective of this project was to create intuitive controls for adjusting the curvature of $\epsilon\kappa$-curves. As a result, it is important that the implemented $\epsilon\kappa$-curves work correctly. In order to test this, the Python implementation is compared to Salvi's Julia implementation [24], by creating open curves with similar control points in both systems.

It's important to note the accuracy of this test. Salvi's implementation does not provide (x, y) coordinates of the control points, therefore exactly replicating the same control points in each system is difficult. However, this method of testing correctness is still suitable as the curve will should be similar in both systems for similarly placed control points.

In addition, one can check that the characteristics of $\epsilon\kappa$-curves are satisfied. Again, this is more challenging than it first appears. There are situations in both systems when a point of maximum curvature appears not to be on a control point, although this may be due to how we perceive curvature. As a result, when testing correctness, parts of the curve that definitely have maximum curvature, such as sharp spikes, are of higher priority.

3.7.1 Testing correctness

3.7.2 Investing the intuitiveness of adjusting α-values for $\epsilon\kappa$-curves

User feedback was gathered in order to test the intuitiveness of the controls. Unfortunately, it was difficult to find participants that had experience with other similar tools, such as the pen tool implemented in Adobe Photoshop® [2] and Illustrator® [1]. This would have allowed for a more direct comparison of how the tools in this implementation compare to the tools currently being used by designers. However, an addition aim for this project was to make a tool that is intuitive for novice users, so the feedback received remains well-aligned with that goal.

Participants were provided with the following brief prior to interacting with the system:

> You are presented with a tool for manipulating curves. You may add, delete and move control points freely. Your task is to experiment by creating different curves and adjusting their curvature as desired. Pay particular attention to the curvature adjustment functionality and how intuitive and easy the system feels to use.

The brief was kept deliberately vague, as the user testing was intended to simulate a real-world scenario where designers have free choice over what they create. Since the aim for the project was to create intuitive controls for adjusting curvature, the brief asks participants to may particular attention to this. Note that the brief does not entail any details on how to use the system - if the system is suitably intuitive, participants should be able to discover how to manipulate the curve without guidance.

User feedback was given verbally, with notes taken manually during the sessions.

4 Results and Evaluation

4.1 Testing for correctness

As mentioned previously, testing for correctness was only somewhat precise, yet suitable. The testing attempts to place the control points in each system in the same arrangement, and evaluate any difference between curves. Note that it is only possible to compare the systems if the α value is the same for all points, since Salvi's implementation [24] does not support different α values for each control point.

Test 1 The first example to be tested was an $\epsilon\kappa$-curve defined by 5 control points, with an α value of 0.66. The results from this test are shown in Figures 4.1 and 4.2

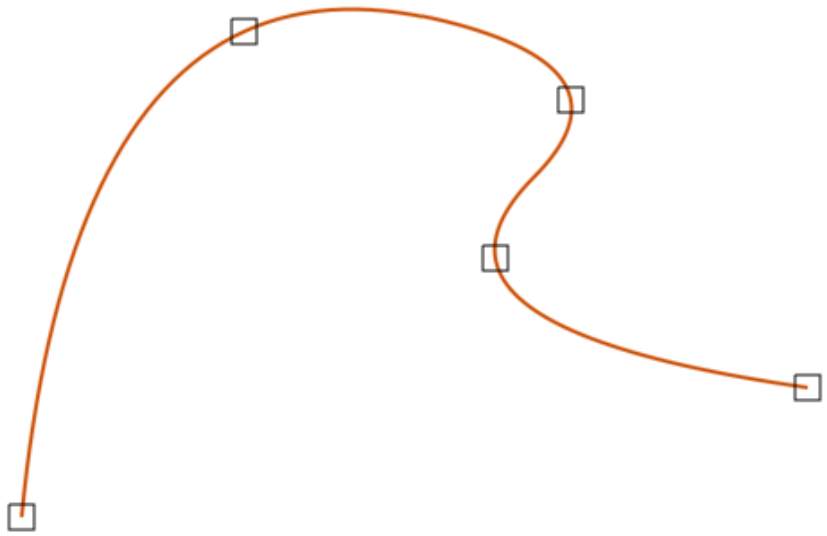


Figure 4.1: Salvi's implementation [24], 5 control points, $\alpha = 0.66$

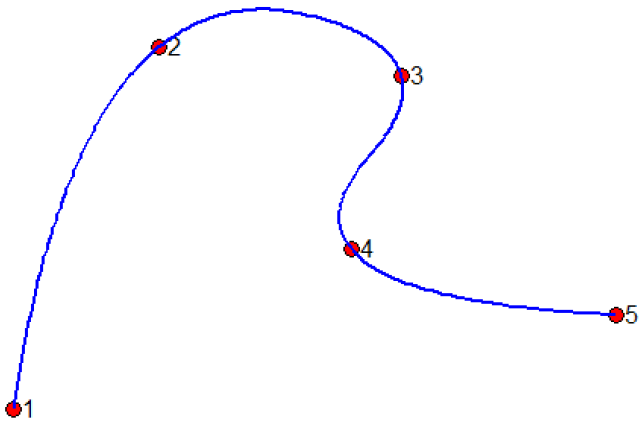


Figure 4.2: Python implementation, 5 control points, $\alpha = 0.66$

These results show an almost identical curve for the given control points. Minor discrepancies between the two curves may result from the position of the control points. For example, in Salvi's implementation, the curve appears flatter after control point 2, which may be due to the height difference between points 2 and 3. While not exact, the curves generated in both examples are mostly identical. However, note that control point 4 in each example are in slightly different places on the curve. In fact, in the Python implementation, this point is slightly off from what one may perceive to be the local maximum point of curvature.

Test 2 In order to investigate these discrepancies further, the curve in Figure 4.2 was generated again, but iterating through the algorithm 400 times, rather the 30 iterations used to generate Figure 4.2. The result of this is shown in Figure 4.3:

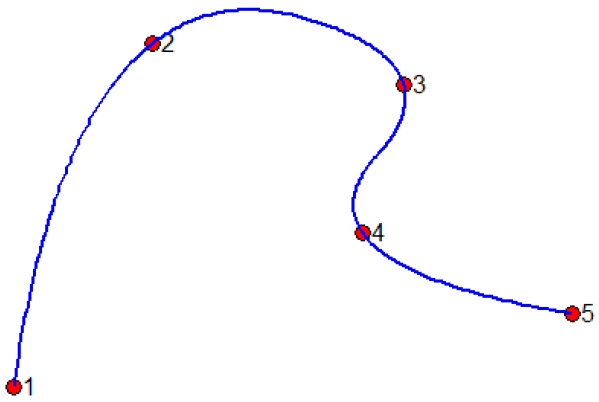


Figure 4.3: Python implementation, 5 control points, $\alpha = 0.66$, 400 iterations

Analysing this curve, it appears that control point 4 is closer to that which may be perceived as a local curvature maximum, and the curve between points 2 and 3 appears flatter. In addition, the small peak between these points appears closer to point 2, and therefore matches Salvi's implementation more accurately. However, the increase in iterations resulted in a significant increase to the computational overhead, resulting in vastly slower system. Therefore, for testing, the number of iterations will be fixed to 30.

Test 3 The next test used the same configuration of control points, but set $\alpha = 0.9$. The results are shown in Figures 4.4 and 4.5. Upon inspection, it appears that the two curves are more similar to each other than in the case where $\alpha = 0.66$. In addition, control point 4 now appears to be situated on the local curvature maximum. Looking closely at control point 3, it appears the curve is sharper in Salvi's implementation than in the Python implementation, but this again could be a result of the control points not being exactly aligned in each image.

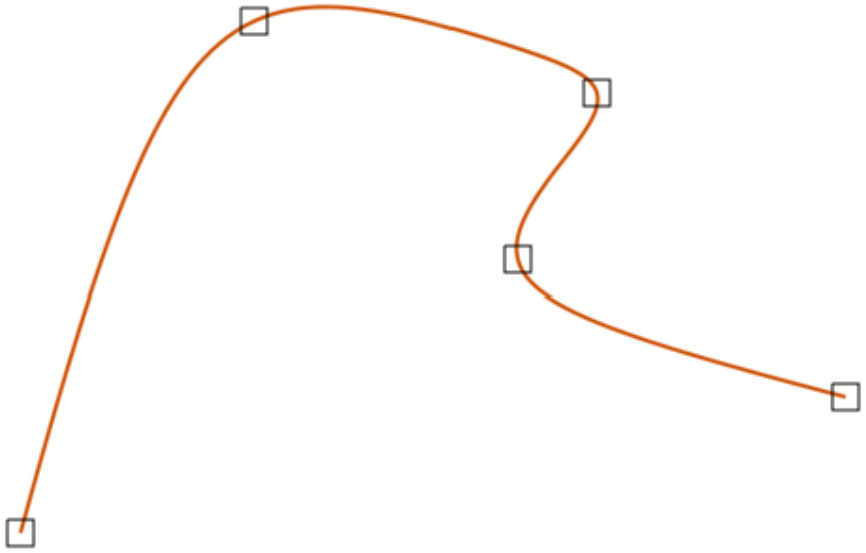


Figure 4.4: Salvi's implementation [24], 5 control points, $\alpha = 0.9$

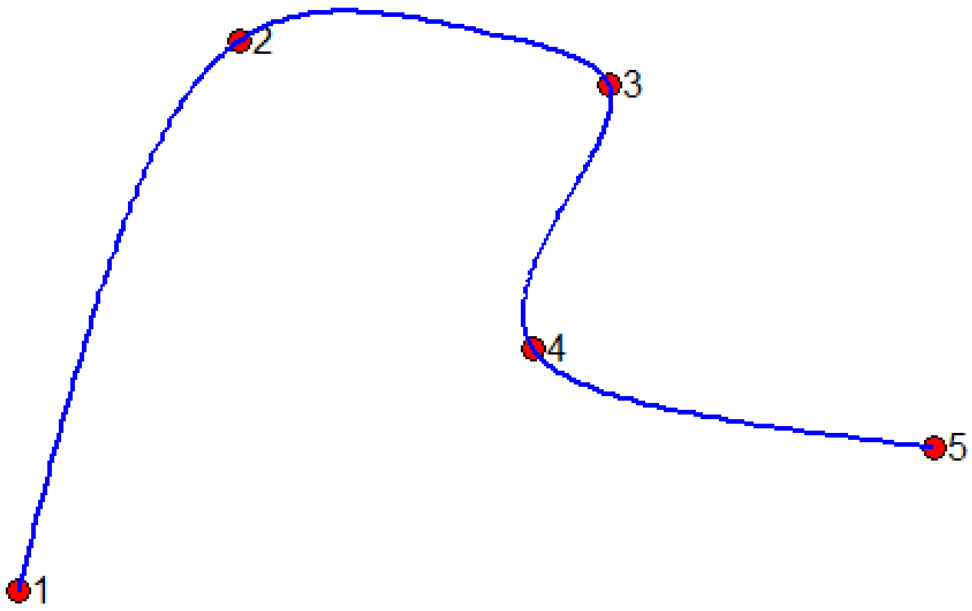


Figure 4.5: Python implementation, 5 control points, $\alpha = 0.9$

Test 4 In addition to testing multiple α values for the same curve, other tests used different numbers of control points. Test 4 used investigated a $\epsilon\kappa$-curve that used 7 control points. The curve was constructed to be more complex, but creating tighter turns and ensuring the curve crossed itself. The results can be seen in Figures 4.6 and 4.7. The value for α was originally set to 0.7, and again one can observe an almost exact match between the output of the two systems.

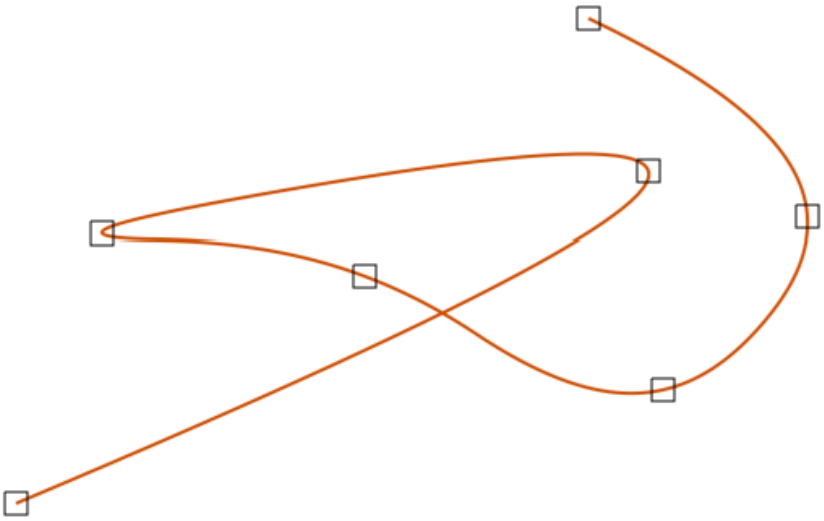


Figure 4.6: Salvi's implementation [24], 7 control points, $\alpha = 0.7$

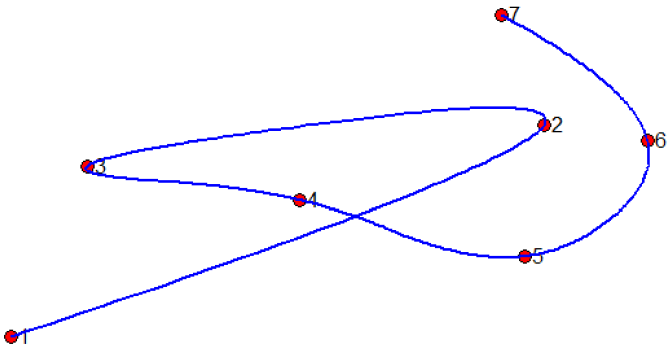


Figure 4.7: Python implementation, 7 control points, $\alpha = 0.7$

In the Python implementation, a small error occurred, with control point 2 not exactly on the point of maximum curvature. This is similar to the instance of point 4 in the previous tests, yet in this example, the point of maximum curvature is more evident. In contrast to this, increasing the value of α to 0.95 resulted in the correct alignment of the point of maximum curvature and control point 2 - a similar behaviour observed in the previous tests. As a result, the Python implementation for this curve resembled Salvi's implementation more closely (see Figures 4.8 and 4.9).

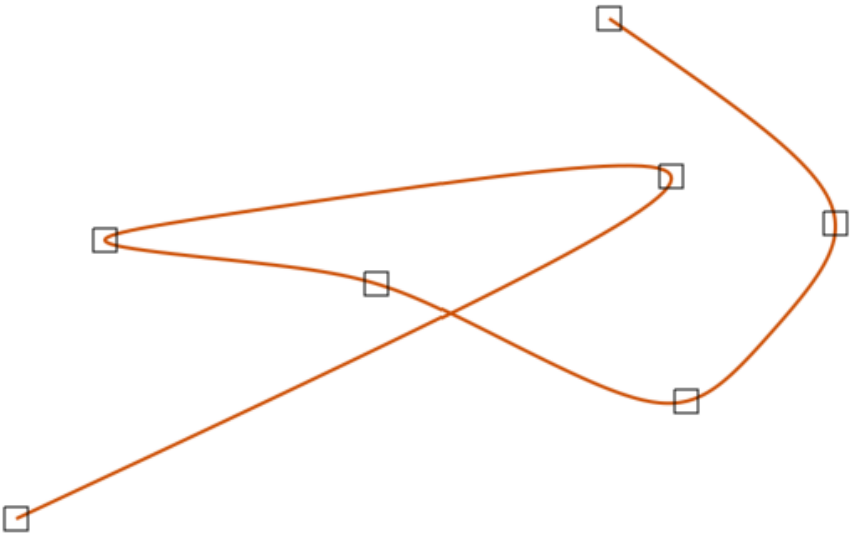


Figure 4.8: Salvi's implementation [24], 7 control points, $\alpha = 0.95$

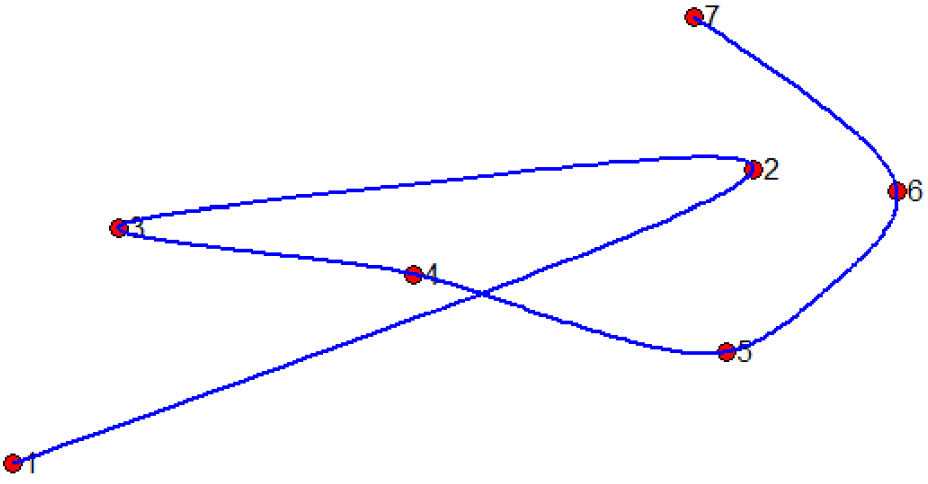


Figure 4.9: Python implementation, 7 control points, $\alpha = 0.95$

More tests were carried out in addition to ones described here, each yielding the same results. $\epsilon\kappa$-curves generated in the Python implementation with higher values of α appeared to more closely resemble to Salvi's implementation. Lower values of α sometimes resulted in points of maximum curvature occurring slightly away from a control point.

Despite this, the Python system appears to work well. Any local discrepancies between this implementation and Salvi's are minor in comparison to the global state of the outputted curve.

4.2 Investigating the intuitiveness of adjusting α-values for $\epsilon\kappa$-curves

To evaluate the intuitiveness of the interface, informal user feedback was gathered during live demonstrations and noted won manually. Four participants had no prior experience with tools similar to the one developed in this report and possessed limited mathematical background. On the other hand, one participant had experience with Inkscape, which has it's own implementation of the pen tool for manipulating curves. This participant will be referred to as Participant 5.

Following the user testing phase, several changes were made to the system in order to improve the issues that the feedback highlighted. Due to time constraints, a second round of testing was not conducted; however the revised system was evaluated through internal testing. Further user testing would be beneficial to confirm the impact of the changes made.

All participants noted that the system was slow to react to their actions. Unfortunately, time limitations meant that there was no time to optimise the program and so this feedback was predicted. Several participants realised that this was especially the case when adjusting curvature. When run on the same device, both the Python implementation and Salvi's implementation [24] can run slow, with the Python implementation suffering more greatly. When adjusting curvature in Salvi's implementation, there is a small delay before the curve updates. Note that, for example, when changing from an α value of 0.66 to 0.9, the jump is discontinuous, resulting in recalculating the curve only once. However, the Python implementation strives to display the change in curvature continuously, therefore the curve must be calculated and redrawn a considerably more number of times. The slight delay inherent in Salvi's implementation accumulates during continuous curvature adjustments, ultimately leading to this issue. Some fixes were suggested for this but not implemented post user testing due to time constraints.

While observing the participants testing the system, it was noted that several participants immediately recognised in which direction to drag the sliders to increase or decrease curvature as desired. Participant 3 initially attempted to increase the curvature but dragged the slider in the incorrect direction. This was quickly realised by the participant and any further adjustments to curvature were performed intentionally after this incident. These results suggest the controls have an intuitive nature for the prototype or implies the existence of a small learning curve if the controls are not immediately obvious to the user. Participant 2 gave the following feedback:

> **Participant 2:** "The controls for changing curvature seem right"

When prompted further on what the participant meant by 'right', they agreed that one could say the system felt natural, except for the latency previously discuss, implying that this tool has the potential to be suitably intuitive if it can respond quicker to user actions

Participant 4 decided to experiment with a large number of control points, and gave the following feedback:

> **Participant 4:** "The software is much slower and the screen becomes cluttered"

Once again, this feedback was expected. When asked to expand on this, they explained that there were too many osculating circles. Due to time constraints, the intuitive controls were designed to show at each internal control point at the same time, leading to a cluttered screen when numerous control points are used. Unfortunately, there was insufficient time to fix this issue, but a solution was considered. To mitigate the issue of computational efficiency for a large number of control points, the system could limit the number of control points, however this would subtract from a designers freedom. It would be sensible to ensure that users are unable to go to an extreme limit of control points, yet the issue noted by Participant 4 occurred when 10 control points were used. Again, another solution is proposed later in this report.

Sometimes, the sliders didn't function as intended. Participants reported jumps, rather than a continuous sliding motion. This is not intended but was improved upon after receiving the feedback. In addition, Participants 1, 2 and 5 note that for extremely curved points, the osculating circle was too small to allow for changing curvature (see control points 2 and 3 in Figure 4.10 for a replication of this issue). In addition, all participants noted that for small curvature, the osculating circle would extend beyond the visible canvas. While this alone is not a critical issue, it sometimes caused the associated curvature slider to render off-screen, preventing further adjustment.

Finally, when asked to compare this tool to the Inkscape, Participant 5 said:

> **Participant 5:** "While I wouldn't switch to using this tool straight away, I can see that it would be good for novice users if the issues are fixed"

This feedback was highly informative, as it demonstrated that at least one of the project's objectives could be successfully met, if the issues could be improved upon.

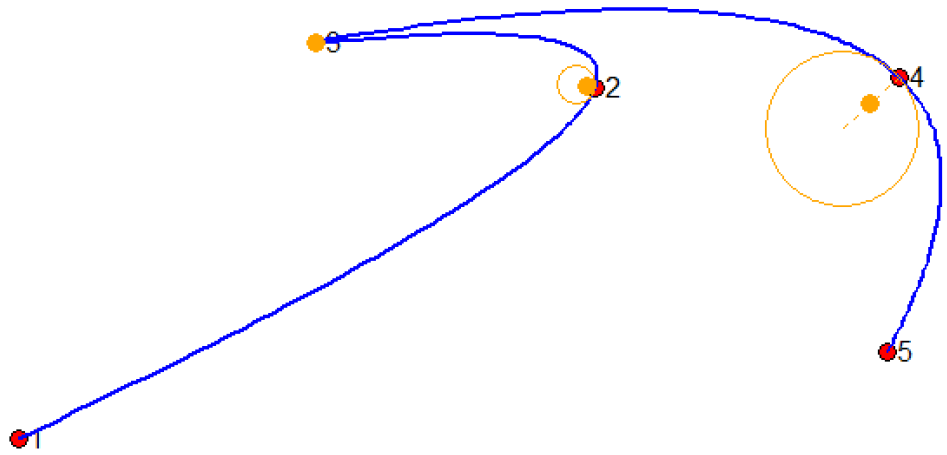


Figure 4.10: Osculating circle too small to adjust curvature

Due to the time frame of this this project, evaluation of the systems intuitiveness was solely based on qualitative user feedback. While this provided valuable insights into user experience, future work should include a structured quantitative study. Such a study could involve metrics like: time to completion for specific curves; error rates when matching curves and user ratings collected through Likert scale surveys. This would allow for a more statistical assessment of the tool's intuitiveness.

4.2.1 Post user testing changes

Taking feedback into careful consideration, a similar prototype tool was created. The osculating circles were still implemented the osculating circle for visualising curvature, but in order to fix the discontinuous jumps and large radii, the sliders were removed. Instead, to adjust curvature, one can hover their mouse inside an osculating circle and use the scroll wheel to adjust curvature. This fixes both issues, but does not fix the issue of small osculating circles. The system still had significant latency and further user testing is require to identify if this tool remains intuitive.

The scroll feature was implemented as a quick fix for some of the issues raised in the user testing. Based on the feedback received, the slider tool seemed intuitive, so keeping this feature but applying necessary improvements appears to be a better approach. Upon close inspection, the discontinuous jumps occurred because the slider was being update with each increment, thus breaking the fluid motion of the dot sliding along the radius. As a result, instead of using radii as the slider bar, lines of fixed length were implemented, each normal to the curve at the corresponding control points. When curvature is adjusted, only the curve and and osculating circle visually adapt. This meant that the system gave continuous visual outputs, but the issue of the radii being too small or large was also fixed. Figure 4.11 shows the GUI for this new implementation. Notice how the osculating circle at control point 4 would have been too small for adjusting curvature in the previous implementation, but this revised version fixes this issue. Again, further user testing is required to evaluate the intuitiveness of this new tool. Note also that the changed to fixed length slider bars makes the system appear faster. More specifically, there appears to be only minor latency when adjusting curvature.

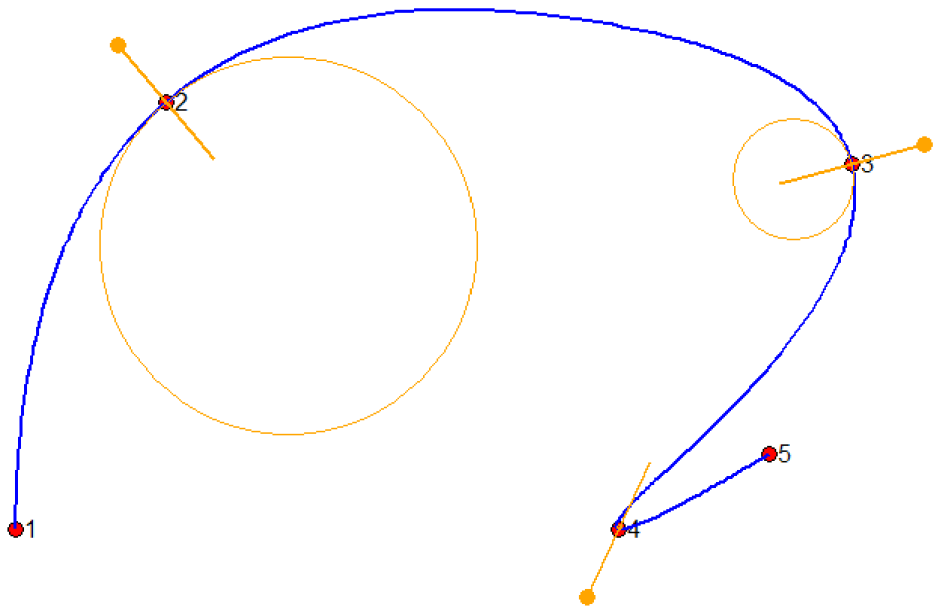


Figure 4.11: Curvature tool updated with fixed length normals

4.3 Comparison of the $\epsilon\kappa$-curves system versus similar systems

Since Salvi's implementation [24] of $\epsilon\kappa$-curves is the only other existing system, it serves as the sole basis for comparison with the Python implementation developed in this project.

Salvi's implementation has two main advantages over the Python implementation: the system is more responsive and control points can be initialised anywhere on the canvas. Ultimately, this would save designers time as they could create an initial curve by placing control points at approximate locations and than move these control points to perform minor adjustments. In addition, Salvi's implementation allows for the creation of closed and open curves. Creating closed curves requires some changes to the linear system and labelling of control points, but implementing both was not the focus of this project. Minor adjustments are required to adapt the curvature control tool for use with closed curves. In addition, as noted in the tests for correctness, the Python implementation does not always produce the correct outputs. Whilst the two implementations produced almost exactly the curves for the same set of control points, points of maximum curvature did not always lie on control points in the Python implementation. This could potentially decrease the intuitiveness of adjusting the curve.

On the other hand, this report's system provides tools and labels that allow the manipulation of the curve to be more user-friendly. For a large number of control points, one can lose track of the order of them in Salvi's system. The Python implementation labels each control point to allow users to easily keep track of the order. Another key benefit of this system over Salvi's is the adjustment of curvature. Each control point has its own α value in the interval $[0.66, 0.99]$, rather than a single α value in the set $\{2/3, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95\}$. This provides designers with more precise control over the curve, and the continuous changes in curvature aid with this further. Finally, the user is not exposed to α values when adjusting curvature. This reduces the short term memory load (from Shneiderman's Eight Golden Rules) and emphasises recognition rather than recall (from Nielsen's Usability Heuristics) [18]. Moreover, this allows the manipulation of $\epsilon\kappa$-curves to stay true to its roots in Bézier curves, where designers need not concern themselves with the underlying mathemtaics in order to produce the desired output.

5 Conclusion

5.1 Summary of Results

This project achieved its primary aim of designing an intuitive interface for adjusting curvature in $\epsilon\kappa$-curves. Through taking advantage of the history of it's formulation, namely the discovery of Bézier curves and κ-curves, this project has been able to make an already intuitive class of curve easier to control, through a useful tool for adjusting curvature. While this implementation may not be as fast as Salvi's system [24] and sometimes may not be entirely correct, it provides more freedom to users by giving them control over curvature at individual control points and allows for more 'natural' adjustment of curvature.

Implementation correctness was verified by direct comparison against existing systems, and initial usability was assessed through qualitative feedback from novice users. However, it should be noted that no structured quantitative usability study was performed, and future work should address this to statistically validate the findings.

The GUI satisfies many of the usability guidelines [18] that Salvi's implementation did not. For example, this report's system permits the easy reversal of actions by allowing users to delete the last control point, rather than a full reset of the system. However, some of these guidelines can be met further. The final system did not improve upon the issue of a cluttered GUI for a large number control points. This will be discussed in the improvements section.

5.2 Achievements

In Table 1, a list of goals and success criteria were presented that were decided at the start of the project. To evaluate the achievements of this project, one can compare these goals to the outputs presented in this report. Table 9 shows these achievements.

Goal	**Success Criteria**	**Achievements**
To understand the mathematical expressions behind Bézier curves and their role in the construction of continuous, smooth curves	1. Implement a system with adjustable control points that creates a Bézier curve.	A Bézier curve system was implemented that allowed users to add, delete and move control points freely, in real time.
To explore a variety of tools that will aid in the visualisation of curvature	1. Implement and assess the use of the osculating circle for visualising curvature for Bézier Curves. 2. Implement and assess the use of dotting the curve at set intervals of t for visualising curvature for Bézier Curves.	The osculating circle was implemented with a slider than allowed the circle to dynamically move across the curve and adjust accordingly in real time. Dots on the curve at fixed intervals of t were also implemented, with control over how many dots appeared given to the user.
To understand the mathematical expressions behind κ-curves and implement them in my own system	1. Experiment with the position of control points to ensure local maximum curvature occurs only at control points. 2. Ensure the resultant κ-curve from a set of control points is correct by comparing the output to existing systems.	κ-curves were implemented. While not mentioned in this report due to the focus on $\epsilon\kappa$-curves, the κ curves were tested to ensure local maximum curvature occurs only at control points.

Continued on next page

Table 9 (continued)

Goal	Success Criteria	Achievements
To understand the evolution step from κ-curves to $\epsilon\kappa$-curves	1. Implement $\epsilon\kappa$-curves and verify the correctness of the implementation by comparing the outputs of my system to similar existing systems.	$\epsilon\kappa$-curves were implemented and tested against Salvi's system [24]. While the outputs of the two systems did not match exactly, the differences were minor
To create an intuitive tool for adjusting curvature around control points	1. Collect user feedback from novice users to gauge the intuitiveness of adjusting curvature.	User feedback was collected for the first prototype of the curvature adjustment tool. This feedback was used to make improvements to the system but further feedback is required to gauge intuitiveness of the new implementation

Table 9: Table of achievements

5.3 Improvements to existing system

As discussed, improvements to the system were made post user testing, but some issues were not addressed. These include:

i. The inherent latency of the system

ii. The cluttered GUI when a large number of control points are used.

In order to address issue i., one could optimise the system, perhaps by finding faster algorithms for root solving or linear system solving. In addition, the user feedback detailed that latency was especially obvious when adjusting curvature. While the fixed slider bars improves upon this issue, there is still work to be done. One suggestion is that, when updating a certain α value, it is not necessary to compute all the parameters again, as the system currently does. Simply, the system is only required to re-calculate the parameters that depend upon the corresponding α value, hence decreasing computational load. To improve upon issue ii. it is possible to show only one osculating circle, since the user cannot adjust the curvature of all points simultaneously. One method of implementing this would be to include the functionality that allows the user to right click a control point in order to show the osculating circle and slider. As a result, only one osculating circle would appear on the canvas at anytime, regardless of the number of control points.

Finally, the accuracy of the system should be improved, such that local curvature maximums appear exactly on control points. The issue may result from inadequate iteration count or a subtle error in the implemented formulas. Further investigation is required to pinpoint a fix for this.

5.4 Further Work

For further work, the intuitive controls could be implemented into Salvi's system [24], since this system is more accurate, more responsive and allows for the creation of closed curves, which the tool can be easily adapted to support.

In addition, the tool can be extended to three dimensions to better allow for the modelling of real world objects. This would require significant work, but a sphere could be used in place of the osculating circle.

A path that was not explored for the creation of the intuitive tool for adjusting curvature was eccentricity. This essential states how 'uncircular' a curve is [17]. It may be possible to design a different curvature tool using this concept.

As aforementioned, future work should also include conducting a structured quantitative usability study, involving more participants. These tests could include, but are not limited to, measurement of task completion times, accuracy rate for matching given curves and usability ratings through Likert scales. Such a study would provide statistically-supported evidence for the intuitiveness of the proposed curvature adjustment tool.
Finally, for other future work, Miura et al. [22] suggests the implementation of $\epsilon\kappa$-curves in Adobe Photoshop® [2] and Illustrator® [1]. In addition, Miura et al. also mentions applying the proposed of $\epsilon\kappa$-curves to other curves, such as log-aesthetic curves [33], σ-curves [20] and τ-curves [21]. In addition, the tool proposed in this report for adjusting curvature could also be implemented to each of these classes of curves.

References

[1] Adobe Inc. Adobe illustrator, 2023. Accessed: 17 April 2025.

[2] Adobe Inc. Adobe photoshop, 2023. Accessed: 17 April 2025.

[3] Jack Baker. Convex polyhedra and graphics – determining outward normals. https://www.math.ucla.edu/~baker/149.1.02w/handouts/s_convex/node6.html, 2002. Accessed: 2025-04-24.

[4] Moshe Bar and Maital Neta. Humans prefer curved visual objects. *Psychological Science*, 17(8):645–648, 2006.

[5] Benjamin T Bertka. An introduction to bézier curves, b-splines, and tensor product surfaces with history and applications. *University of California Santa Cruz*, 2008.

[6] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. Julia: A fresh approach to numerical computing. *SIAM review*, 59(1):65–98, 2017.

[7] Wolfgang Boehm and Andreas Müller. On de casteljau's algorithm. *Computer Aided Geometric Design*, 16(7):587–605, 1999.

[8] Kuang-Hua Chang. Chapter 2 - geometric modeling. In Kuang-Hua Chang, editor, *Product Design Modeling Using CAD/CAE*, pages 39–123. Academic Press, Boston, 2014.

[9] Wen-Hui Du and Francis J.M. Schmitt. On the g1 continuity of piecewise bézier surfaces: a review with new results. *Computer-Aided Design*, 22(9):556–573, 1990.

[10] Martin Fitzpatrick. Which python gui library should you use?, 2023. Accessed: 17 April 2025.

[11] Charles R. Harris, K. Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[12] Erich Hartmann. On the curvature of curves and surfaces defined by normalforms. *Computer Aided Geometric Design*, 16(5):355–376, 1999.

[13] Freya Holmér. The beauty of bézier curves. https://www.youtube.com/watch?v=aVwxzDHniEw, 2022. YouTube video; accessed on 3 October 2024.

[14] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[15] Lumen Learning. Components of the acceleration vector. https://courses.lumenlearning.com/calculus3/chapter/components-of-the-acceleration-vector/, n.d. Accessed: 2025-04-22.

[16] Fredrik Lundh. An introduction to tkinter. *URL: www. pythonware. com/library/tkinter/introduction/index. htm*, 1999.

[17] MathsIsFun. Eccentricity, 2024. Accessed: 19 April 2025.

[18] Fourcan Karim Mazumder and Utpal Kanti Das. Usability guidelines for usable user interface. *International Journal of Research in Engineering and Technology*, 3(9):79–82, 2014.

[19] Aaron Meurer, Christopher P Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K Moore, Sartaj Singh, et al. Sympy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, 2017.

[20] Kenjiro T. Miura, Shingo Suzuki, R. U. Gobithaasan, Shin Usuki, Jun-ichi Inoguchi, Masaru Sato, Kenji Kajiwara, and Yoshinobu Shimizu. Fairness metric of plane curves defined with similarity geometry invariants. *Computer-Aided Design and Applications*, 15(2):256–263, 2018.

[21] Kenjiro T. Miura, Shingo Suzuki, Shin Usuki, and R. U. Gobithaasan. τ-curve: Introduction of cusps to aesthetic curves. *Journal of Computational Design and Engineering*, 7(2):155–164, 2020.

[22] K.T. Miura, R.U. Gobithaasan, P. Salvi, et al. $\epsilon\kappa$-curves: Controlled local curvature extrema. *The Visual Computer*, 38:2723–2738, 2022.

[23] Jakob Nielsen. Enhancing the explanatory power of usability heuristics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 152–158. ACM, 1994.

[24] P. Salvi. Implementation of κ-curves. https://github.com/salvipeter/ekcurves/, 2020. Accessed: 7 January 2024.

[25] Péter Salvi. $\epsilon\kappa$-curves: Controlled local curvature extrema. https://github.com/salvipeter/ekcurves/commit/557cbb7bdd329c2dec7cf401cc35ba4f5691e531, 2020. GitHub repository, commit `557cbb7`; accessed 17 Apr. 2025.

[26] SciPy Community. *scipy.optimize.brentq — SciPy v1.11.3 Manual*, 2023. Accessed: 17 April 2025.

[27] Codecademy Team. Best programming languages for math, 2022. Accessed: 17 April 2025.

[28] IONOS Editorial Team. Python vs. matlab – a comparison, 2023. Accessed: 17 April 2025.

[29] Labdeck Team. Comparing python gui libraries, 2023. Accessed: 17 April 2025.

[30] The Maxima Development Team. *Maxima, a Computer Algebra System.* Maxima Project, 2020.

[31] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual.* CreateSpace, Scotts Valley, CA, 2009.

[32] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[33] Dan Wang, R. U. Gobithaasan, Tadatoshi Sekine, Shin Usuki, and Kenjiro T. Miura. Interpolation of point sequences with extremum of curvature by log-aesthetic curves with g^2 continuity. *Computer-Aided Design and Applications*, 18(2):399–410, 2021.

[34] Eric W. Weisstein. Osculating circle. https://mathworld.wolfram.com/OsculatingCircle.html. From MathWorld–A Wolfram Web Resource. Accessed: 3 October 2024.

[35] Zhipei Yan. *Control of Curvature Extrema in Curve Modeling.* PhD thesis, Texas A&M University, 2021.

[36] Zhipei Yan, Stephen Schiller, Gregg Wilensky, Nathan Carr, and Scott Schaefer. κ-curves: Interpolation at local maximum curvature. *ACM Transactions on Graphics (TOG)*, 36(4):1–7, 2017.